تعلم لغة DART

ملخص مبسط جدا لفهم أساسيات لغة DART



Dart

تم جمع هذا المعلومات من كورس لغة Dart للمهندس وائل أبو حمزة من قناته على اليوتيوب

جدول المحتويات

Syntax 3
Datatype & variable 4
Arithmetic Operators 10
quality and Relational Operators 12
Type test Operators And Assignment Operators 15
Logical Operators 19
comment 22
if else statement 23
Switch case 29
For loop 31
While Loop 36
Do While Loop 37
break 38
continue 40
Number 42
String Method 50
List 57
MAP 70
Dyanmic Varibles 74
final and const 75
Set 76
Convert between 77
List Map Set 77
Function 79
Function types 83
Function scope 86
import 89
المسارات 90
Runes 92

Adel AboBaker

Assert 94
Max And Min 95
DataType 96
Iterator And Iterable 106
Map Method 108
Try And Catch 110
shorthand if 111
OPP 112
Reset Variable 114
Setter And Getter 117
Static 119
Cascade operator 121
Constructor 122
inheritance 123
Override 124
Mult-level inherate 125
Super Class 127
Abstract class 128
implements class 129
Collection 131
Enum 132
Regular expressions 134
Private 137
Mixin 139
Null Safety 141

Adel AboBaker

Syntax

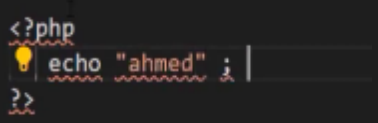
هو مجموعة القواعد لكتابة برنامج ما

الامر مشابه تماما للغات التي يتحدث بها البشر مثل الإنكليزية والعربية على سبيل المثال اذا اردنا قول القاء التحية على احد ما نقول بلغة العربية مرحبا وبلغة الإنكليزية Hello الكلمتين تختلفان بلفظ وطريقة الكتابة ولكن الكلمتين تؤديان نفس المعنى وهو الترحيب.

كذلك الامر بلنسبة للغات البرمجة يوجد العديد لغات البرمجة



في هذا المثال نقوم بطباعة كلمة ahmed بلغة ال DART



وبهذه الصورة سوف نقوم بطباعة كلمة احمد بلغة Php

وخرج البرنامج سيكون نفسه مع اختلاف طريقة الكتابة واختلاف لغة البرمجة أيضا .

Datatype & variable

هي أسماء تخزن في ذاكرة الحاسوب وتأخذ قيم ويوجد عدة أنواع لل variable

اول نوع من ال variable هو ال String.

من الضروري ان نقوم بوضع النص داخل المتغير String ضمن

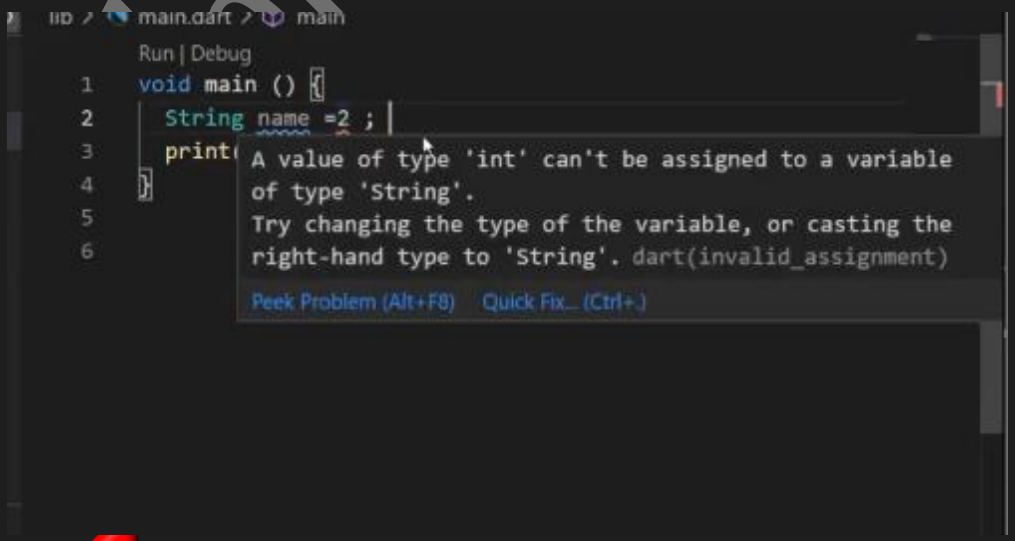
(دزبل كوتشن " ") او(سنجل كوتشن ' ')

```
String name = "wael";
print(name)
```

نقوم بتخزين نص في داخله text تستطيع اختيار اسم المتغير الذي تريده ولكن بشرط ان لا يحوي الاسم على رموز او ان لا يبدأ اسم المتغير برقم .

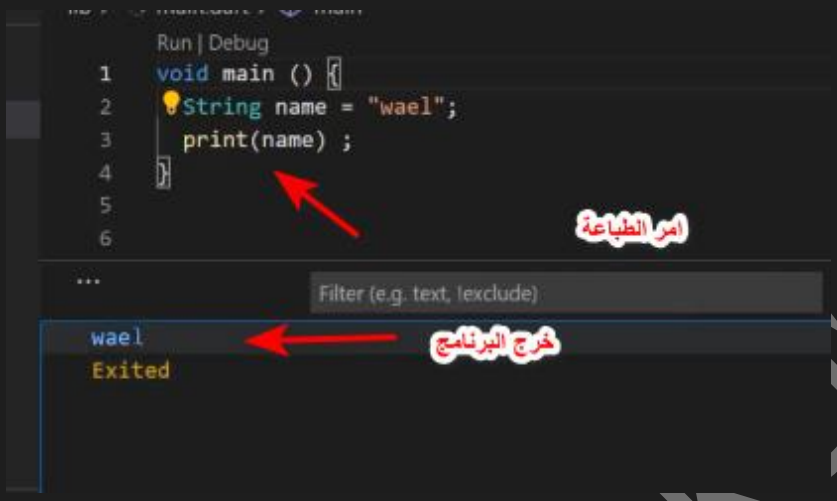


لا نستطيع تخزين قيمة من نوع عدد في المتغير من نوع String

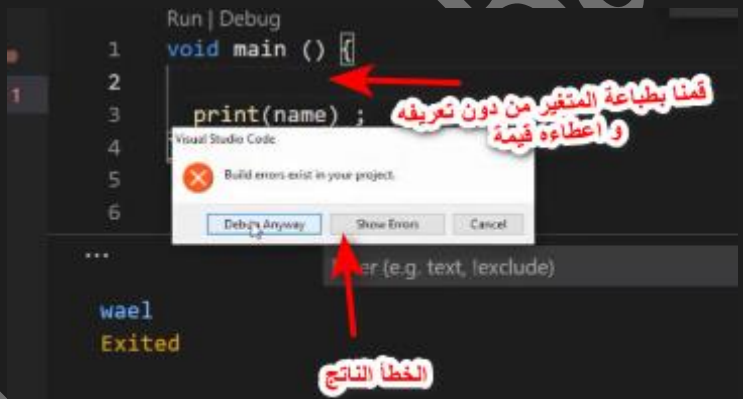


Wael Baker

اذا قمنا كتابة المتغير داخل امر الطباعة سوف يقوم البرنامج بطباعة قيمة المتغير من دون أي مشاكل كما في الصورة بلاسفل :



اذا لم نقوم بتعريف متغير لن يعمل البرنامج وسوف يظهر لنا خطأ

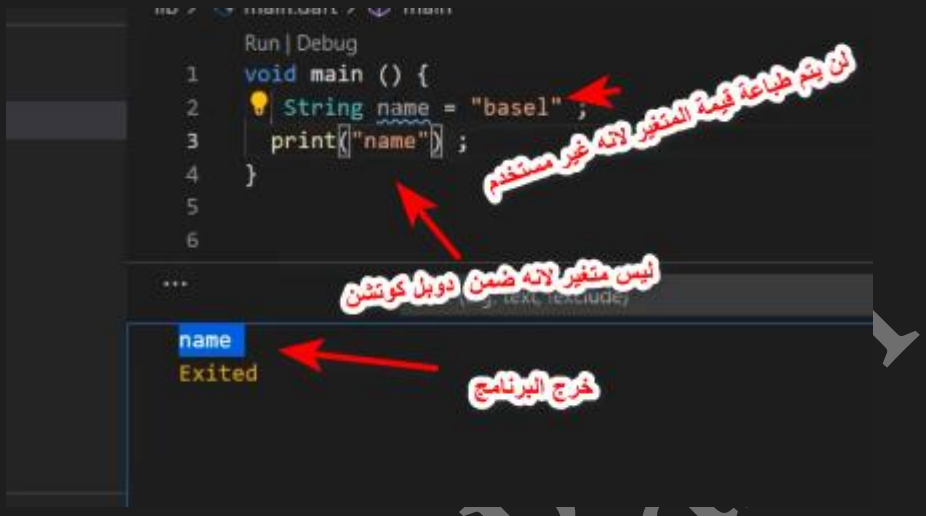


اذا اردنا وضع المتغير ضمن " " سوف يتم تعرف عليه على انه نص text

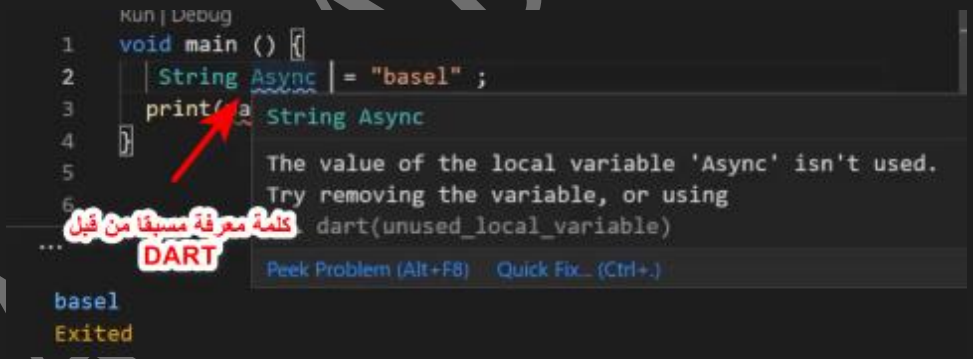
واردنا لجهاز الحاسوب ان يتعرف عليه على انه متغير وهو ضمن الدوبل كوتشين فقط نقوم بوضع علامة او رمز دولار ساين $ قبل اسم المتغير



اذا قمنا بطباعة اسم المتغير من دون وضع علامة $ قبل اسم المتغير وف يراه الحاسوب على انه نص عادي وليس متغير يحمل داخله قيمة وسوف يقوم بطباعة اسم المتغير فقط كما هو واضح في الصورة فب الأسفل :

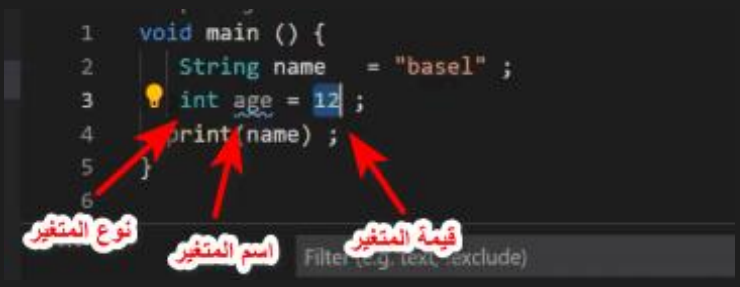


يوجد في لغة ال Dart مجموعة من الكلمات الأساسية التي لا يمكن ان نقوم بأستخدامها في برنامجنا وهي عبارة عن كلمات معرفة مسبقا في اللغة نستطيع الوصول الها ومعرفتها من خلال صندوق إرشادات المدرب .

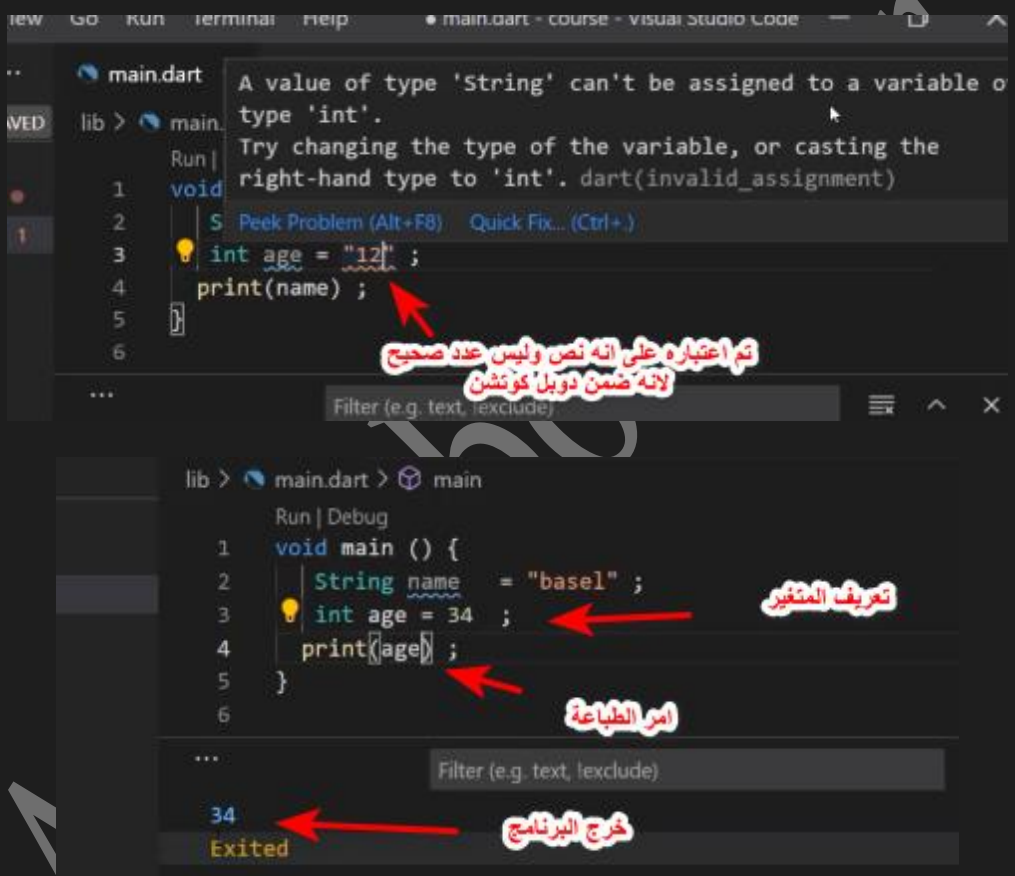


يوجد نوع ثاني من المتغيرات وهو INT

نستطيع من خلاله تخزين متغيرات الأرقام الصحيح فقط لا غير مثل (٩٠، ١ ،٩٠) أي ان الأرقام التي لا تحوي على فاصلة عشرية .

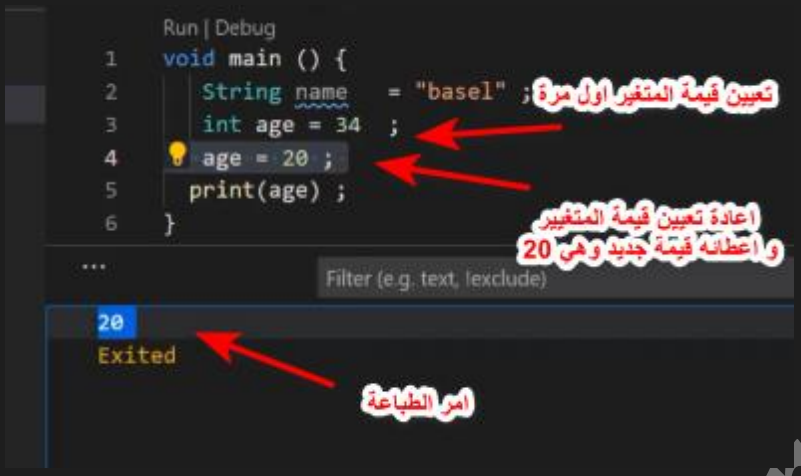


لا يجب وضع الرقم ضمن دوبل كوتشن وهو في متغير من نوع INT لان سيعتبره نص من نوع String .



لا نستطيع تعريف المتغير اكثر من مرة في البرنامج يتم تعيينه مرة واحدة فقط .

يمكننا إعادة تعيين ((قيمة المتغييرات))



لان محرر الاكواد في البرمجة يقوم بتنفيذ السطور البرمجية سطر سطر.

لا نستطيع طباعة قيمة اكثر من متغير في نفس الوقت من دون تعريف لحاسوب والا سوف يعتبره متغير واحد كما في الصورة في الأسفل :



بوضع علامة دوبل كوتشن قبل كل متغير نستطيع تعريف الحاسوب اننا نقوم بطباعة متغيرين في نفس الوقت .



نوع الثالث من المتغيرات وهو double

نستطيع من خلاله تخزين الأرقام العشرية .

نسطيع أيضا تخزين الأرقام الصحيحة ضمن متغير double ولكن لا نستطيع تخزين الأرقام العشرية ضمن المتغير int .



نوع الرابع من المتغيرات وهو bool يرجع قيمة true او false.

Adel AboBaker

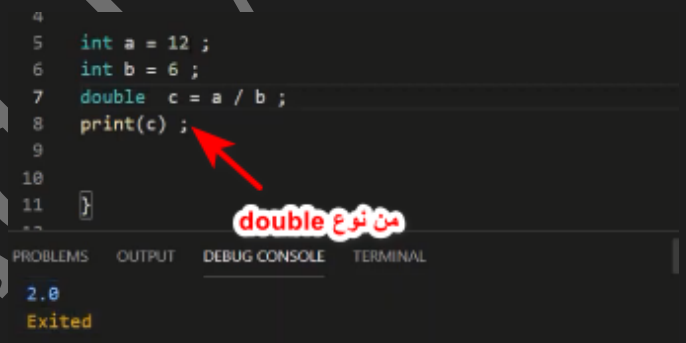
Arithmetic Operators

نستطيع اجراء العمليات لحسابية على المتغيرات مثل ما هو واضح في الصورة في الأسفل



كما هو معروف في الرياضيات يوجد العديد من العمليات مثل الطرح والجمع والقسمة والضرب والتي نستطيع ان نستخدمها قي برنامجنا الخاص لاجراء وظيفة معينة .

فقط في القسمة يجب ان نستخدم متغير من نوع double لان القسمة ينتج عنها اعداد عشرية كما هو موضح في لصورة في الأسفل .

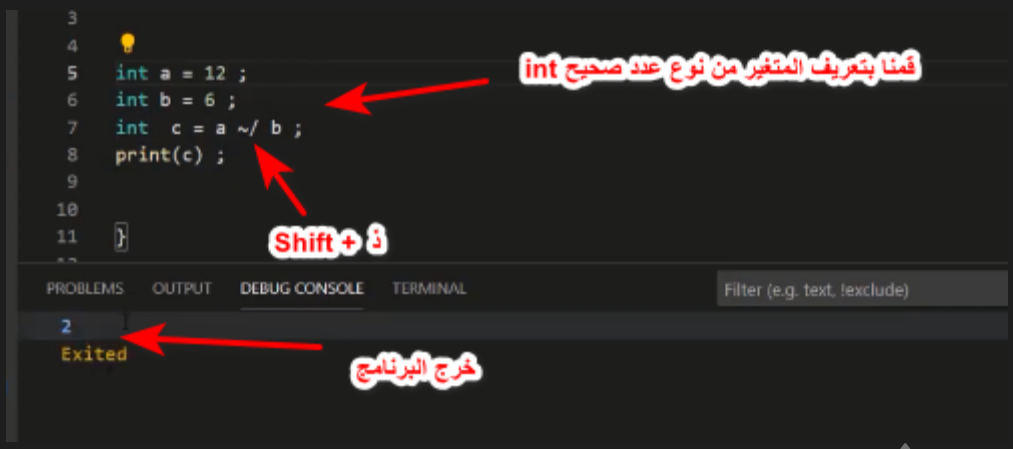


نستطيع ان نقوم بتحويل خرج ناتج القسمة لعدد صحيح من خلال

وضع علامة ~ قبل علامة القسمة بلغة الإنكليزية كما موضح في الصورة بلاسفل

:

Ad Abo Ba



ومن الإشارات المعروفة في الرياضيات وهي إشارة باقي القسمة %

كما هو موضح في المثال في الصورة بلاسفل :



وأيضا من العمليات كما هو موضح في الصورة في الأسفل :

تقوم بأضافة ١ الى خرج البرنامج ويوجد عكسا وهي c-- تقوم بطرح ١ من خرج البرنامج .



Adel Ker

quality and Relational Operators

عمليات المساوة و المقارنة تقوم بأرجاع قيمة true او false

< اكبر

> اصغر

= يساوي

=! لا يساوي



في المثال الموضح أعلاه اذا كانت قيمة المتغير a تساوي قيمة المتغير b سوف يقوم بأرجاع قيمة true.

والا اذا كانت قيمة المتغير a لا تساوي قيمة المتغير b سوف يقوم بأرجاع قيمة false كما هو واضح في الصورة في الاسفل.

Ade ker

```
1   void main(){
2
3   int a = 10 ;
4   int b = 20 ;
5
6   print(a == b) ;
7
8   }
9
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL
false
Exited
```
خرج البرنامج

اذا كانت قيمة المتغير a اكبر من قيمة المتغير b سوف يقوم بأرجاع قيمة true.

الا اذا كانت قيمة المتغير a اصغر من قيمة المتغير b سوف يقوم بأرجاع قيمة false.

و اذا كانت قيمة المتغير a تساوي قيمة المتغير b سوف يقوم بأرجاع قيمة false.

كما هو موضح في الصورة في الأسفل :

```
Run | Debug
1   void main(){
2
3   int a = 30 ;
4   int b = 20 ;
5
6   print(a > b) ;
7
8   }
9
```
عملية مقارنة

اشارة الاكبر

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   Filter (e.g
true
Exited
```
خرج البرنامج

وتوجد حالات اخرة للمقارنة على سبيل المثال اذا كانت قيمة المتغير a اكبر او تساوي قيمة المتغير b سوف يقوم بأرجاع قيمة true.

كما هو موضح في المثال في الصورة بلاسفل :

A'ShoBaker

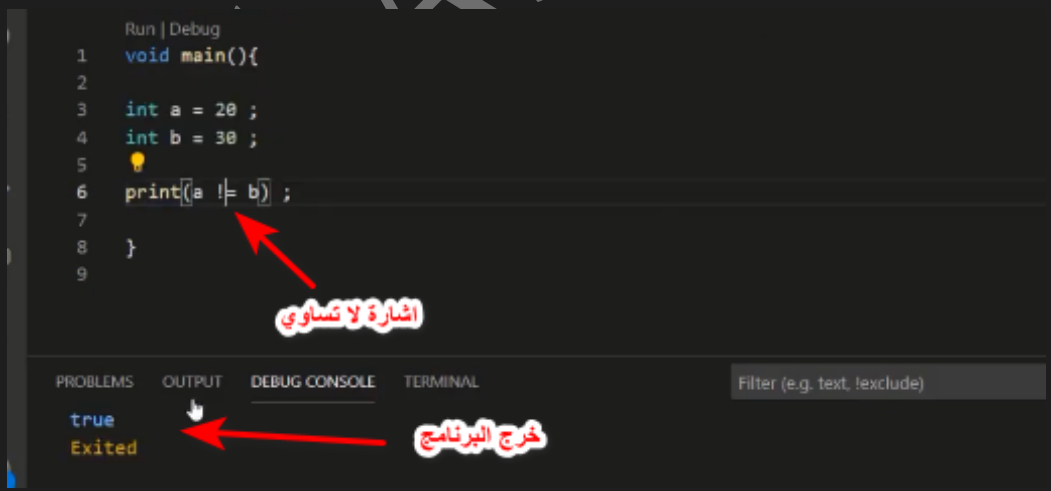


اخر اشارة لدينا في عمليات المقارنة وهي إشارة لا يساوي =!

اذا كانت قيمة المتغير a لا تساوي قيمة المتغير b سوف يقوم بأرجاع قيمة true.

اذا كانت قيمة المتغير a تساوي قيمة المتغير b سوف يقوم بأرجاع قيمة false.

كما هو موضح في الصورة في الأسفل :



AboBaker

Type test Operators And Assignment Operators

عمليات الاختبار تستخدم هذه العلميات من اجل معرفة نوع المتغير وترجع قيمة اما true او false.

Is هل يساوي

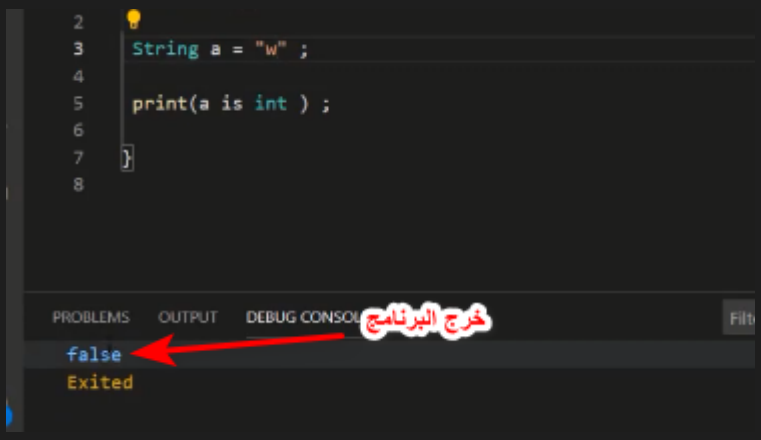
Is! هل لا يساوي

كما هو موضح في الصورة في الأسفل يوجد عملية اختبار على المتغير كأننا نقوم بتوجيه سؤال للحاسوب مضمونه هل المتغير a من نوع int كما هو موضح في المثال في لاسفل يكون الخرج true لان المتغير من نوع int لان المتغير قيمته عدد صحيح هو العدد ١٠ كما هو واضح في لصورة في الأسفل :



ولو ان المتغير قيمته ليست من نوع int سوف يقوم البرنامج بأرجاع قيمة false كما هو موضح في الصورة في الأسفل :

Ard AboBaker



كما يوجد مثال على نوع الثاني من عمليات الاختبار الا وهو !is



في المثال الموضح أعلاه نقوم بتوجيه سؤال للحاسوب مضمونه هل المتغير a ليس من نوع int كما هو موضح في المثال يكون الخرج true لان المتغير ليس من نوع int لان المتغير قيمته text.

AdelAl

في المثال الموضح ادناه لقد قمنا بأعادة تعيين قيمة المتغير a ولكن عند تشغيل البرنامج نتفاجئ ان قيمة المتغير a لم يتم اعدة تعيينها وفي الدروس السابقة تكلمنا ان الحاسوب يقوم بتنفيذ الاكواد سطر سطر فما هو سبب عدم إعادة التعيين



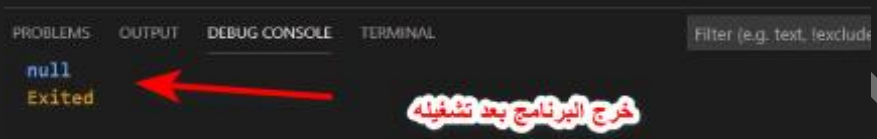
وسبب في هذا الامر وهو إشارة مساوة =?? في حال كان المتغير لا يحتوي بداخله على أي قيمة وهذه الإشارة تعني ان اذا كان المتغير ليس له قيمة قم بأسناد قيمة له والقيمة التي سوف يقوم بأسنادها هي نفسها القيمة التي قمنا بأدخالها للمتغير وهي ٢٠ كما هو موضح في الصورة في الأسفل :



Adel Abo

Null تعني ان المتغير معرف ولكن لا يحوي بداخله أي قيمة





ومن الإشارات المهمة + a

اسم المتغير ثم وضع إشارة العملية الحسابية ثم قيمة مراد اضافتها الى المتغير

تستطيع استخدام جميع العمليات الحسابية من ضرب وقسمة وطرح وجمع

ولا ننسا ان نقوم بتغير نوع المتغير في حال الفسمة الى double لان ناتج القسمة ينتج عنه اعداد عشرية .



Adel AboBaker

Logical Operators

عمليات المقارنة وتقوم أيضا بأرجاع قيمة من نوع bool أي ان الخرج سيكون اما true او false.

&& And

|| Or

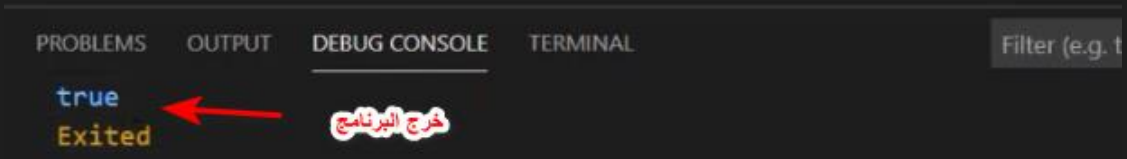
! Not

اول إشارة من عمليات المقارنة الا وهي ال AND (و)

نقوم بلضغط على زر SHIFT + 7

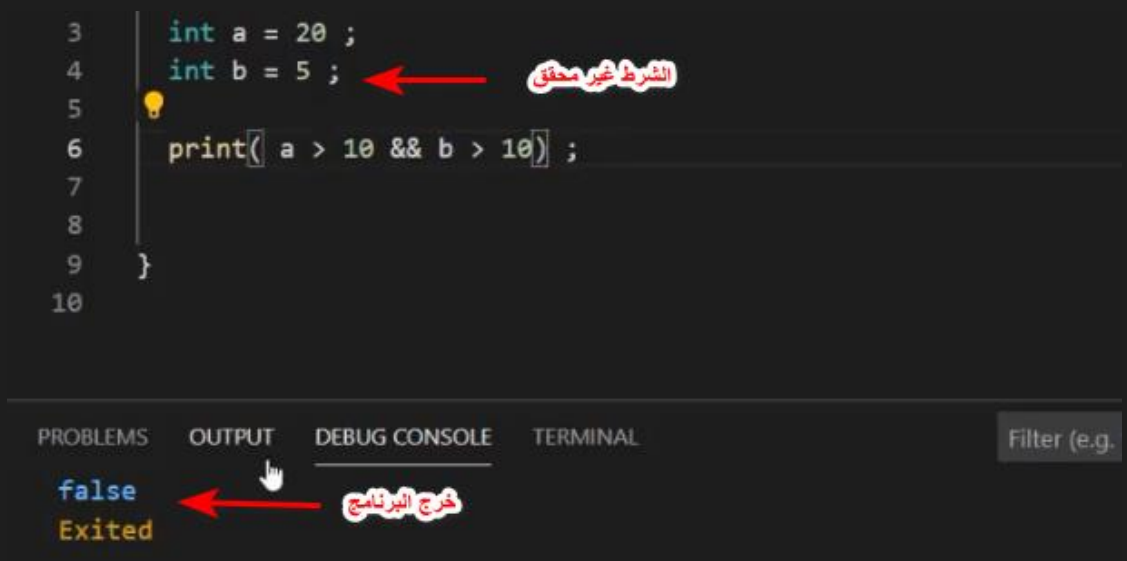
كما هو موضح في المثال في الأسفل فأن الشرط محقق بحيث ان المتغير a و متغير b اكبر من ١٠ فأن خرج البرنامج true .





عند استخدام AND في حال اختلال احد الشرطين سيرجع البرنامج قيمة False كما هو موضح في الصورة في الأسفل :

hoBaker

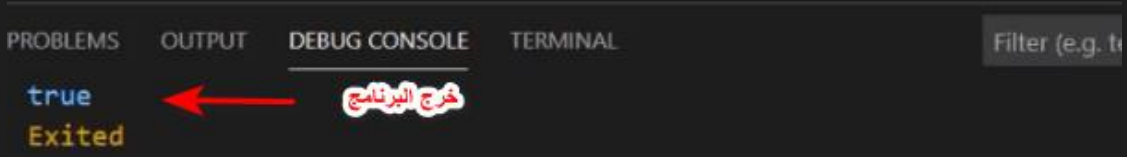


الإشارة الثاني من علميات المقارنة الا وهي OR (او)

نقوم بلضغط على زر \ + Shift

حتى يرجع البرنامج قيمة true يوجد عدة حالات اما ان يكون طرفين الشرط محقق او احد الطرفين محقق كما في المثال الموضح ادناه :





حتى يرجع البرنامج قيمة false يجب اختلال طرفين أي ان المتغير a و b
قيمتهم ليست اكبر من ١٠ كما هو واضح في المثال في الأسفل :



الإشارة الثالثة وهي NOT! وتقوم بعكس الحالة

نقوم بلضغط على زر 1 + Shift

لنقوم بتوضيح المثال الذي في الأسفل لدينا متغيرين a و b وان قيمتهم اكبر من ١٠ والشرط محقق ويقوم بأرجاع قيمة true في البرنامج ولكن لوجود إشارة not تقوم بعكس الحالة من true الى false وفي حال ارجع البرنامج قيمة false تقوم not بعكس الحالة وطباعة true في خرج البرنامج .



Adel Aboba

comment

التعليق لا يستطيع محرر الاكواد او الحاسوب قراءة او تنفيذ التعليق ضمن البرنامج.

يستخدم التعليق من اجل تسهيل قراءة الاكواد او مثل وضع عناوين على مجموع من الاسطر البرمجية لتوضيح او شرح مهمة هذه الاكواد في البرنامج .

توجد التعليقات في كل لغات البرمجة .

تستطيع كتابة التعليق اما بلضغط على زر / + Ctrl

أو بكتابة // قبل الكود

او بكتابة /* comment */

كما موضح في المثال ادناه :



Ahmed AboBaker

if else statement

اذا كان الشرط داخل if محقق true سوق يقوم البرنامج بتنفيذ الاكواد التي بداخل if.



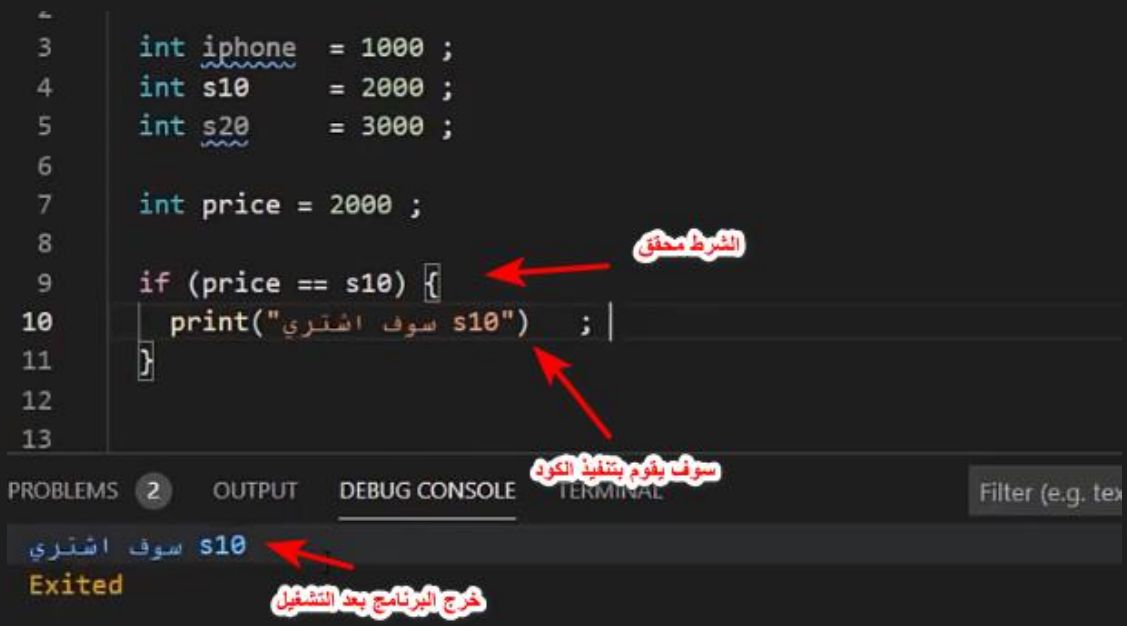
اذا كان الشرط داخل if غير محقق false لن يتم تنفيذ ما داخل اقواس الدالة if.



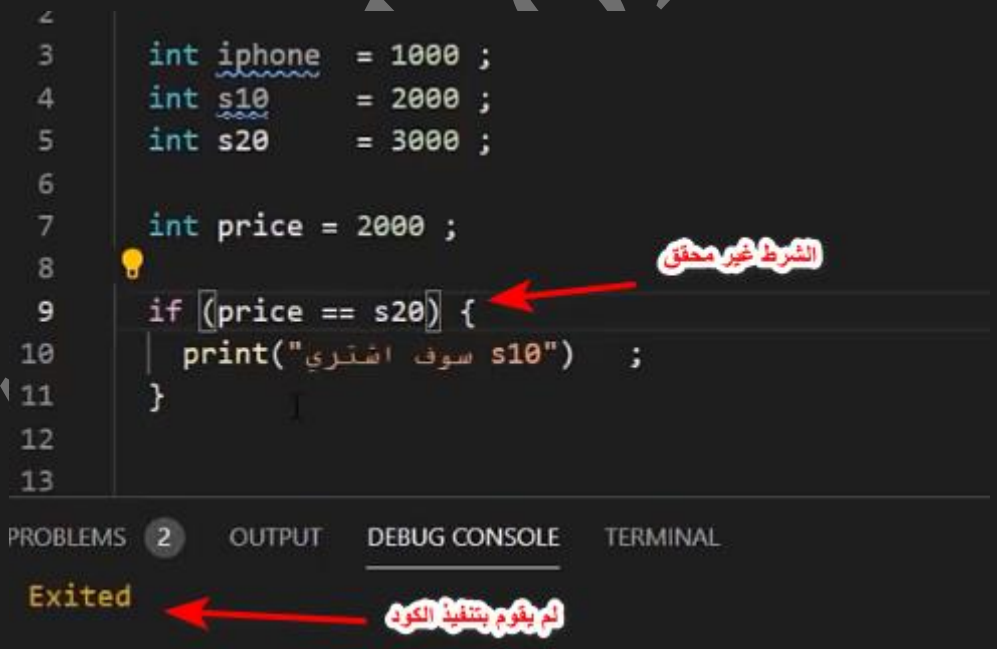
في المثال الذي لدينا في الأسفل ثلاث متغيرات لهم قيم مختلفة من نوع int وقد قمنا بلاستعانة بدالة if الشرطية بتنفيذ الكود في حال تحقق الشرط داخل if سوف يقوم الحاسوب بتنفيذ الاكواد التي بداخل if كما نلاحظ ان price وهي قيمة المال التي معنا تساوي ٢٠٠٠ وان علينا الاختيار بين هذه الأجهزة الثلاث فنقوم بكتابة الشرط اذا كان s10 == price قم بطباعة (" سوف اشتري s10 ")

وبلفعل الشرط محقق لان price == 2000 وقيمة s10 == 2000

كما هو موضع في المثال في الأسفل :



في حال كان الشرط غير محقق false لن يقوم بتنفيد الكود داخل if لان price لا تساوي قيمة S20 كما هو واضح في الصورة في الأسفل :



لدينا مثال اخر ...

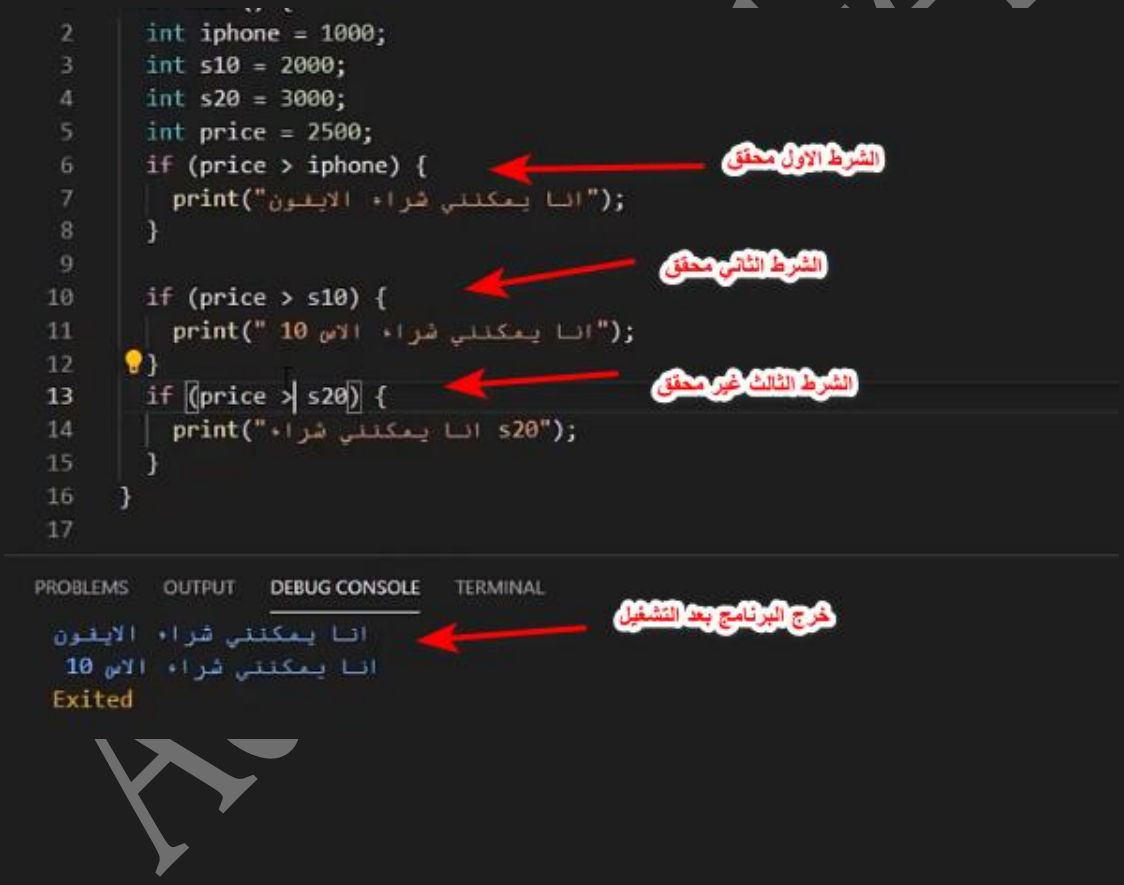


في هذا المثال لدينا ثلاث دالات شرطية سيقوم الحاسوب بفحص كل دالة في حال كان محقق سيقوم بتنفيذ ما داخلها ونجد ان دالة الثانية هي محققة كما هو ظاهر في خرج البرنامج في الأسفل :



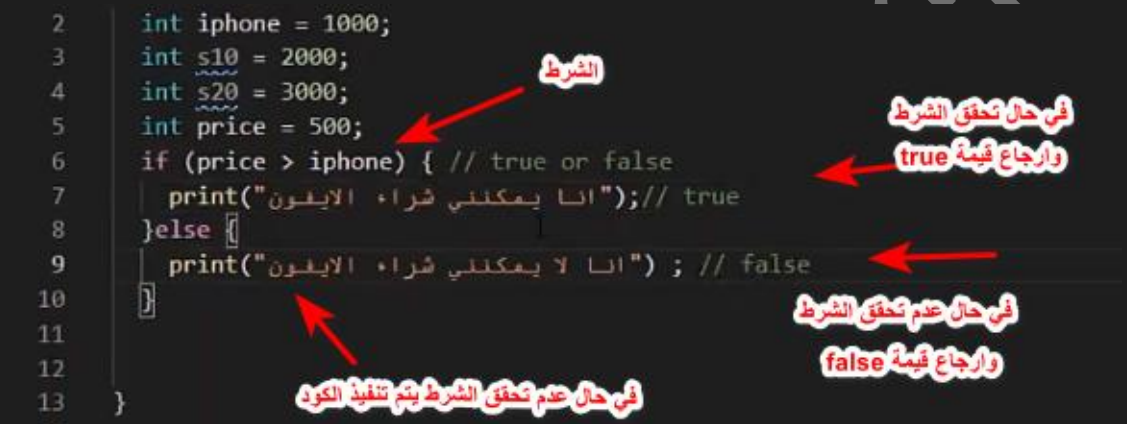
أيضا مثال اخر لدينا

نلاحظ ان الشرط الأول وثاني محقق لان قيمة price اكبر من قيمة iphone و قيمة S10 ونلاحظ ان قيمة S20 في الشرط الثالث غير محققة لانها لاتساوي قيمة Price بهذا سوف يقوم الحاسوب بتنفيذ الشرط الأول والثاني فقط كما هو واضح في المثال في الأسفل :



لدينا أيضا مثال عن else وتعني و (الا في حال عدم تحقق الشرط وارجاع قيمة false يقوم الحاسوب بتنفيذ ما داخل دالة else .

الان لنقوم بتوضيح المثال الذي بلاسفل نلاحظ ان قيمة price هي اقل من قيمة iphone ففي الدالة if سيقوم البرنامج بأرجاع قيمة false لانها غير محققة ولأنها غير محققة ولوجود ال else سوف يقوم البرنامج بتنفيذ ما بداخلها كما هو واضح في الأسفل .





نلاحظ في المثال في الأسفل ان قيمة price هي اكبر من قيمة جيمع المتغيرات الموجودة في البرنامج ومع هذا فأن الشرط الثاني محقق ومع ذلك لم يتم تنفيذه وطباعته لماذا ..؟؟

كما ذكرنا سابقا ان ان الدالة else لا تعمل الا في حال عدم تحقق الشرط الأول وارجاع قيمة false ولكن هنا الشرط الأول تحقق وقام بأرجاع قيمة true هنا سيقوم البرنامج بتنفيذ الشرط الأول فقط دون النظر الى الشرط الثاني بمعنى ان تقوم بأعطاء عدة احتمالات للحاسوب في حال تحقق الاحتمال الأول فأنه لن يقوم بلنظر الى باقي الاحتمالات لان الاحتمال الأول قد تحقق فعلا كما هو واضح في الصورة في الأسفل .

```
2   int iphone = 1000;
3   int s10 = 2000;
4   int s20 = 3000;
5   int price = 3500;
6   if (price > iphone) {
7     // true or false
8     print("انا يمكنني شراء الايفون"); // true
9   } else if (price > s20) {
10    print("انا يمكنني شراء الاس 20"); //
11  } else {
12    print("انا لا يمكنني شراء الايفون"); // false
13  }
14 }
15
```

الشرط الأول محقق

الشرط الثاني محقق

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL
انا يمكنني شراء الايفون
Exited
```

ملاحظة : يمكنك وضع اكثر من شرط ضمن دالة IF الشرطية

Switch case

تؤدي دالة Switch cace نفس عمل الدالة if else تماما ولكن فقط تختلف بطريقة الكتابة مثل ما هو واضح في المثال في الأسفل :



نجد في المثال هذا ان قيمة price تساوي ١٠٠٠ وان قيمة الحالة الأولى في البرنامج في أيضا ١٠٠٠ هذا يعني ان البرنامج سيقوم بتنفيذ الحالة الأولى .



في نفس المثال لو قمنا بأعطاء قيمة للمتغير price =3000 في هذه الحالة لن تتحقق الحالة الأولى وثانية وستقوم بأرجاع قيمة false ومن ثم سيقوم البرنامج بتنفيذ الاكواد بداخل defaulte لان defaulte تعمل فقط في حالة عدم تحقق الحالات وراجاع قمية false.



For loop

الحلقات التكرارية : لنفترض انك تريد طباعة الأعوام من من ١٩٠٠ الى ٢٠٢٠

بتأكيد انت لا تنوي ان تقوم بتكرار هذه العملية وتكرار امر الطباعة ١٢٠ مرة حتى تصل الى هدفك من هنا تستطيع استخدام الحلقات التكرارية من خلالها تستطيع توفير الوقت والجهد وأيضا سيكون أداء برنامجك افضل .

```
Run | Debug
1  void main(){
2
3   print(1900) ;
4   print(1901) ;
5  print(1902) ;
6   print(1903) ;
7
8  }
```

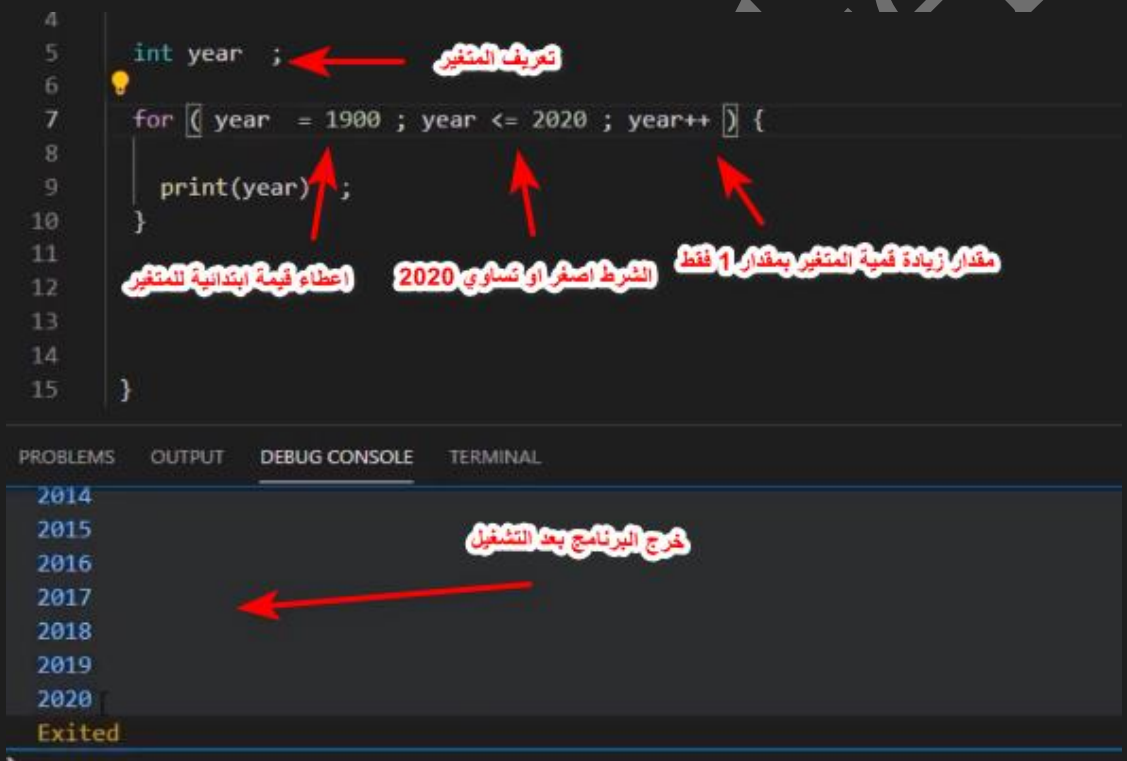
شكل طريقة كتابة الحلقة التكرارية FOR :

```
Run | Debug
1  void main(){
2
3   // for (var = value  ;  condtion ; opreator  ) {  code  }
4
5
6  }
```



في هذا المثال نريد طباعة الأعوام من ١٩٠٠ الى ٢٠٢٠

يجب كخطوة أولى تعريف المغير ومن ثم كتابة حلقة FOR ومن ثم إعطاء قيمة ابتدائية للمتغير وتعتبر نقطة بداية لبدأ العد ثم لا ننسا كتابة الفاصلة المنقوطة ثم كتابة الشرط الا وهو ان تكون year اصغر او تساوي ٢٠٢٠ ومن ثم نقوم بأدخال عداد الزيادة وهو بمقدار ١ فقط .



الية عمل الحلقة for loop كما هو واضح في الصورة في الأسفل خطوات ومراحل عمل الحلقة في البرنامج وسوف أقوم بشرح خطوة خطوة .

١ – يقوم بجلب القيمة الابتدائية للمتغير .

٢ – يقوم بأضافة مقدار الزيادة على قيمة المتغير ومقدار الزيادة هنا هو واحد كما شرحنا في الدروس السابقة ان كتابة اسم المتغير ثم وضع علامة ++ بعده معناها إضافة مقدارها واحد على قيمة المتغير .

٣- تحقق من الشرط : بعد إضافة مقدار الزيادة واحد الى المتغير يجب تأكد من تحقق الشرط هل ١٩٠١ اصغر او تساوي ٢٠٢٠ هنا نجد ان الشرط محقق وقام بأرجاع قيمة true ينتقل الى الخطة الرابعة .

٤ – تنفيذ ما داخل الحلقة التكرارية

٥ – تنفيذ امر الطباعة

وإعادة العمية تكرار حتى يصل لمرحلة عدم تحقق الشرط و أرجاع قيمة false والخروج من الحلقة التكرارية بشكل نهائي.



AboBaker

الان لنقوم بشرح هذا المثال البسيط واستخدام دالة IF داخل الحلقة FOR

في هذا المثال تقوم الحلقة التكرارية بنفس الية عملها كما تكلمنا عنها سابقا حيث تقوم بعد الأعوام من ١٩٠٠ حتى تصل الى ١٩٥٠ هنا قد تحقق شرط في دالة الشرطية IF هنا سوف يقوم البرنامج بتنفيذ امر الطباعة فيها ومن ثم يكمل العد حتى يصل الى ٢٠٢١ يقوم بتحقق منها وترجع قيمة FALSE لان الشرط غير محقق وهنا يقوم البرنامج بلخروج من الحلقة التكرارية .

كما هو واضح في المثال في الأسفل :

```
11  for ( year  = 1900 ; year <= 2020 ; year++ ) {
12      if (year == 1950) {
13          print("year ====== 1950") ;
14      }
15      print(year) ;
16  }
17
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
1947
1948
1949
year ====== 1950
1950
1951
1952
1953
```

خرج الدالة if

خرج حلقة FOR

مثال اخر عن حلقة for كما هو واضع في الصورة في الأسفل لدينا حلقة for لكن مقدار الخطوة لدينا هو ١٠ وهو مقدار تناقص حيث ستقوم الحلقة for بنفس الية عمليها حيث ستقوم بأنقاص ١٠ من النقطة الابتدائية ٢٠٠٠ حتى تصل الى ١٨٩٠ ترجع قيمة false وتخرج من الحلقة بأمكانك ادخال مقدار قيمة التناقص او الازدياد في خطوة مهما كانت بشرط ان لا ترجع قيمة false.

```
4   int year;
5   for (year = 2000 ; year >= 1900; year = year - 10) {
6     print(year);
7   }
8 }
9
```

مقدار النقصان 10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
2000
1990
1980
1970
1960
1950
1940
1930
```

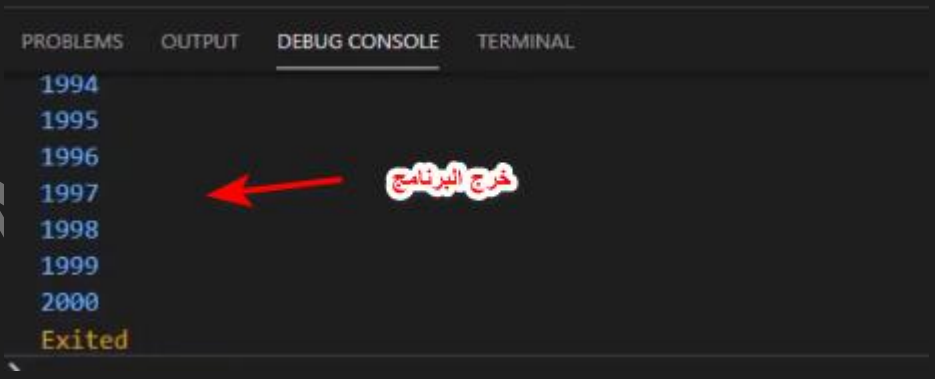
خرج البرنامج بعد التشغيل

While Loop

تتشابه اليه عمل الحلقة While مع الحلقة for مع الاختلاف بطريقة الكتابة

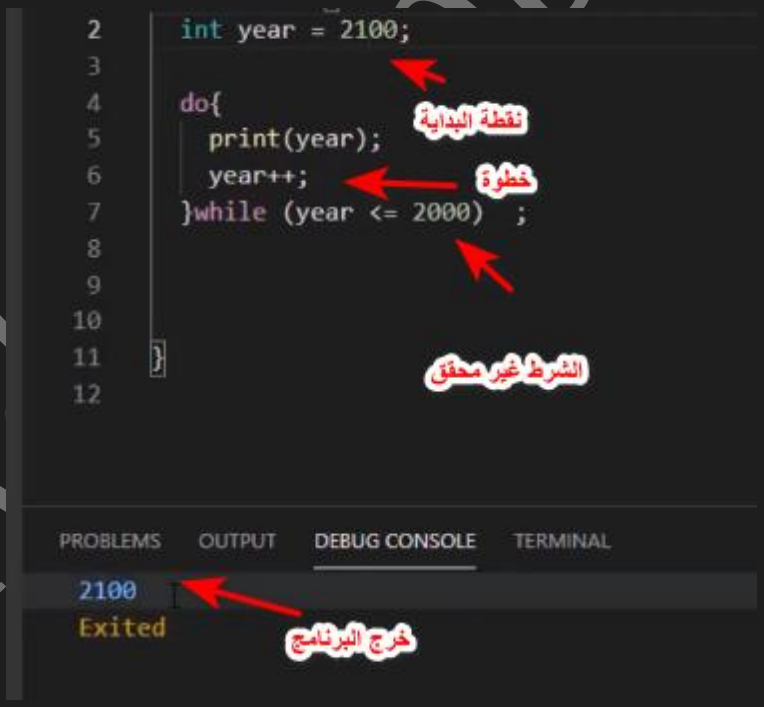
كما هو واضح في الصورة في الأسفل :





Do While Loop

تعمل حلقة Do while loop نفس عمل حلقة while حلقة for ولكن تختلف عنهم انه تقوم بلطباعة سواء الشرط تحقق او لن يتحقق كما سبق وذكرنا في الدورس السابقة انه في حال عدم تحقق الشرط في الحلقة ارجاع قيمة false فأن الكود الذي في داخل الحلقة لن يتم تنفيذه ولكن هذا الامر يختلف في حلقة while كما هو واضح في الصورة في الأسفل ان الشرط غير محقق لان ٢١٠٠ اكبر من ٢٠٠٠ ومع ذلك قام البرنامج بطباعة نقطة البداية والخروج من الحلقة.



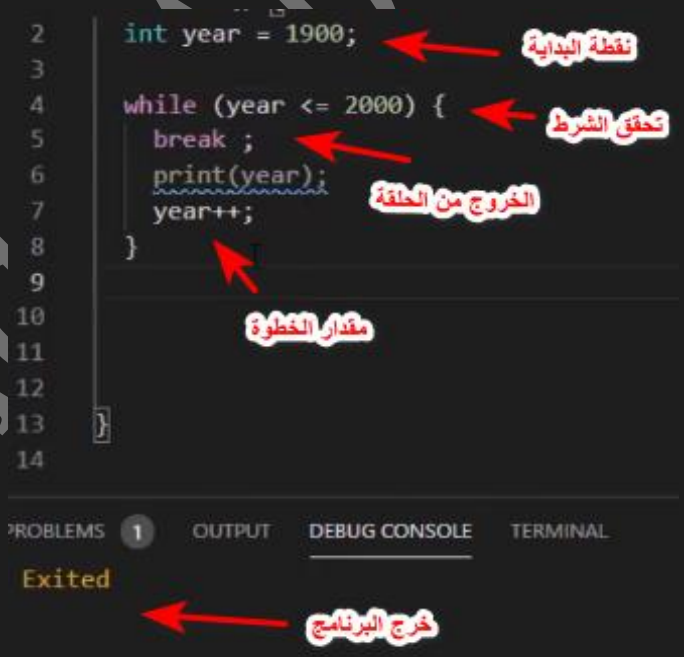
AhmedBakeel

break

مفهوم ال break مهمته هو الخرج من الحلقة التكرارية فقط لاغير وتعمل ال break مع جميع أنواع الحلقات التكراية

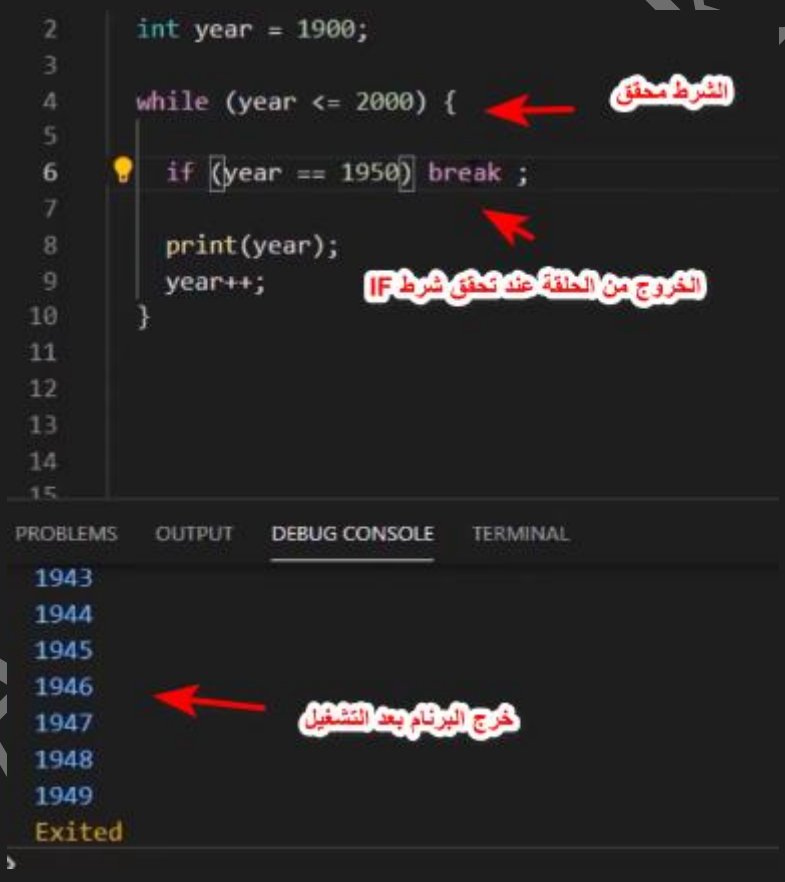
لدينا المثال التالي :

نلاحظ ان لدينا حلقة تكرارية while ونقطة الابتدائية هي ١٩٠٠ ونريد عد الأعوام من ١٩٠٠ الى ٢٠٠٠ ونلاحظ أيضا تحقق الشرط الحلقة لان قيمة year اصغر من ٢٠٠٠ وبلتالي ارجاع قيمة true وبسبب وجود break بعد تحقق الشرط تقوم بلخروج من الحلقة التكرارية بشكل نهائي وعدم تنفيذ امر الطباعة .



Ad Kho Baker

ليكن لدينا دالة IF داخل الحلقة التكرارية While نجد في الصورة في الأسفل ان الشرط حلقة محقق أي ان قيمة متغير year اصغر من ٢٠٠٠ بهذا فأن الشرط محقق وسيقوم الحاسوب بدخول الى الحلقة وتحقق من شرط دالة if عند وصول العد الى ١٩٥٠ سيتحقق شرط if سيقوم الحاسوب بعمل break والخروج من الحلقة بشكل نهائي كما هو واضح ولن يقوم بطباعة ١٩٥٠



continue

تقوم continue بنفس عمل break ولكن يوجد اختلاف الا وهو ان break تقوم بلخروج من الحلقة التكرارية بشكل كامل ونهائي اما continue تقوم بلخروج من الدورة او الخطوة لنفهم الامر بشكل افضل .

ليكن لدينا هذا المثال الموضح في الأسفل :

ليكن لدينا دالة IF داخل الحلقة التكرارية for نجد في الصورة في الأسفل ان الشرط حلقة محقق أي ان قيمة متغير year اصغر من ٢٠٠٠ بهذا فأن الشرط محقق وسيقوم الحاسوب بدخول الى الحلقة وتحقق من شرط دالة if عند وصول العد الى ١٩٥٠ سيتحقق شرط if سيقوم الحاسوب بعمل continue والخروج من دورة فقط كما هو واضح ولن يقوم بطباعة ١٩٥٠ لكن لن يقوم بلخروج من الحلقة بشكل نهائي.

```
2     int year = 1900;
3     for (year = 1900 ; year <= 2000 ; year++){
4       if (year == 1950) continue ;
5       print(year) ;
6     }
7   }
8
```

الشرط محقق

الخروج من الدورة

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL   Filter (e.g. text, !exclu

1945
1946
1947
1948
1949
1951
1952

Please start a debug session to evaluate expressions
```

عند الطباعة قام بتخطي 1950

كما هو واضح في خرج البرنامج

AboBaker

في هذا المثال المطابق تماما أعلاه قمنا فقط بأعطاء شرطين في دالة if هو ان عند وصول العد الى اكبر من ١٩٥٠ واصغر من ١٩٧٥ قم بعمل continue وتجاهل الدورات من ١٩٥١ الى و١٩٧٤ وبذلك لن يقوم الحاسوب بطباعتها كما هو واضح في المثال في خرج البرنامج .

```
2     int year = 1900;
3
4     for (year = 1900 ; year <= 2000 ; year++){
5         if (year > 1950 && year < 1975) continue ;
6             print(year) ;
7     }
8  }
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
1945
1946
1947
1948
1949
1950
1975
```

سيقوم البرنامج بتجاهل دورات من 1951 الى 1974

Number

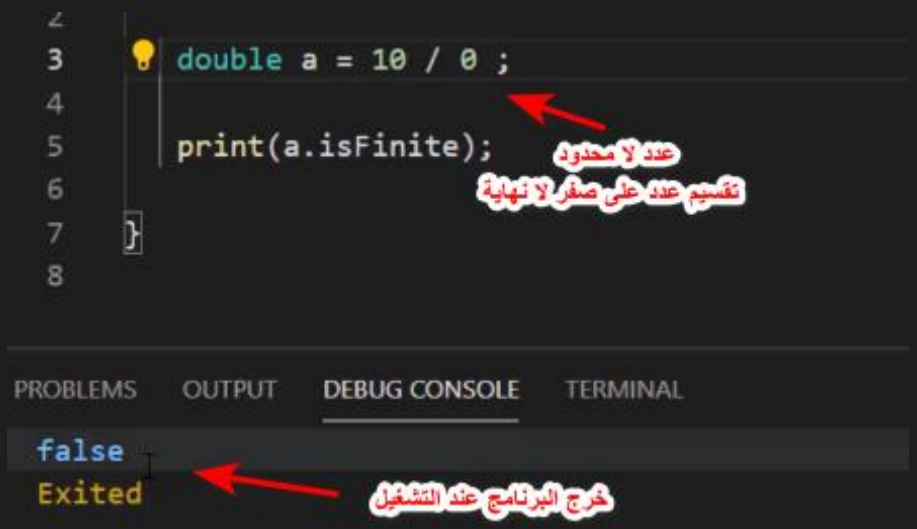
العمليات على الاعداد في المثال الموضح ادناه لدينا كأننا نقوم بتوجيه سوال للحاسوب مضمونه هل المتغير قيمته محدودة وهذا ما تعنيه isFinite ماذا يعني رقم محدود ورقم غير محدود :

الرقم لمحدود مثل ١٠ او ٩٩ او ١ او -١

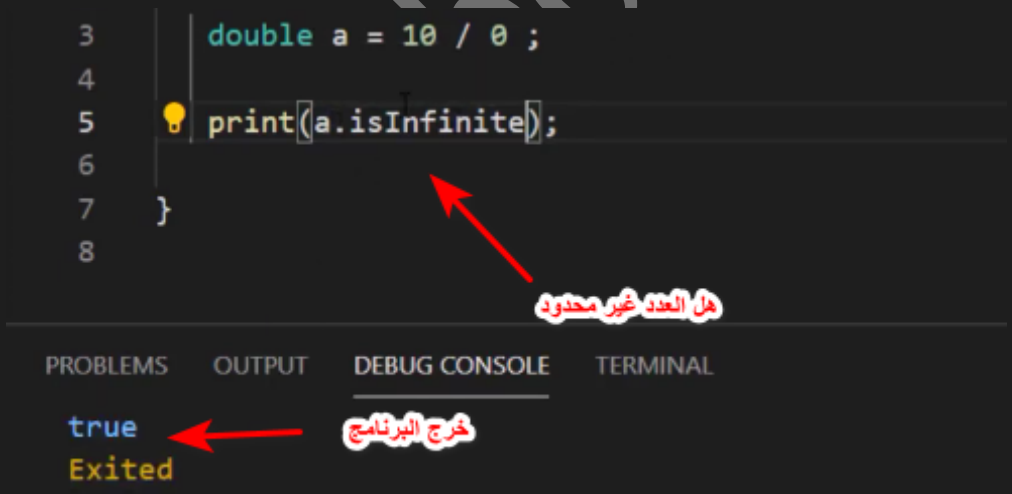
ونلاحظ ان قام البرنامج بأرجاع قيمة true لان الرقم ١٠ رقم محدود.



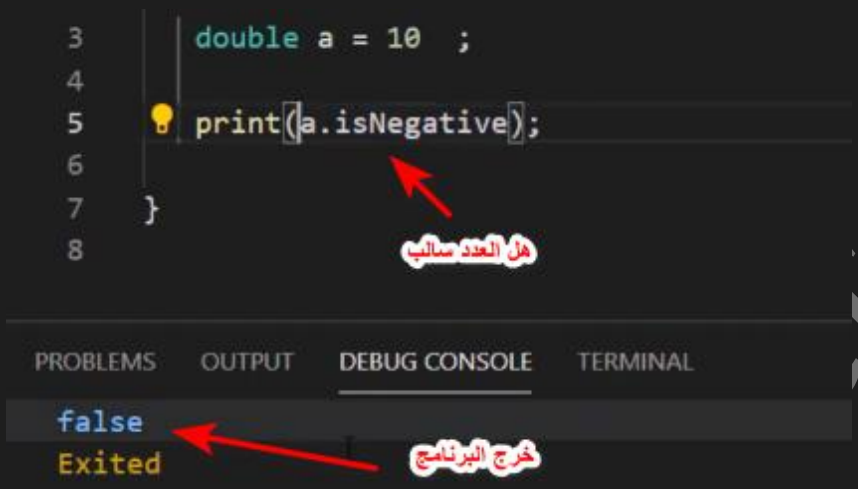
ولكن العدد الغير محدود مثل تقسيم عدد على صفر ترجع قيمة لا نهاية وهذا الأمور لا تتعلق بعلم الحاسوب انما هي قواعد رياضية كما هو واضح في الصورة في الأسفل :



ويوجد أيضا عكس isFinite وهي isInFinite هنا نقوم بتوجيه سؤال للحاسوب هل العدد a غير محدود سيقوم بأرجاع قيمة true لان العدد فعلا غير محدود كما هو واضح في الصورة في الأسفل :



IsNegative هل العدد سالب نلاحظ ان قيمة متغير a موجبة وبلتالي سيقوم بأرجاع قيمة false في خرج البرنامج ولو كان عكس ذلك سيقوم بأرجاع قيمة true كما هو واضح في الصورة في الأسفل :



Sign تقوم بأرجاع قيمة ١ للعد الموجب في خرج البرنامج او -١ للعدد السالب وتقزم بأرجاع قيمة صفر اذا كان العدد صفر المثال كما هو واضح في الصورة في الأسفل :



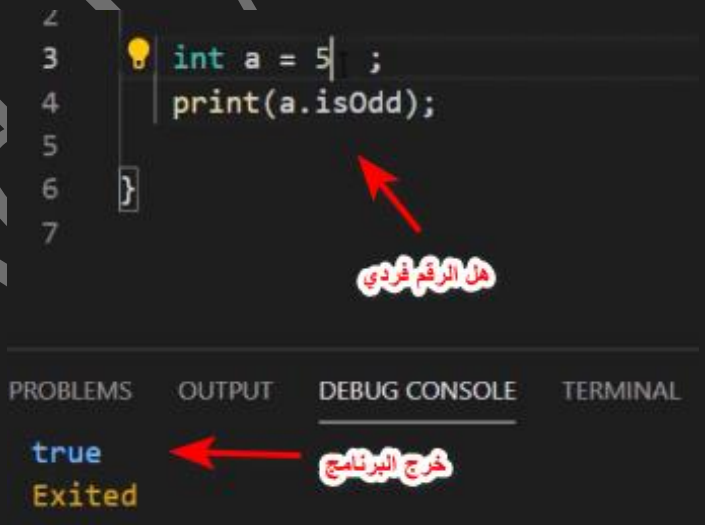
IsEven هل العدد زوجي

في حال كان قيمة المتغير عدد زوجي سيقوم بأرجاع قيمة true واذا كان فردي سيقوم بأرجاع قيمة false كما هو واضح في اصورة في الأسفل :



IsOdd هل الرقم فردي

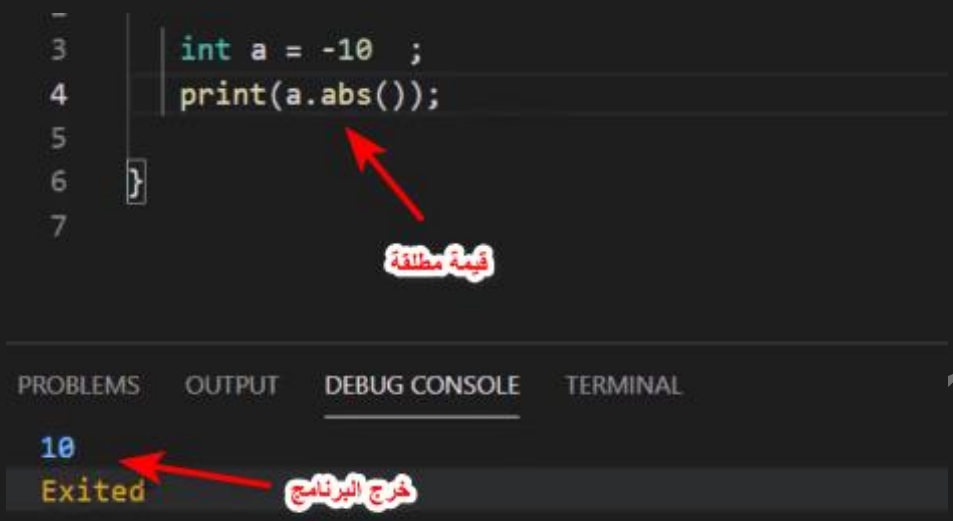
في حال كان قيمة المتغير عدد فردي سيقوم بأرجاع قيمة true واذا كان زوجي سيقوم بأرجاع قيمة false كما هو واضح في اصورة في الأسفل :



Ahmed Abou

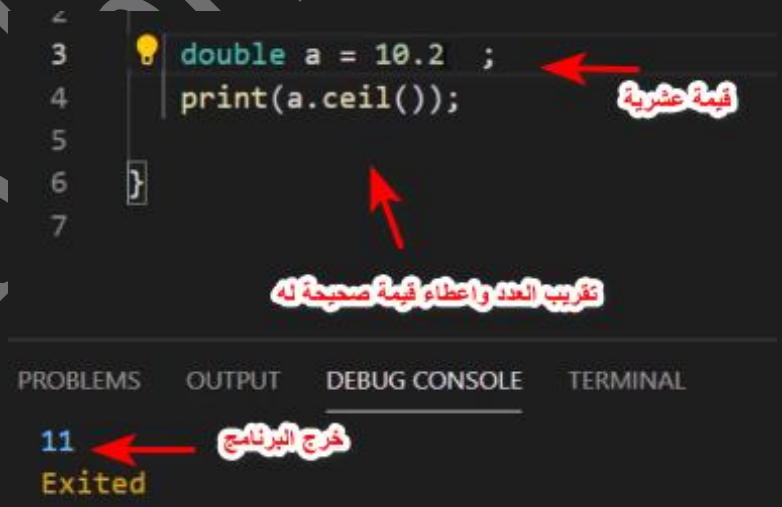
Abs() قيمة مطلقة (أي قيمة سالبة يبقوم بتحويلها الى قيمة موجبة)

كما هو واضح في المثال في الأسفل :



Ceil() إعطاء قيمة تقريبية للعدد العشري(تحويله لعدد صحيح من دون فواصل).

ودائما ما يقرب للعدد الأكبر كما هو واضح في الصورة في الأسفل :



Ajal Abou

compareTo(number) يقوم بلمقارن بين عدد واخر ويرجع ثلاث قيم :

عند التساوي العددين يرجع قيمة صفر

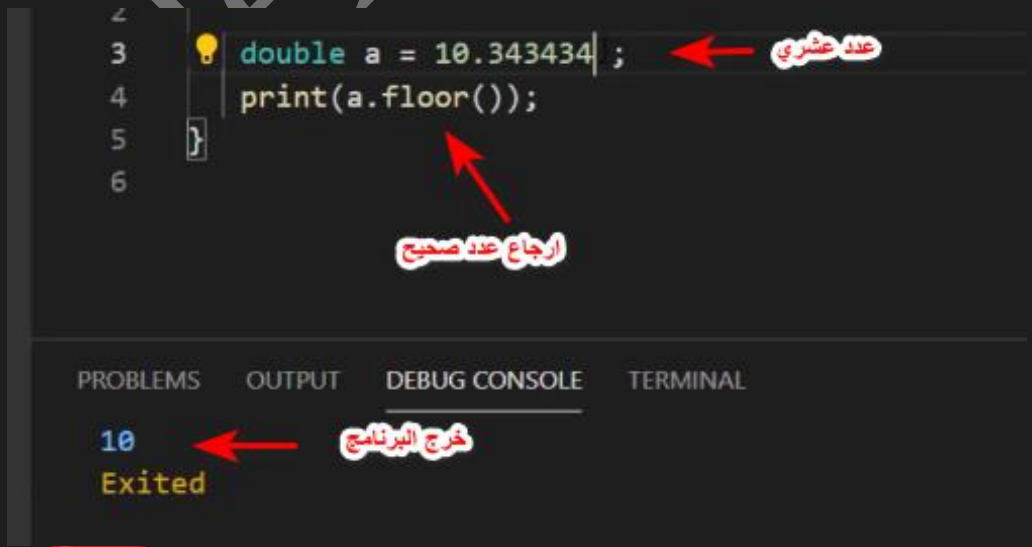
عندما يكون العدد الأول اكبر من العدد الثاني يرجع قيمة ١

عندما يكون العدد الأول اصغر من العدد الثاني يرجع قيمة -١

كما هو واضح في المثال في الأسفل :



Floor() تقوم بأرجاع العدد العشري الى عدد صحيح وتجاهل الأرقام التي تأتي بعد الفاصلة كما هو واضح في المثال في الأسفل :



Round() تقوم بنفس عمل Ceil() ولكن تقوم بتقريب الى العدد الأقرب

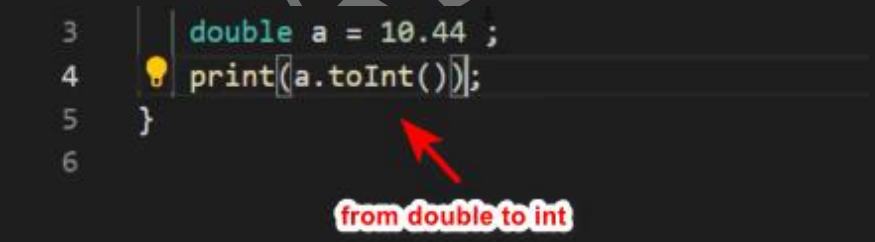
كما هو واضح في المثال في الأسفل :

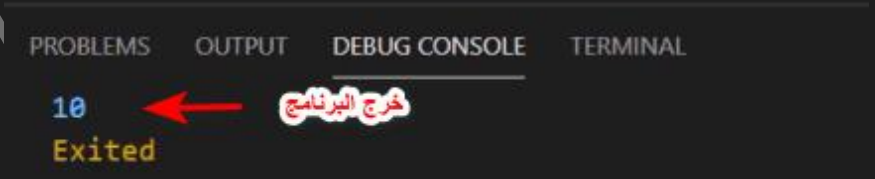




ToInt() تحويل قيمة المتغير من double الى int :

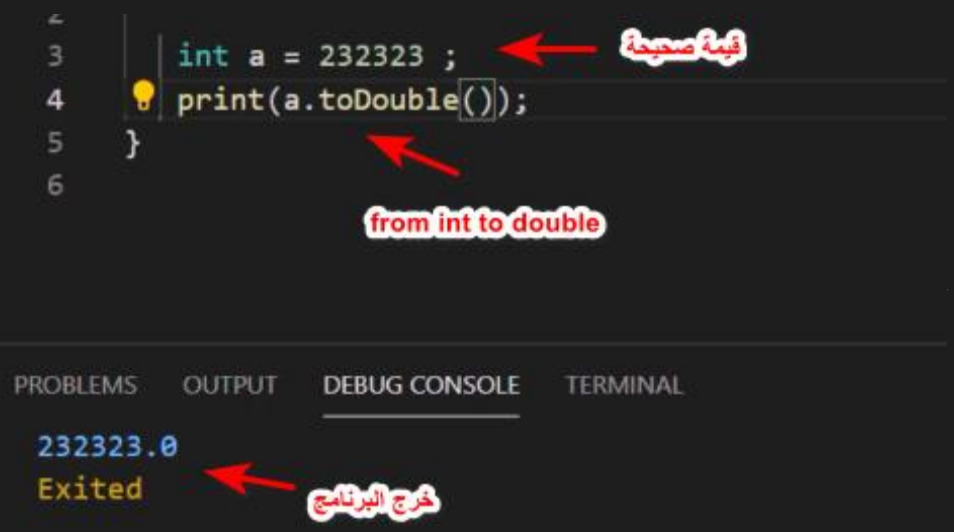
كما هو واضح في الصورة في الأسفل :





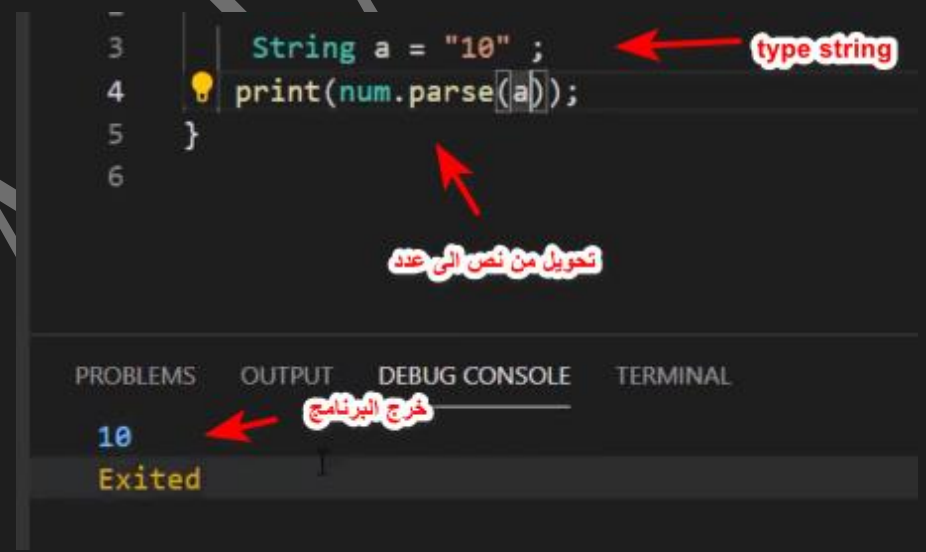
toDouble()تحويل قيمة المتغير من int الى double :

كما هو واضح في الصورة في الأسفل :



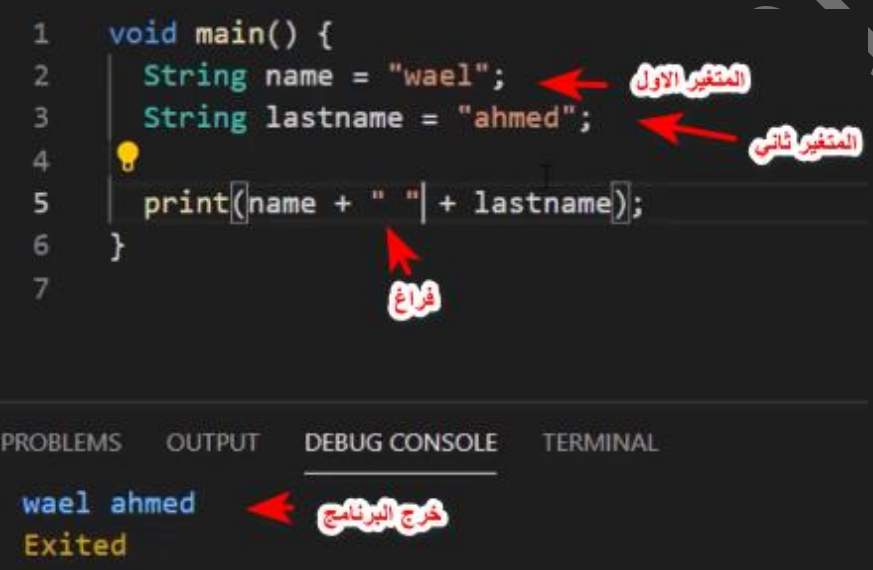
Parse(variable) تحويل قيمة المتغير من نوع string الى int

كما هو واضح في الصورة في الأسفل :



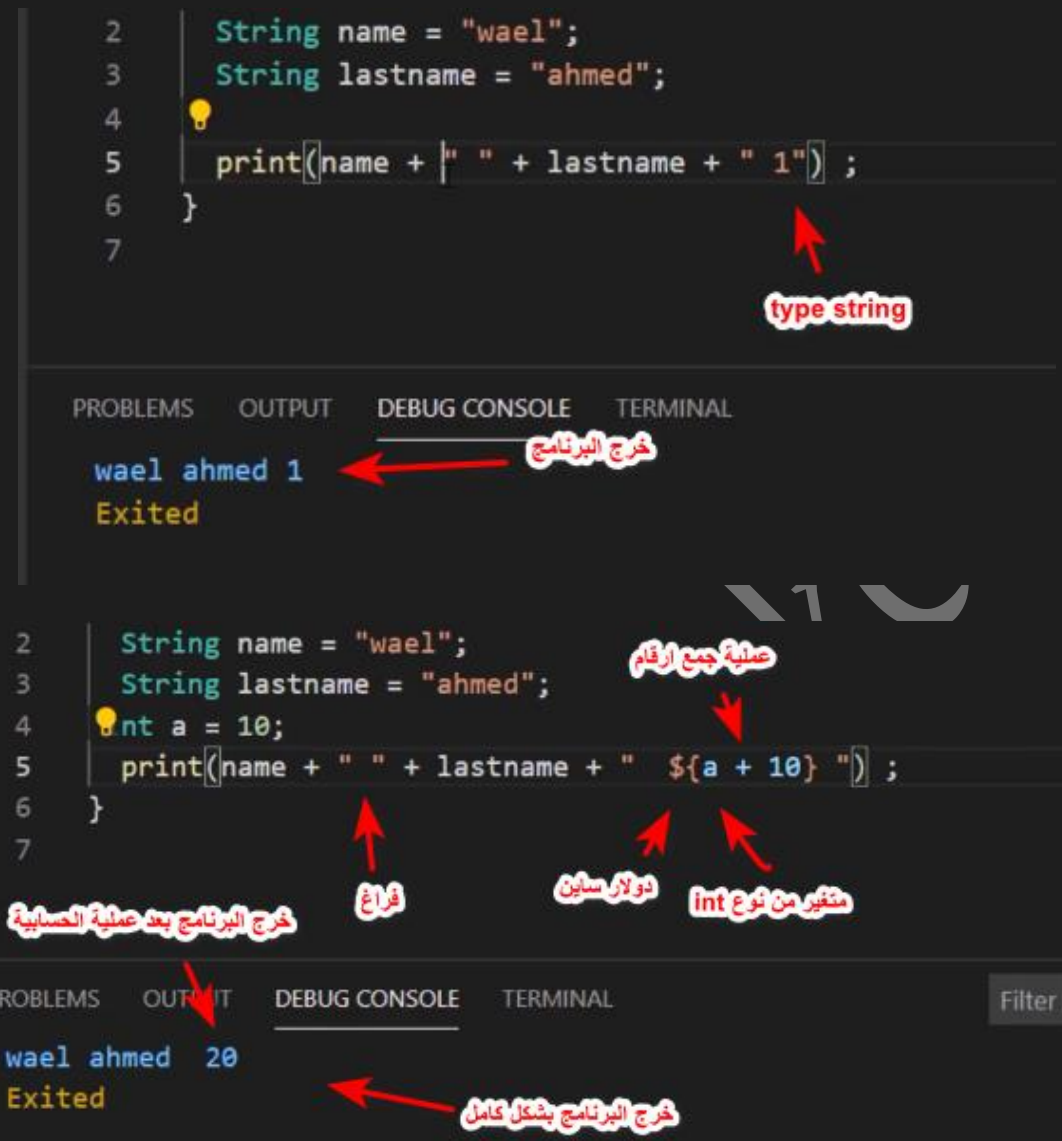
String Method

علامة + في متغيرات من نوع STRING هي دمع لعى عكس الأرقام تكون وظيفتها جمع الاعداد مثل ما هو واضح في المثال في الأسفل لدينا متغيران من نوع string اردنا وضع المتغيران في امر الطباعة ووضع مسافة بينهما بستخدام دوبل كوتشن.



لا يمكن وضع الأرقام من نوع int ودمجها مع متغيرات من نوع string الا اذا كانت ضمن دوبل كوتشن الرقم هنا هو من نوع string لانه ضمن دوبل كوتشن.

Adel



في المثال الموضح أعلاه نريد الربط بين متغيرات int ومتغيرات string وتنفيذ عملية جمع بين عددين دون ظهور خطأ في البرنامج.

نقوم بوضع المتغيران من نوع int ضمن { } ونضع قبله إشارة $ ليقوم جهاز الحاسوب بتعرف عليهم على انهم اعداد صحيحة من نوع int .

Operations on variables of type String

IsEmpty نقوم بأستخدامها اذا اردنا ان نعرف اذا كان المتغير فارغ او لا ويقوم بأرجاع اما true او false .

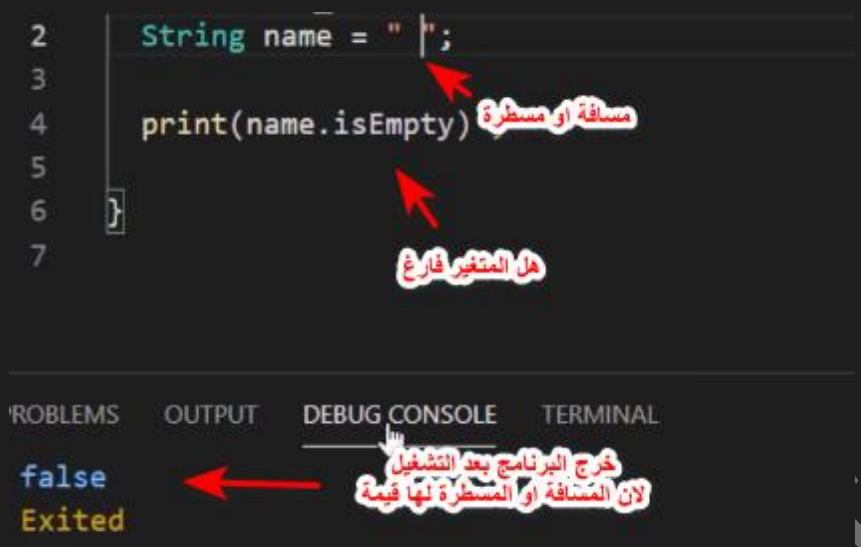
في حال كان لديه قيمة سيقوم بأرجاع قيمة false.

كما هو واضح في الصورة في الأسفل :

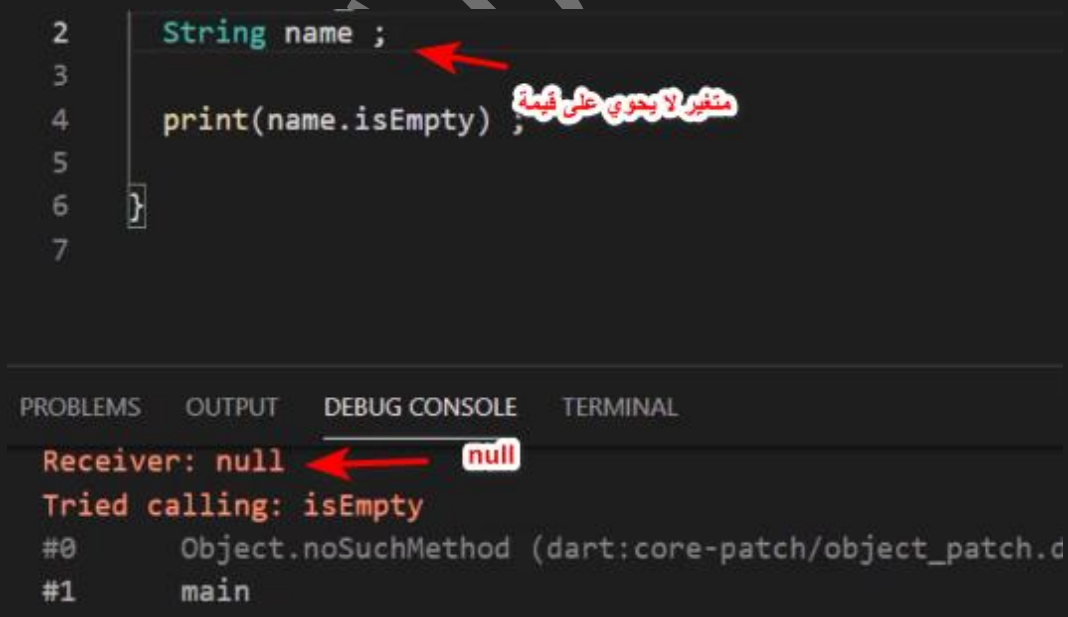


ملاحظة مهمة جدا : المسافة او المسطرة لها قيمة مثلها مثل أي حرف او رقم او إشارة او علامة في الحاسوب.

Ader



علينا التفريق بين متغير فارغ ومتغير ليس له قيمة كما في الصورة في الأسفل لان المتغير الذي لا يحوي على قيمة يرجع قيمة null.

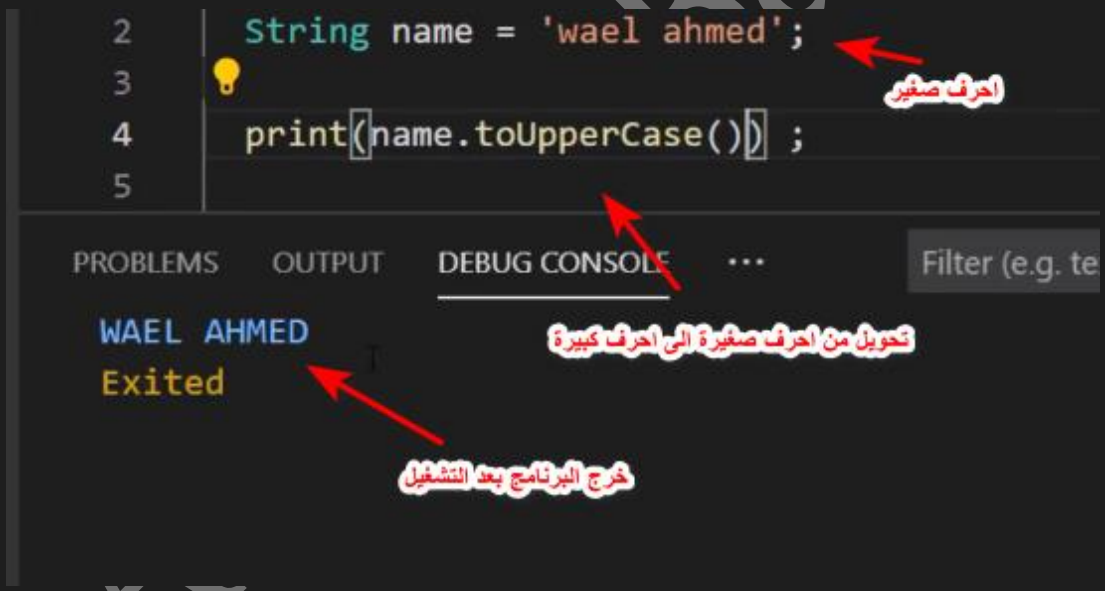


Length عدد الاحرف او طول string يقوم بعد الاحرف بلمتغير من نوع string ويرجع قيمة من نوع int لدينا المثال في الأسفل نريد معرفة عدد الاحرف في متغير name وعند التنفيذ قام بطباعة عدد ٨ عدد احرف هو ٧ بلاضافة للفراغ فقم بطباعة عدد ٨ في الخرج كما هو واضح في الصورة في الأسفل :



toUpperCase() تحويل من حروف الصغيرة الى حروف كبيرة

كما هو واضح في المثال في الأسفل :



toLowerCase() تحويل من احرف كبيرة الى احرف صغيرة

كما هو واضح في الصورة في الأسفل :



Trim() تقليم او حذف الفراغات سبق وذكرنا في الدروس السابقة ان الراغ له قيمة وعند استعمال ال trim في برنامجنا قام بتجاهل الفراغ وطباعة عدد احرف الكلمة التي تحوي على ٤ حروف فقط كما هو واضح في الصورة في الأسفل :

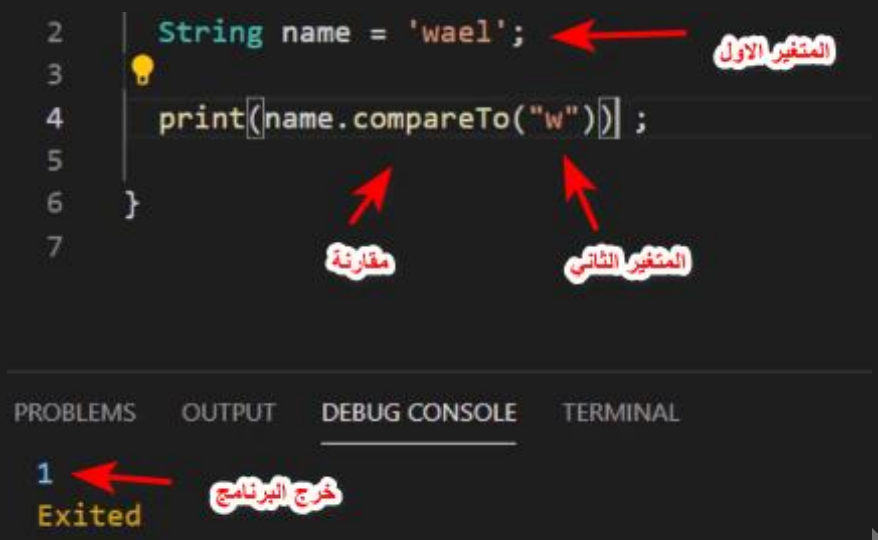


compareTo("string") مقارنة بين متغيرين من نوع string يقوم بأرجاع قيمة ١ و -١ و صفر.

في حال كان عدد احرف متغير الأول اكثر من متغير ثاني سيقوم بأرجاع القيمة ١

في حال كان عدد احرف متغير الأول اقل من متغير ثاني سيقوم بأرجاع القيمة -١

في حال يوجد تطابق تام بينهم سيقوم بأرجاع قيمة صفر.



replaceAll(from , replase)

تستطيع من خلالها استبدال كلمة ما في المتغير سيقوم بلبحث عن كلمة you واستبدالها بكلمة all .

كما هو واضح في المثال في الأسفل :



List

القوائم او مصفوفة : وهي نوع من أنواع المتغيرات في البرمجة.

تقبل عدة قيم بداخلها بمختلف أنواعها اعداد او نصوص نفصل بين القيمة والأخرى بأشارة الكومة او فاصلة (،).

عند كتابة مجموعة من القيم بداخلها فأن اول قيمة فيها تحجز قيمة صفر في ذاكرة الحاسوب وثاني قيمة تأخذ قيمة ١ وثالث قيمة تأخذ قيمة ٢ وهكذا الى لا نهاية.

كما هو واضح في الصورة في الأسفل :

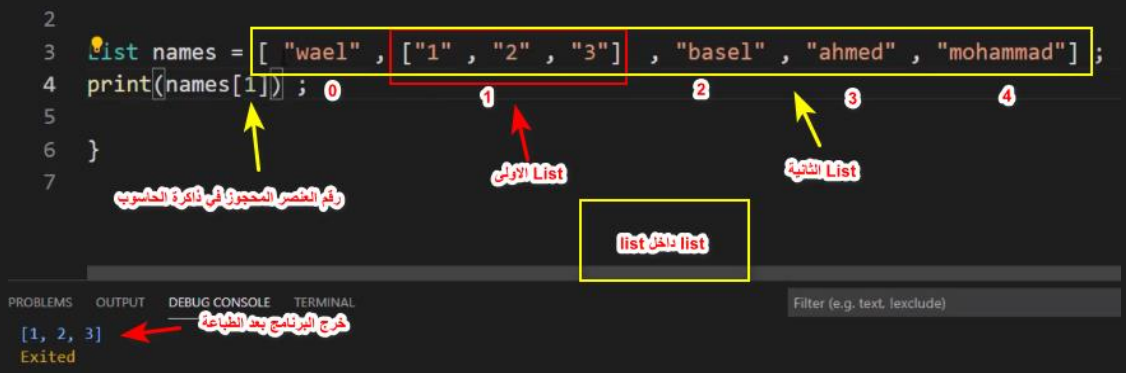




Baker

نستطيع كتابة او انشاء list داخل list انشاء عدد لا نهائي من list بداخل بعضهم البعض.

كما هو واضح في الصورة في الأسفل :



ليكن لدينا المثال الذي في الأسفل نلاحظ انه يوجد list بداخل list أخرى ونريد الوصول وطباعة عنصر معين على سبيل المثال الرقم ٢ من ال list الثانية من ال list الثانية نلاحظ ان حجز list الثانية في list الأولى وتأخذ قيمة [1] في ذاكرة الحاسوب وان الرقم ٢ يحجز أيضا قيمة [1] من list ثانية

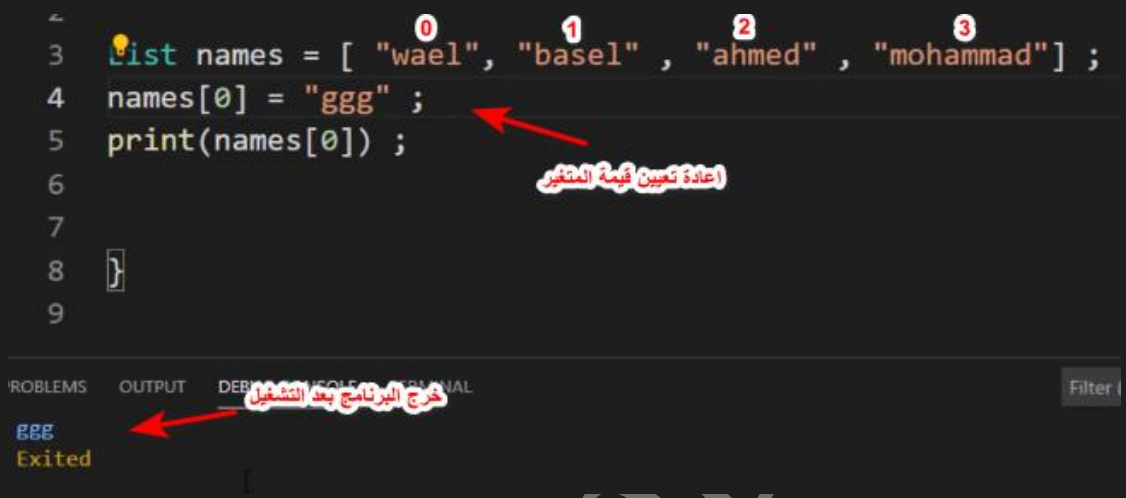
اذا اردنا طباعة العنصر نقوم بكتابة اسم list ثم رقم حجز list ثانية قيمته [1] ثم رقم حجز العنصر ٢ داخل list ثانية وقيمته أيضا [1]

كما هو واضح في الصورة في الأسفل :



Adel AboBan

يمكننا ان نقوم بأعادة تتعين قيمة العناصر من داخل list كما هو واضح في الصورة لدينا في الأسفل :



إضافة عنصر جديد من خلال add كما هو واضح في الصورة في الأسفل :



لو اردنا على سبيل المثال ان نقوم بطباعة جميع عناصر list بلتأكيد يمكننا ان نقوم بأستخدم امر الطباعة وطباعة كل عنصر على حدا كما هو واضح في الصورة في الأسفل :

```
3  List names = [ "wael", "basel" , "ahmed" , "mohammad"] ;
4
5  print(names[0]) ;
6  print(names[1]) ;
7  print(names[2]) ;
8  print(names[3]) ;
9
10
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL        Filter (e.g. text,
wael
basel
ahmed
mohammad
Exited
```

ولكن في حال كان لدينا عناصر كثير جدا في list هل نقوم بتكرار امر طباع على عدد عناصر list بلتأكيد هذا الامر سيأخد منا وقت طويل جدا .

لهذا فأن لدينا حل لمشكلة التكرار كما ذكرنا في الدروس السابقة يمكننا استعمال الحلقات التكرارية لطباعة عناصرlist .

Adel Abu

في هذا المثال الموضح في الأسفل :

لدينا مصفوفة تحوي على ٤ عناصر نريد ان نطبع عناصر المصفوفة بأستخدام الحلقة التكرارية for نقوم بأنشاء متغير نسميه x لنقوم بتخزين عناصر المصفوفة ونعطيه قيمة ابتدائية صفر ثم نقوم بكتابة الشرط داخل for ان متغير x يجب ان يكون اصغر من عدد عناصر المصفوفة وكما نعمل من الدروس السابقة ان length تستخدم للعد عناصر او محارف المتغيرات ثم ندخل مقدار الخطوة قيمته واحد لان (x++ = 1) ثم نقوم بتنفيذ امر الطباعة .



لدينا طريقة اخر لطباعة عناصؤ المصفوفة وهي دالة مخصصة لل list

وهي forEach((variable)) نقوم بتعريف متغير لنقوم بتخزين عناصر المصفوفة بداخله ثم نقوم بطباعة هذا المتغير كما هو موضح في الأسفل :

```dart
void main() {
  List names = ["wael", "basel", "ahmed", "mohammad"];

  names.forEach((name){
    print(name) ;
  }) ;
}
```

دالة مخصصة لطباعة عناصر المصفوفة

المتغير الذي سنقوم بتخزين عناصر المصفوفة بداخله

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
wael
basel
ahmed
mohammad
Exited
```

خرج البرنامج بعد التشغيل

First سيقوم بجلب اول عنصر من list

كما هو واضح في الصورة في الأسفل :

```dart
List names = ["wael", "basel", "ahmed", "mohammad"];
print(names.first)  ;


}
```

اول عنصر في مصفوفة

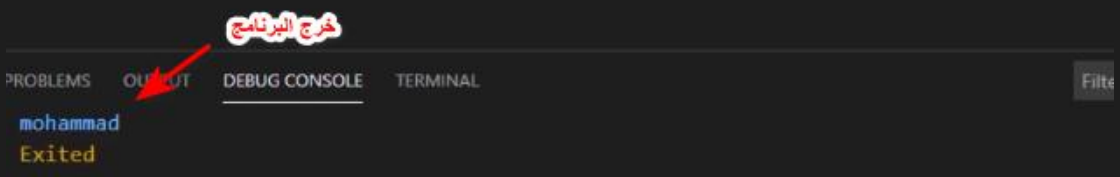
خرج البرنامج

```
ROBLEMS  OUTPUT  TERMINAL
wael
Exited
```

Last سيقوم بجلب اخر عنصر من list

كما هو واضح في الصورة في الأسفل :

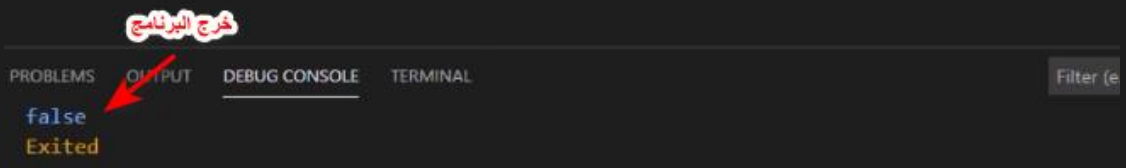
```
2   List names = ["wael", "basel", "ahmed", "mohammad"];
3   print(names.last)  ;
4
5
6   }
7
```



isEmpty هل المصفوفة فارغة سيقوم بأرجاع قيمتان : اما true او false

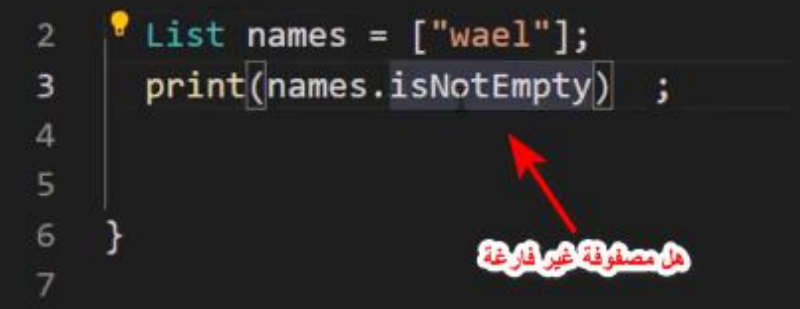
كما هو واضح قام بأرجاع قيمة false لان المصفوفة تحوي على عناصر

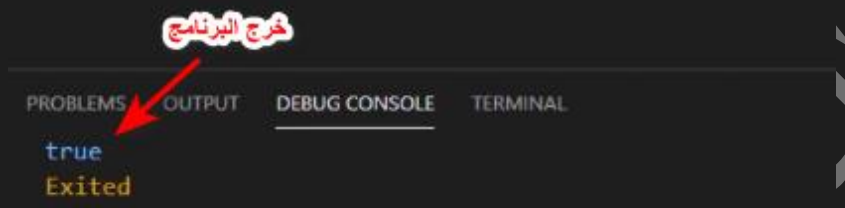
```
2   List names = ["wael", "basel", "ahmed", "mohammad"];
3   print(names.isEmpty)  ;
4
5
6   }
7
```



IsNotEmpty هل المصفوفة غير فارغة أيضا سيقوم بأرجاع قيمة true او false

كما هو واضح في المثال في الأسفل :

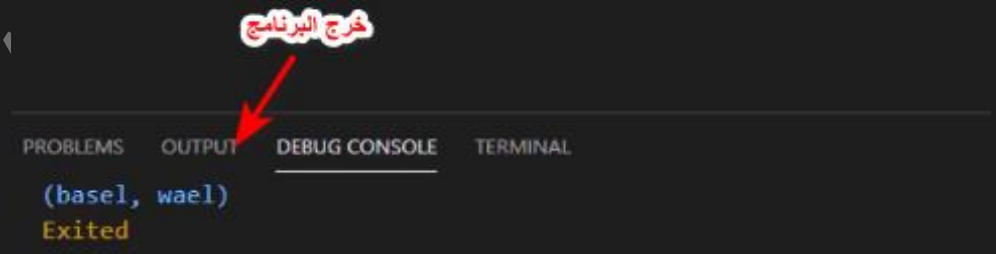




Reversed يقوم بعكس عناصر المصفوفة ولكن لا يرجع الخرج على شكل list

كما هو واضح في المثال في الأسفل :





اذا اردنا طباعة الخرج على شكل list نقوم بكتابة reversed.tolist()

Single هل العنصر وحيد في حال كان عنصر وحيد في المصفوفة يقوم بأرجاع قيمته وطباعته في حال كان يوجد عدة عناصر بأظهار رسالة خطأ كما هو واضح في الصورة في الأسفل :

```
3   List names = ["wael" , "basel" , "ahmed"];
4
5   print(names.single)  ;
6
7
8   }
9
```

هل العنصر وحيد

خرج البرنامج المصفوفة تحوي على عدة عناصر

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL                    Filter (e.g. te
Unhandled exception:
Bad state: Too many elements
#0      List.single (dart:core-patch/growable_array.dart:261:5)
#1      main
#2      _startIsolate.<anonymous closure> (dart:isolate-patch/isolate_patch.dart:301:19)

#3      _RawReceivePortImpl._handleMessage (dart:isolate-patch/isolate_patch.dart:168:12)
Exited (255)
```

addAll إضافة عناصر list لل list اخرى كما هو واضح في الصورة في الأسفل :

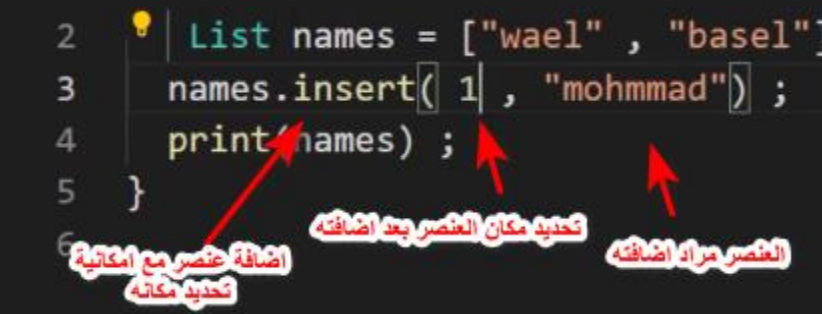
```
2   List names = ["wael" , "basel"] ;
3   names.addAll(['mohmmad' , 'ahmed']) ;
4   print(names) ;
5   }
6
```

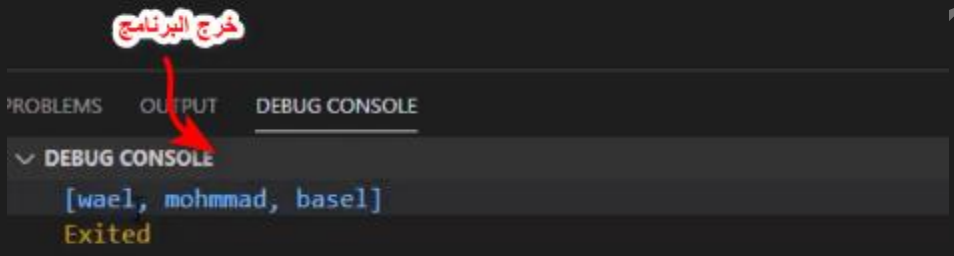
اضافة عناصر مصفوفة لمصفوفة اخرى

خرج البرنامج بعد اضافة عناصر المصفوفة

```
PROBLEMS  OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
    [wael, basel, mohmmad, ahmed]
    Exited
```

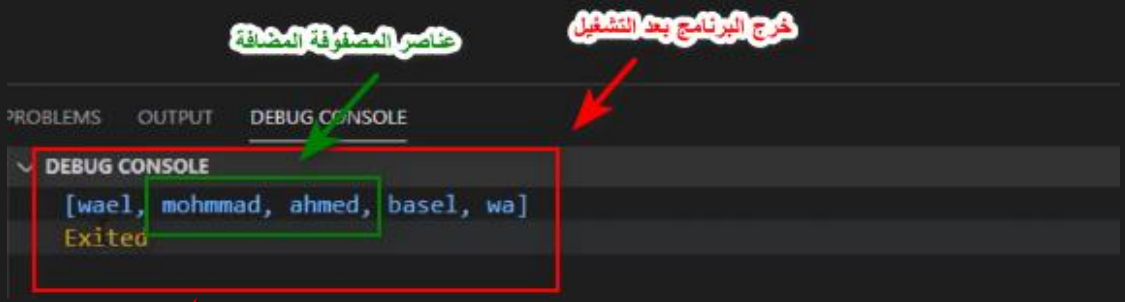
Insert تقوم بأضافة عنصر مثها مثل add ولكن مع إمكانية تحديد ماكن العنصر في list كما هو واضح في الصورة في الأسفل :





insertAll(index , [ ]) تقوم بأضافة list كاملة الى list أخرى مع إمكانية تحديد موقعها كما هو واضح في الصورة في الأسفل :





reblacerange(start , end , reblase)

تقبل ثلاث متغيرات :

Start هو العنصر الأول.

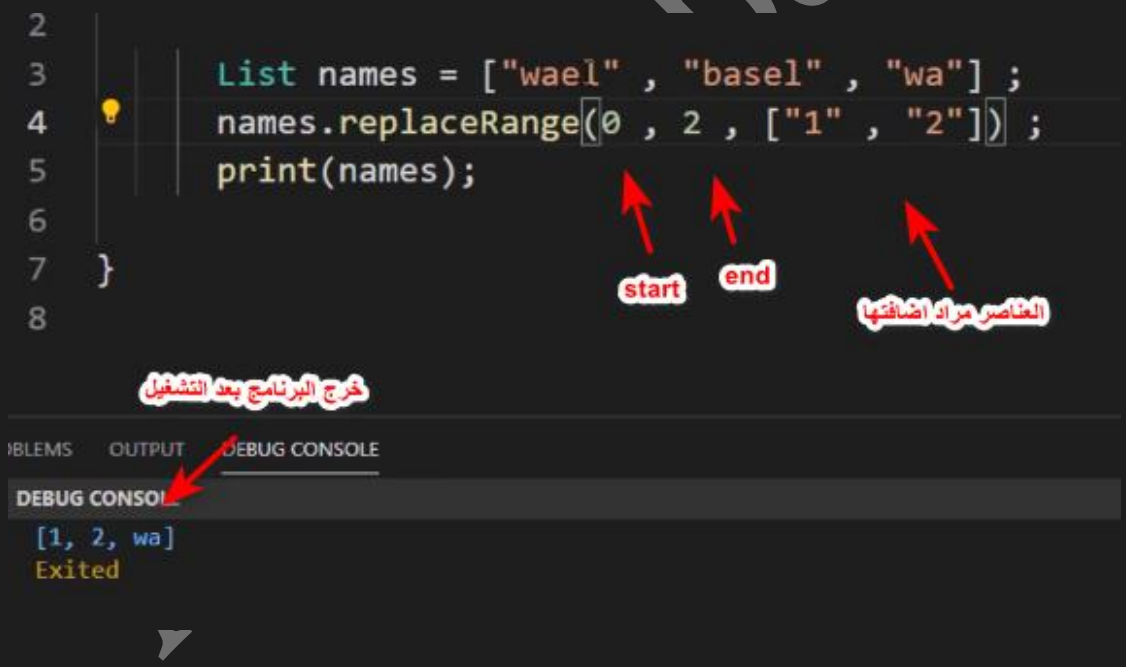
End هو العنصر الأخير.

Reblase القيم او العناصر المراد اضافتها.

يبدأ من الصفر ولكن ينتهي عند ٢ ولا يقوم بأستبدالها.

يقوم بأستبدال العناصر من 0 range الى 2 حتى ولو كان هناك عنصر واحد في Reblase.

كما هو واضح في الصورة في الأسفل :



Baker

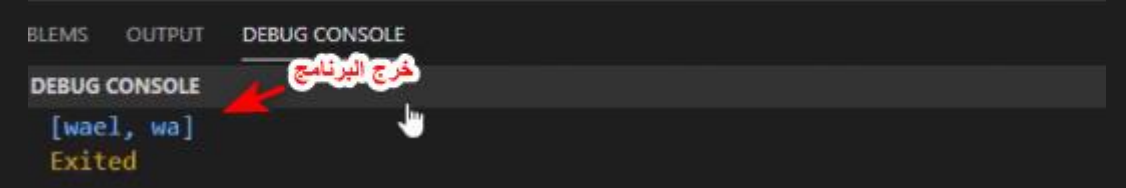
Remove يقوم بحذف عنصر من المصفوفة يقوم بلبحث عن الكلمة المطابقة تماما للكلمة المراد حذفها أي انه حساس جدا لحالة المحارف.

كما هو واضح في الصورة في الأسفل :

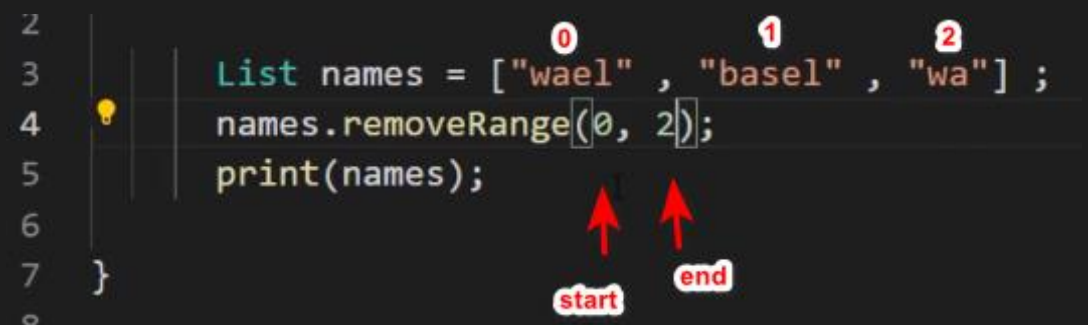


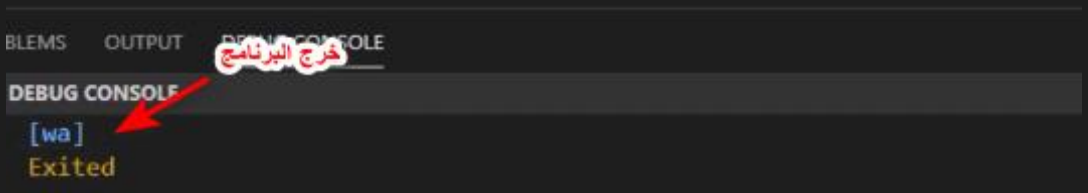
removeAt حذف عنصر ما من المصفوفة ولكن بتحديد رقم موضعه في المصفوفة كما هو واضح في الصورة في الأسفل :





removeRange يقوم بحذف عدة عناصر من المصفوفة وذلك بتحديد مجال الحذف من صفر وينتهي عن ٢ ولكن بدون حذف عنصر ٢ كما هو واضح في المثال في الأسفل :





Adel Ab

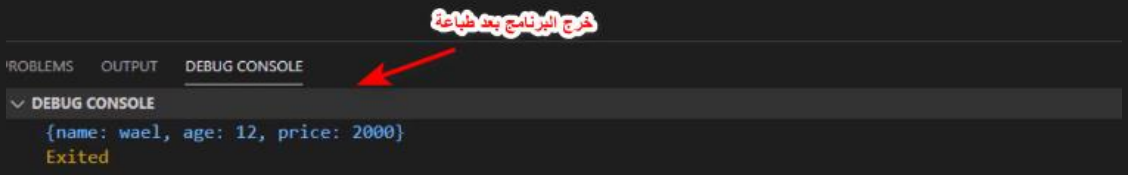
MAP

وهي نوع من أنواع المتغيرات تقبل بداخلها عدة عناصر وقيم

على شكل key : value

كما هو واضح في الصورة في الأسفل

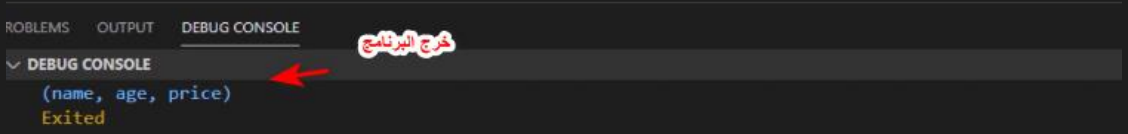




في حال اردنا طباعة key من map كما هو واضح في الصورة في الأسفل :

كما هو الحال لو اردنا طباعة value نفس الامر تماما





نستطيع اجراء العمليات على MAP :

isNotEmpty هل Map ليست فارغة.

isEmpty هل Map فارغة.

Lenght عدد العناصر في Map.

Clear تفريغ Map بشكل كامل من عناصر.

واستخدام ForEach لطباعة value او key كما هو واضح في الصورة في الأسفل:

```
3
4   Map info = {  "name"  : "wael"  , "age" : 12 , "price" : 2000 }  ;
5   info.forEach((key, value) {
6     print(value) ;
7     }) ;
8   }
9
```

لطباعة عناصر MAP

```
PROBLEMS  OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
خرج البرنامج
wael
12
2000
Exited
```

Remove حذف عنصر من Map وتتعامل فقط مع Key

Key : Value لانه يعتبر عنصر واحد

Adel

```
2   Map info = {"name": "wael", "age": 12, "price": 2000};
3
4   info.remove("name");
5   print(info);
6 }
7
```

يقوم بحذف عنصر من ال map

```
OBLEMS  OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
{age: 12, price: 2000}
Exited
```

خرج البرنامج

addAll إضافة عنصر Key : Value الى Map

كما هو واضح في الصوةر في الأسفل :

```
2   Map info = {"name": "wael", "age": 12, "price": 2000};
3   info.addAll({ "lastname" : "ahmed" }) ;
4   print(info);
5 }
6
```

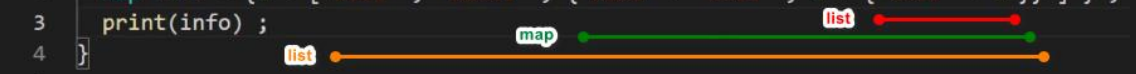
اضافة عنصر الى Map

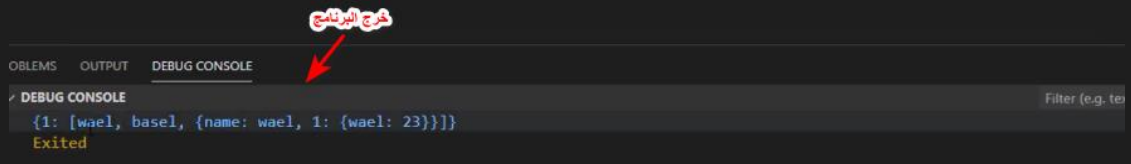
```
PROBLEMS  OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
{name: wael, age: 12, price: 2000, lastname: ahmed}
Exited
```

خرج البرنامج

نستطيع كتابة عدد لا نهائي من list وMap داخل بعضهم البعض كما هو واضح في الصورة في الأسفل :

```
2   Map info = {1 : ["wael" , "basel" , {"name" : "wael" , 1 : {"wael" : 23}} ] } ;
3   print(info) ;
4   }
5
```

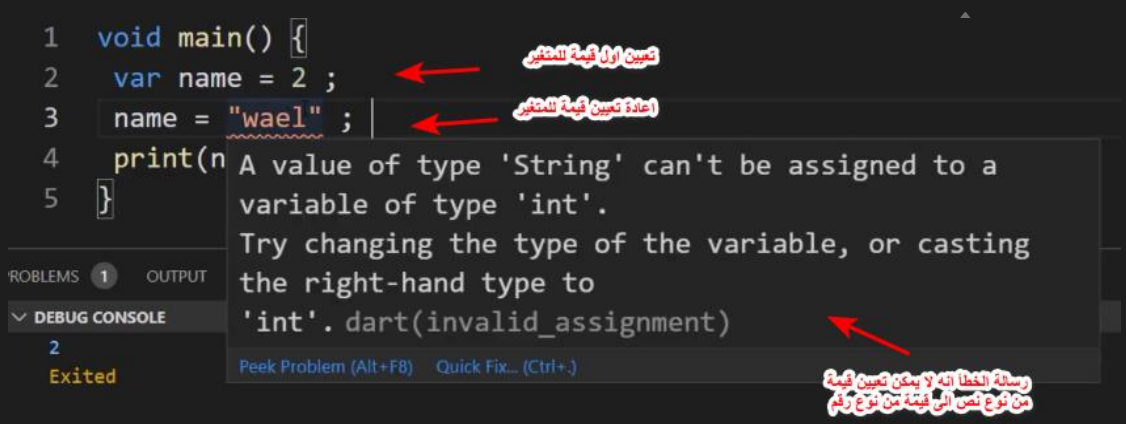




Adel AboBaker

Dyanmic Varibles

هو نوع من أنواع المتغيرات مثله مثل String او int ولكن متغير ديناميكي var يقبل أي قيمة مهما كان نوعها سواء رقم او نص او مصفوفة ويتم تعينه لمرة واحدة فقط كما هو واضح في الصورة في الأسفل :



يمكننا إعادة تعيين قيمة للمتغير في حال كان نفس نوع المتغير الأول :

المتغير الأول من نوع string اذا اردنا إعادة تعيين قيمة يجب ان يكون حصرا من نوع string كما هو واضح في الصورة في الأسفل :



final and const

consts لا يستطيع احد تغير قيمة المتغير او إعادة تعينه

كما هو واضح في الصورة في الأسفل : لدينا متغير من نوع string اسمه name

بعد ما قمنا بأضافة const قبله لا يمكننا إعادة تعيين قيمته.



Final نستطيع تمرير قيمةللمتغير مرة واحدة فقط.

كما هو واضح في الصورة في الأسفل :



Set

هي نوع من أنواع المتغيرات أيضا مثل String و int و Map ولكن تختلف Set عن Map انها لا تقبل قيم من نوع key و value كما هو الحال في Map ولكن تقبل قيم عادية مثلها مثل List تتميز Set بعدم التكرار أي لو فرضنا اننا قمنا بتكرار قيمة معينة مثل ما هو واضح في المثال في الأسفل :

لا يقوم البرنامج بأظهار خطأ انه يوجد داخل Set قيمة متكرر ولكن سيقوم بعدم طباعتها في خرج البرنامج.

```
2    Set names = { "wael"  , "basel"  , "ahmed"  , "ahmed" } ;
3    print(names) ;
4  }
```

القيمة المتكررة

```
BLEMS 1   OUTPUT   DEBUG CONSOLE
DEBUG CONSOLE
{wael, basel, ahmed}
Exited
```

خرج البرنامج

نستطيع اجراء العلميات على Set مثلها مثل أي نوع من أنواع المتغيرات

مثل singel و last و first و add و addAll .... وغيرها من أنواع العمليات على المتغيرات.

يمكننا اضاقة عدة عناصر عن طريق addAll مثل list لل Set

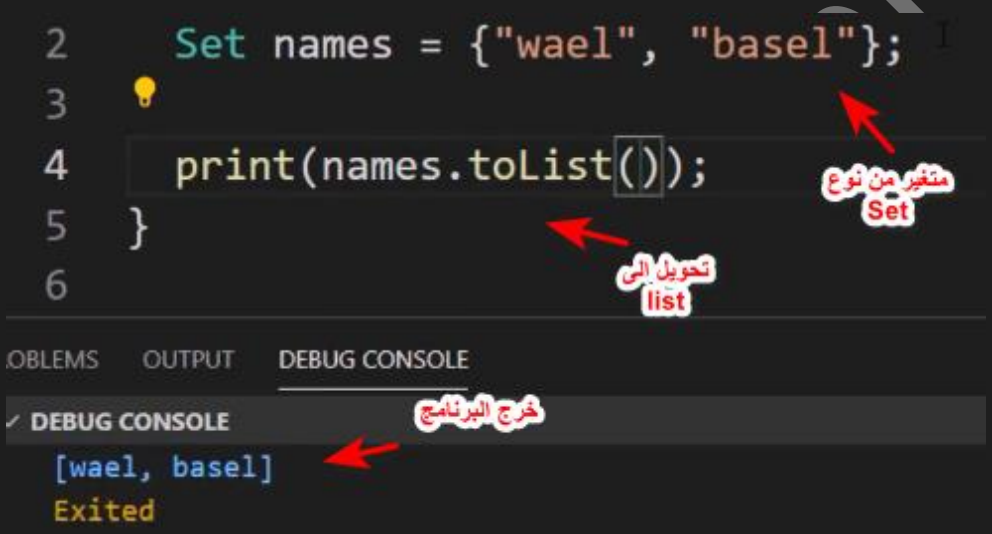
و Set لل Set.

Adel Baker

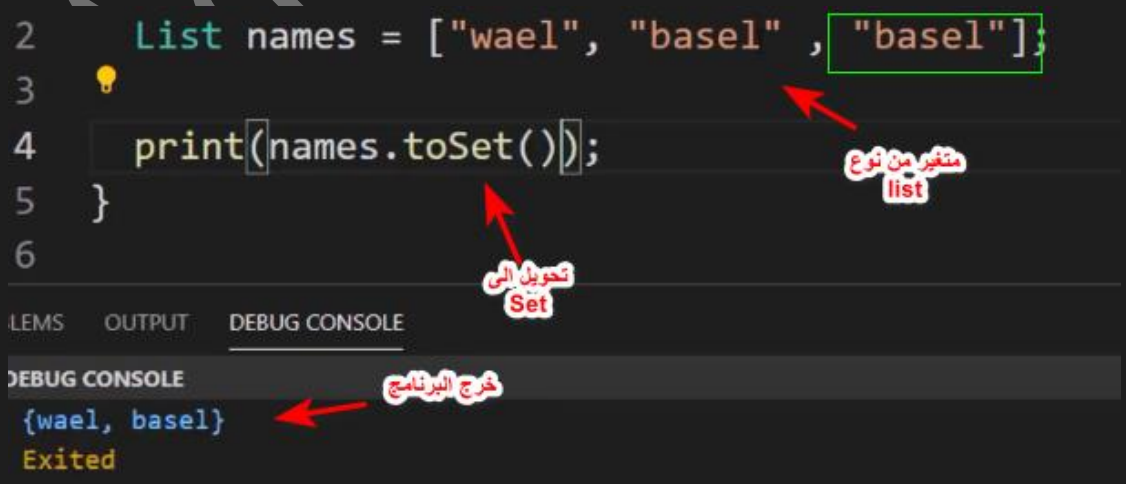
Convert between List Map Set

تحويل Set الى list

كما هو واضح في المثال في الأسفل :



تحويل list الى Set نلاحظ عدد تكرار قيمة basel كما سبق وذكرنا ان Set تتميز بعدم التكرار على عكس List و Map كما هو واضح في المثال في الأسفل :



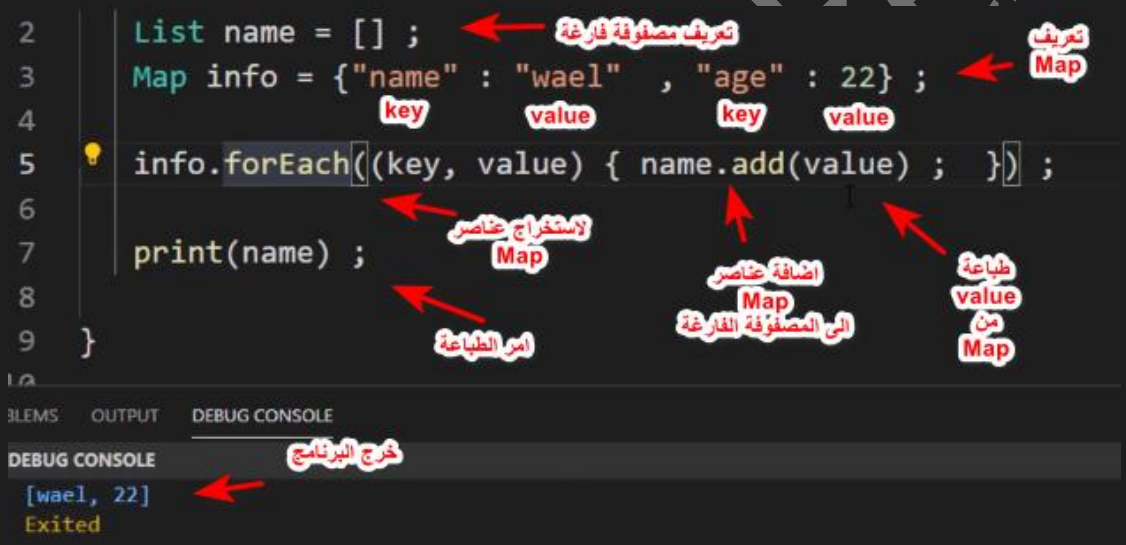
تحويل list او Set الى Map

لنقوم بشرح المثال الذي في الأسفل :

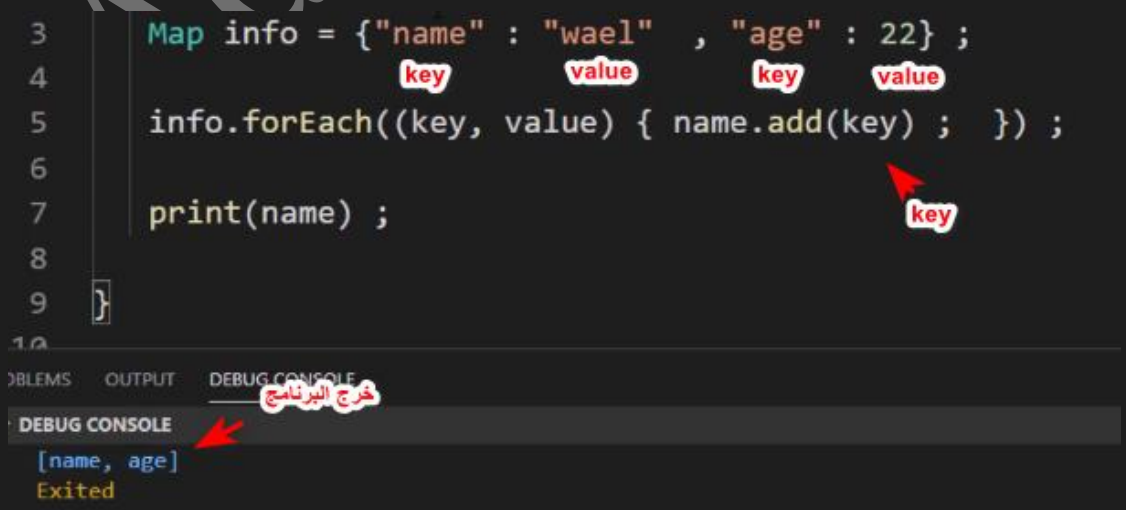
لقد قمنا بتعريف List فارغة وقمنا بأعطائها اسم name

قمنا بتعريف Map واضافة عناصر فيها

الان نريد اجراء عملية لتفريغ عناصر Map وجعلها في List نقوم بأستخدام forEach نقوم بأدخال اسم المصفوفة فارغة ثم Add ونقوم بتحديد نوع العناصر المراد جلبها من Map سواء كان value او Key .



في حال اردنا طباعة عناصر key من Map



Function

بفرض اننا قمنا بكتابة مجموعة من الاكواد والاسطر البرمجية ونريد استخدامها في اكثر من مكان في برنامجنا هل من المعقول ان نقوم بتكرار نفس الأوامر في اكثر من مكان في برنامجنا.

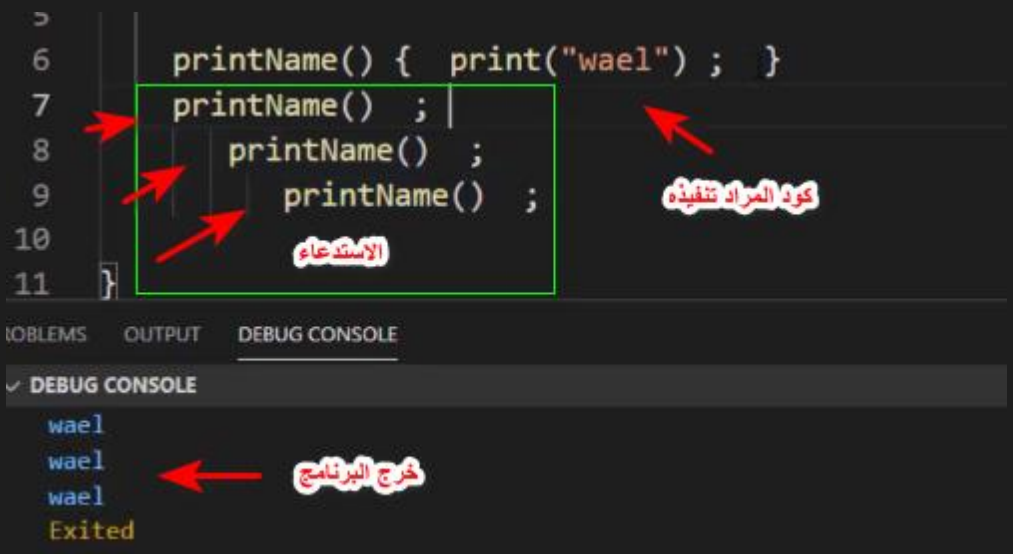
من هنا تأتي أهمية ***Function*** في لغات البرمجة اذ اننا نستطيع كتابة الكود مرة واحد وضعه في ***Function*** واستدعائها عند الحاجة.

التصريح او ابسط طريقة لكتابة ***Function*** في لغة البرمجة dart.

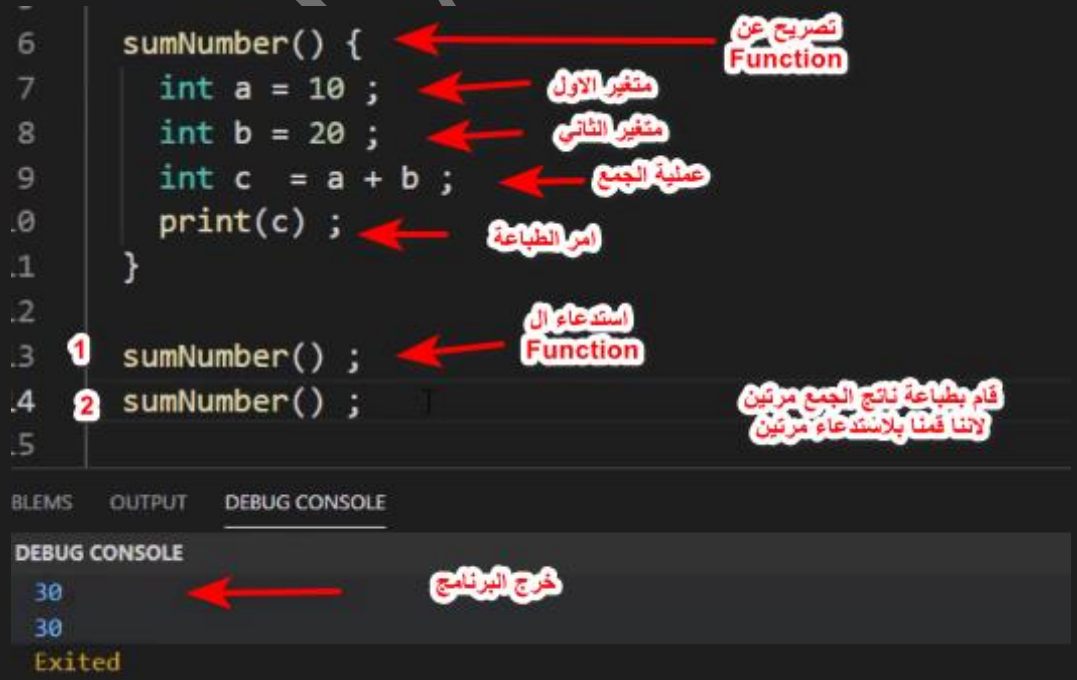


Ahmed Abo Baker

كما هو واضح في المثال البسيط في الأسفل نريد طباعة اسم wael قمنا بلتصريح عن **Function** وكتابة امر طباعة في داخلها وعند استدعائدها يتم تنفيذ الكود بعدد المرات التي يقوم فيها المبرمج بستدعائها كما هو واضح في المثال في الأسفل:



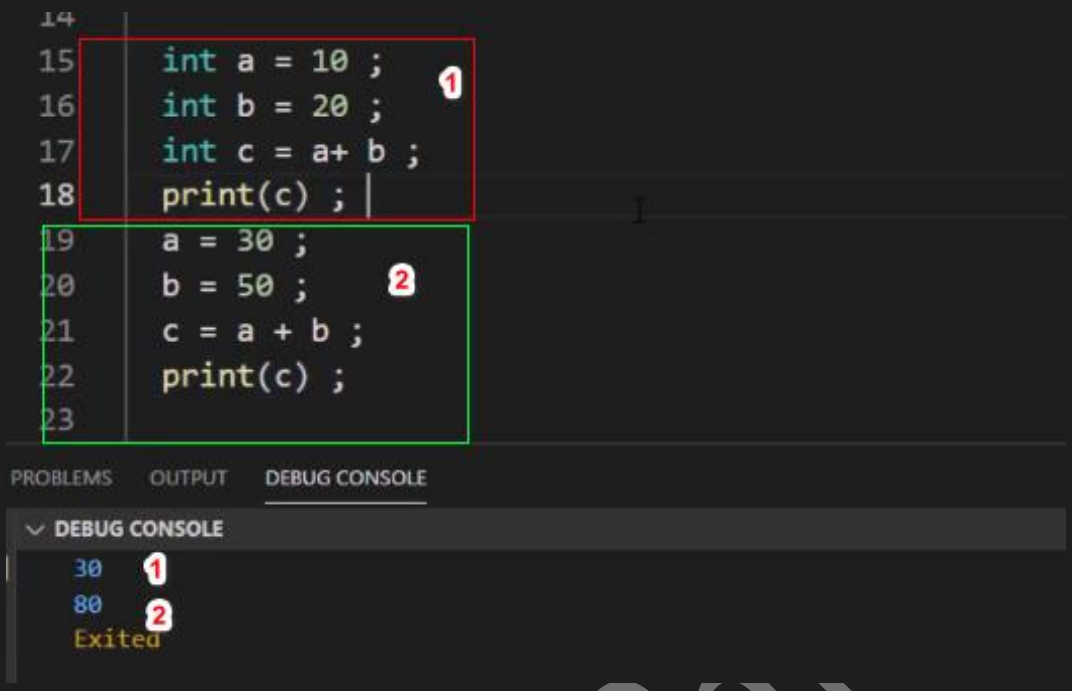
وأيضا لدينا المثال نريد ان نقوم بطباعة ناتج جمع عددين صحيحين :



الان لنقوم بأخذ مثال قوي نستطيع من خلاله فهم فائدة **Function** في لغات البرمجة من خلال برنامج طباعة ناتج جمع عددين ولكن هذه المرة من دون إعطاء قيم للمتغيرات قمنا بكتابة المتغيرات داخل **Function** ولكن على شكل برامترات لتقوم بأدخال قيم مختلفة عند الاستدعاء ثم قمنا بكتابة عملية جمع عددين داخل **Function** وفي الاستدعاء الأول قمنا بأدخال ١٠ و ٢٠ وناتج جمع هذين العددين هو ٣٠ كما هو واضح نفس الامر بلنسبة للاستدعاء الثاني بلقيم المختلفة الذي يحويها وناتج الجمع هو ٥٠ ونستطيع كتابة عدد لا نهائي من الاستدعاء.



بهذا نكون قد قمنا بتوفير وقت وجهد كبير بدلا من تكرار الكود كما هو واضح :


int a = 10 ;
int b = 20 ;
int c = a+ b ;
print(c) ;
a = 30 ;
b = 50 ;
c = a + b ;
print(c) ;
1
2
PROBLEMS
OUTPUT
DEBUG CONSOLE
DEBUG CONSOLE
30
80
Exited
1
2

Adel AboBa

Function types

تنقسم أنواع **Function** الى :

Void لا يرجع قيمة

Return يرجع قيمة

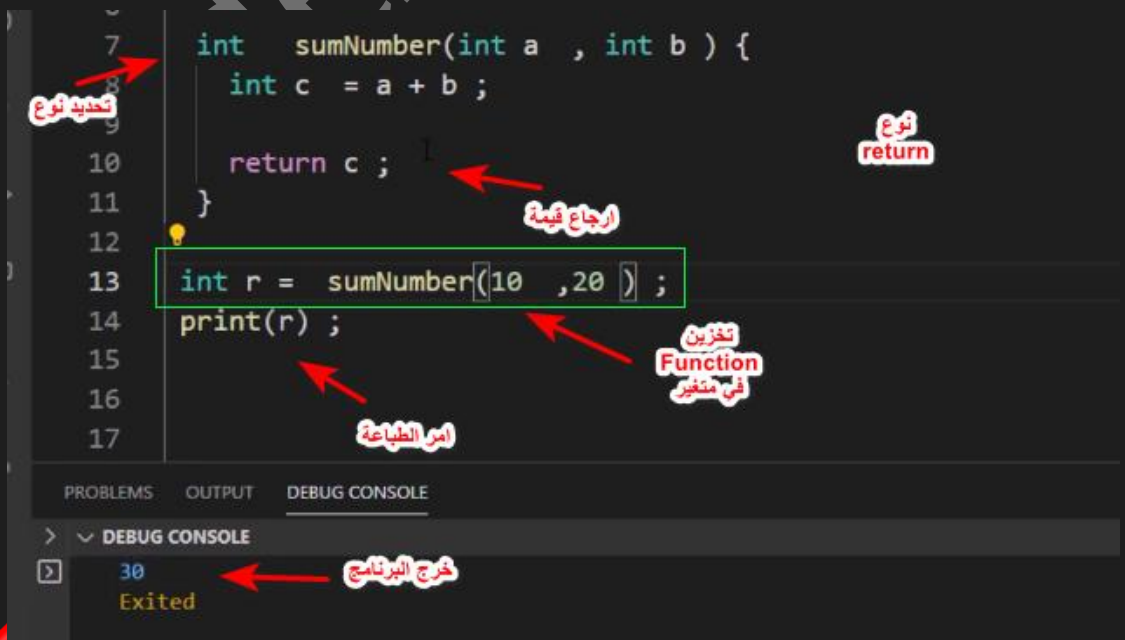
ما هو الفرق الجوهري بين void و return..؟؟

نوع void (لا يرجع قيمة) أي انه يقوم فقط بتنفيذ الكود داخل **Function** فقط.

اما return (يرجع قيمة) يقوم بأرجاع قيمة قمت بتحديدها حتى أقوم بتسجيلها في متغير.

جميع الأمثلة السابقة كانت **Function** من نوع void.

اما المثال الذي في الأسفل هو من نوع return قمنا فقط بكتابة كلمة return داخل Function وتحديد نوع القيمة الذي سيقوم بأرجاعها ثم تخزين اسم Function مع البرامترات في متغير اسمه r وقمنا بطباعة هذا المتغير كما هو واضح في الصورة في الأسفل :



Ola AboBaker

يمكننا ارجاع أي قيمة في Function من نوع return قمنا بأرجاع قيمة متغير a وتخزينه في متغير اخر وطباعة المتغير كما هو واضح في المثال في الأسفل :

```
7   int    sumNumber(int a  , int b ) {
8     int c  = a + b ;
9   1 print(c) ;
10    return a ;
11    }
12
13  int r =  sumNumber(10  ,20 ) ;
14  2 print(r) ;
15
16
17
```

ارجاع قيمة a

int a — int b

a =10

```
ROBLEMS   OUTPUT   DEBUG CONSOLE
DEBUG CONSOLE
  30  1
  10  2
  Exited
```

خرج البرنامج

في حال كانت Function من نوع void وقمنا بتخزين قيمة معينة داخل متغير من Function سيقوم البرنامج بأظهار رسالة خطأ لانه لا يمكن ان يعيد قيمة ويخزنها كما هو واضح في المثال في الأسفل :

```
lib > main.dart > ma
6
7   void | su
8     int c
9     print(
11    }
12
13  int r =  sumNumber(10  ,20 ) ;
14  print(r) ;
15
16
17
```

```
void sumNumber(int a, int b)
This expression has a type of 'void' so its value can't be used.
Try checking to see if you're using the correct API; there might be a
function or call that returns void you didn't expect. Also check type
parameters and variables which might also be
void. dart(use_of_void_result)
Peek Problem (Alt+F8)   No quick fixes available
```

رسالة الخطأ

لان نوع Function (void)

لا يمكن تخزين Function داخل متغير

لنقوم الان بتوضيح بعض الأمور عن ***Function*** وعلى أي أساس يتم تحديد نوعها..!!

يتم تحديد نوعها على حسب الكود المكتوب بداخل ***Function***

في حال لم يتم كتابة Return داخل ***Function*** مع الكود سيتم اعتبارها من نوع Void او اذا تم كتابة Return في الكود داخل ***Function*** يتم اعتبارها من النوع الثاني وهو Return.

في حال لم يتم تحديد نوع القيمة التي سوف يقوم بأرجاعها فأن الحاسوب سيقوم بفحص نوع القيمة بشكل اتوماتيكي

قام بتحديد نوع Function انه من نوع int بناء على قيمة التي سوف يرجها وكما هو الواضح فأن المتغير a قيمته عدد صحيح من نوع int كما هو واضح في المثال الذي في الأسفل :

```
int sumNumber(int a, int b)
sumNumber(int a  , int b ) {
  int c  = a + b ;
  print(c) ;
  return a ;
}
```

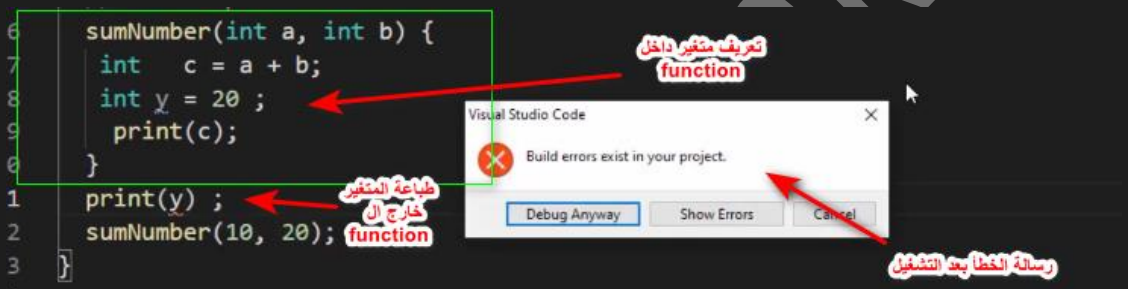


AdelBoBaker

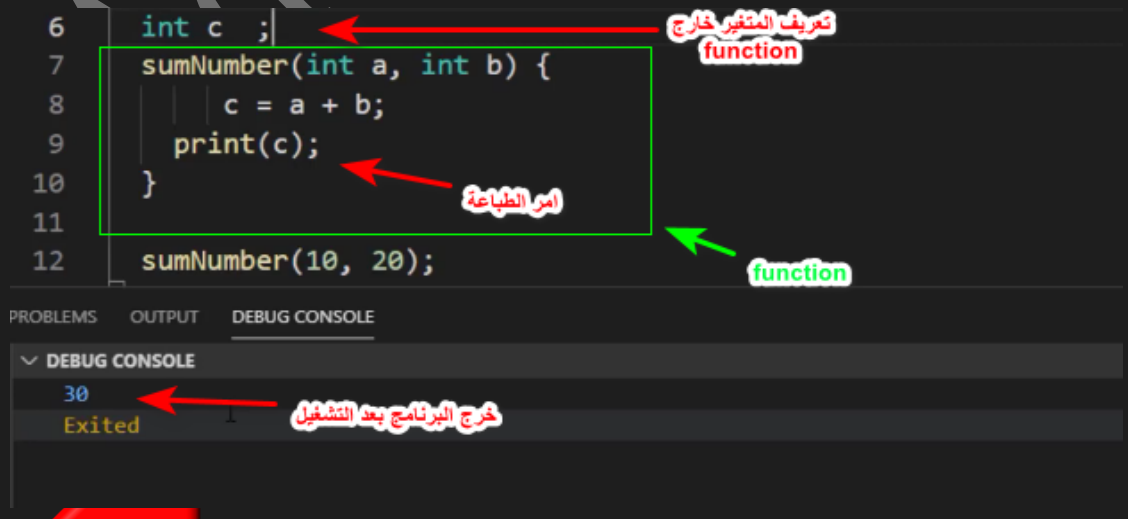
Function scope

مفهوم Scope يعبر عن مدى إمكانية الوصول لل varible.

في المثال الموضح في الأسفل لدينا function ينقوم بتعريف متغير داخلها واعطائه قيمة ٢٠ ثم ننتقل الى خارج function لطباعة هذا المتغير وعند تشغيل البرنامج سيقوم الحاسوب بأظهار رسالة خطأ هذا يعني اذا قمنا بتعريف متغير داخل function نستطيع استخدامه فقط داخل function التي قمنا بتعريفه بداخلها.



لدينا حالة أخرى للمتغير : لو قمنا بتعريف المتغير خارج function وقمنا بتنفيذ امر الطباعة من داخل ال function سيعمل البرنامج من دون أي برامج كما هو واضح في الصورة في الأسفل :



في حال قمنا بتعريف function خارج ()Void Main ستعمل من دون أي مشاكل كما هو واضح في المثال في الأسفل :

```
8   sumNumber(int a, int b) {
9       int  c = a + b;
10    print(c);
11  }
12
```

قمنا بتعريف function خارج ال Void Main{}

```
OBLEMS  OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
  30
  Exited
```

خرج البرنامج

نستطيع أيضا تعريف المتغيرات خرج ()Void Main واستخدام هذا المتغير في أي function في البرنامج كما هو واضح في المثال في الأسفل :

```
1   int c;
    Run | Debug
2   void main() {
3     // 1 -  namefunction ( argument  ) { //code  }
4     // 2 -  namefunction () ;
5     // 3 -  type function a  - void   b - return
6     // 4 - scope
7     c = 500  ;
8     print(c) ;
9     sumNumber(10, 20);
10  }
11
12  sumNumber(int a, int b) {
```

تعريف متغير خارج Void Main()

تنفيذ امر الطباعة من داخل Void Main()

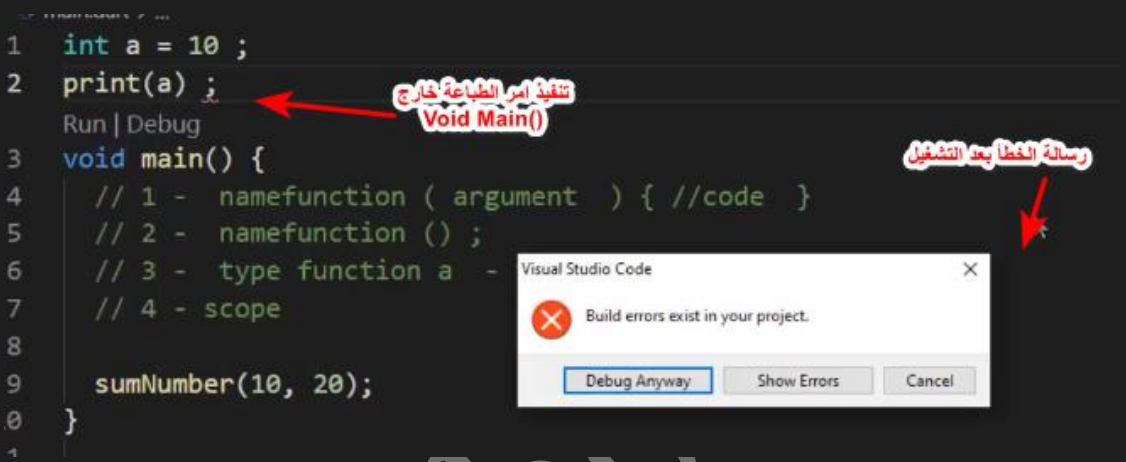
Void Main()

```
PROBLEMS  OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
  500
  30
  Exited
```

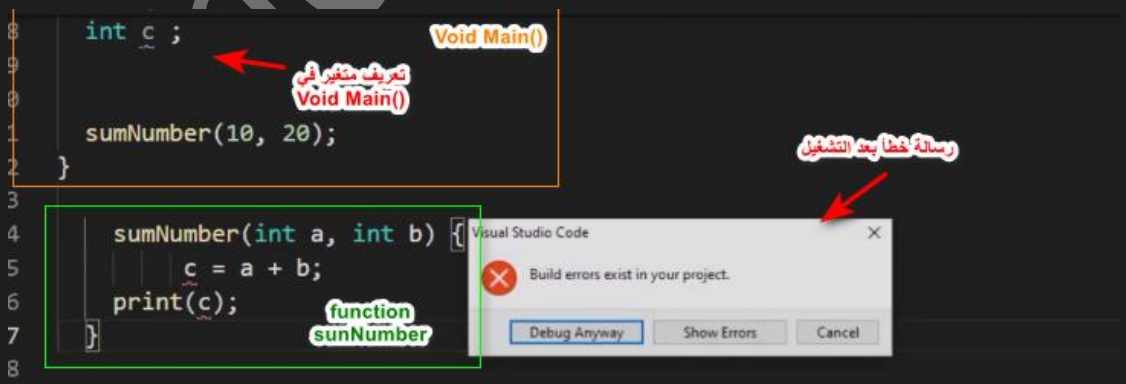
خرج البرنامج

ملاحظة : ()Void Main هي function معرفة مسبقا في لغة Dart.

لا نستطيع تنفيذ أي امر خارج ()Void Main بلمختصر نستطيع فقط تعريف المتغيرات في الخارج كما هو واضح في الصورة في الأسفل :

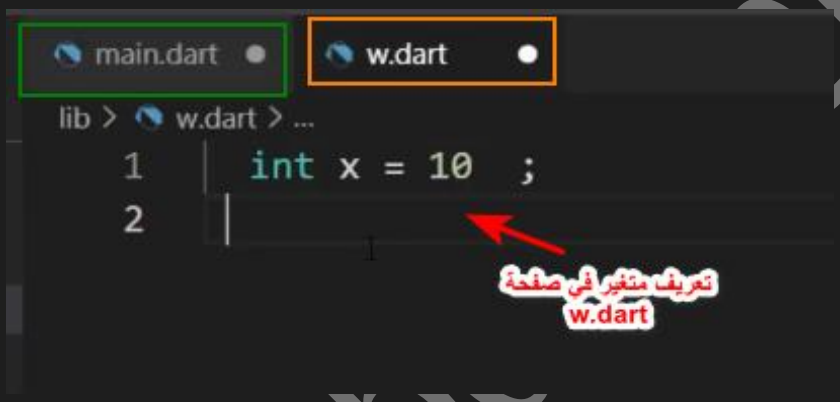


في حال كان لدينا function اسمها sumNumber خارج ()Void Main وقمنا بتعريف متغير ما داخل ()Void Main وقمنا بتنفيذ امر الطباعة داخل هذه function سيقوم بأظهار رسالة خطأ كما هو واضح في المثال في الأسفل :

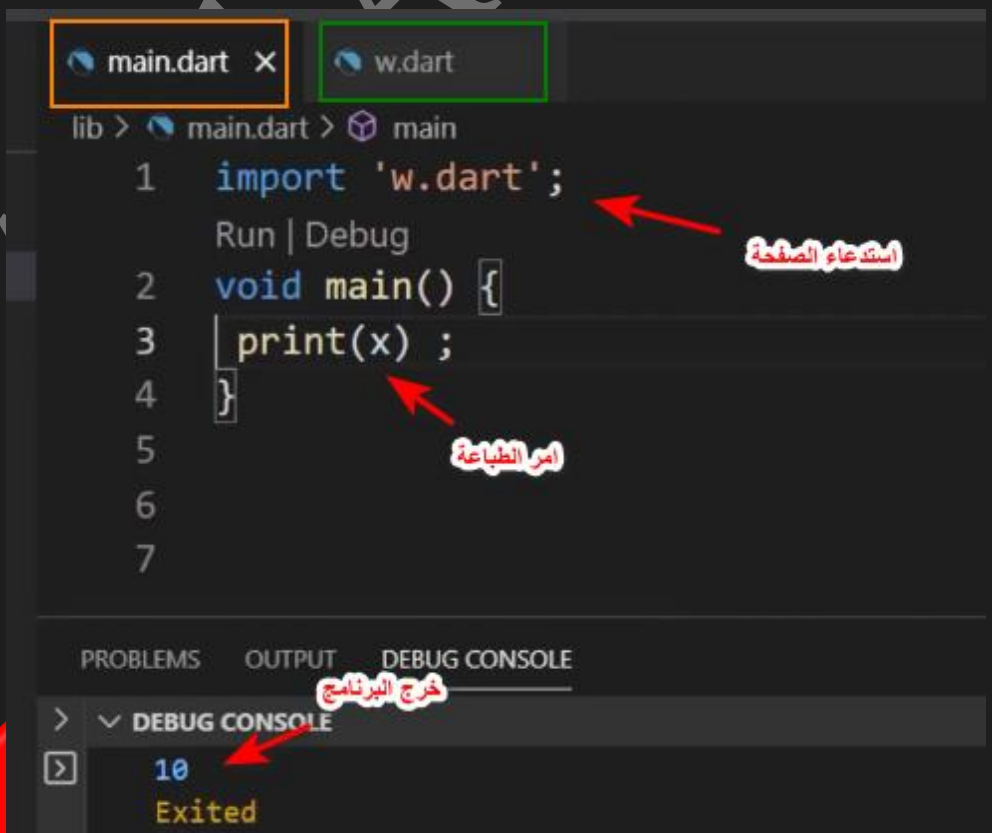


import

والمقصود به وهو الاستدعاء سنقوم بمثال بسيط عن الاستدعاء سنقوم بعمل ملف باسم w.dart وسنقوم بتعريف متغير فيه ونسميه x ونعطيه قيمة 10 كما هو واضح في المثال في الأسفل:



ثم نتوجه الى صفحة main.dart ونقوم بكتابه الاستدعاء خارج Void Main()



المسارات

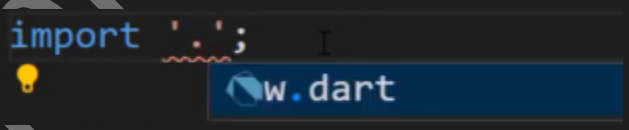
في حال اردنا الدخول الى مجلد معين نقوم بكتابة import ثم نقوم بعمل دوبل كوتشن " " ونكتب بداخلها /. بهذا الشكل :

```
import 'w.dart';
```

او في حال اردنا الخروج او رجوع للخلف من مجلد معين نفس العملية تماما نكتب داخل دوبل كوتشن " " ونكتب بداخلها /.. بهذا الشكل :

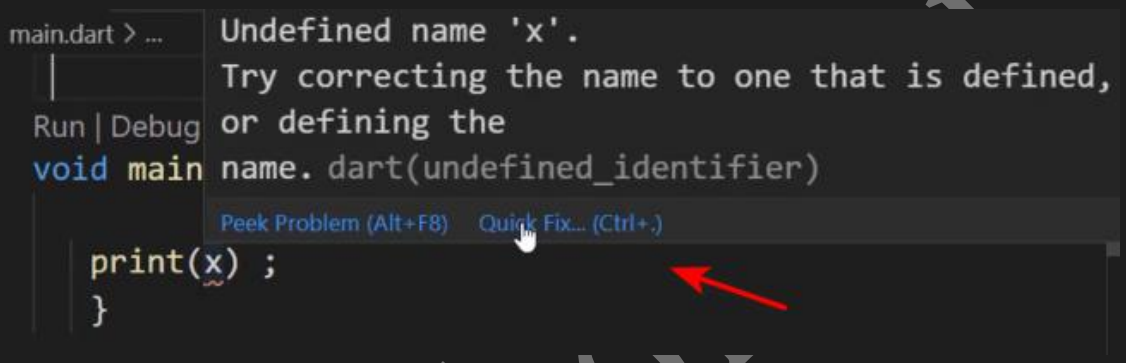
```
import '../w.dart';
```

لعرض محتويات المجلد نقوم بكتابة نقطة داخل دوبل كوتشن " " بهذا الشكل :

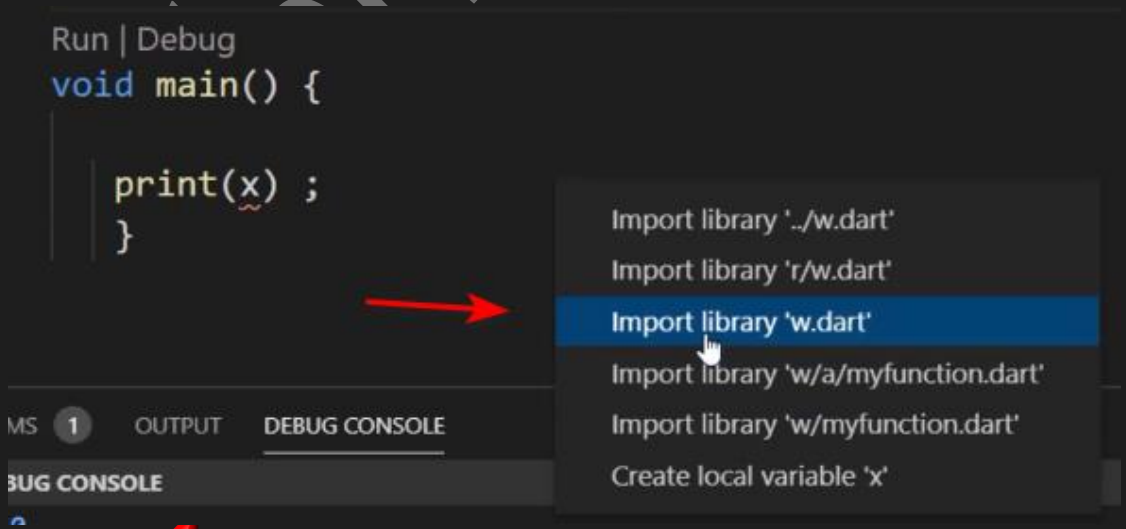


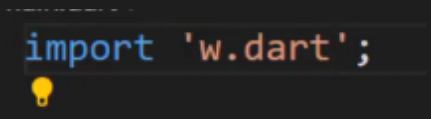
Adel Al-Baker

يوجد طريقة اسهل للاستدعاء نستطيع من خلالها أيضا استدعاء الصفحة المعرف فيها المتغير من خلال الوقوف فوق المتغير ونقوم بلضغط على Quick Fix كما هو ظاهر في الأسفل :



ثم نجد اسم الملف الذي يوجد به المتغير الذي قمنا بتعريفه نضغط عليه كما هو واضح في الصورة في الأسفل :





Runes

codeUnits تحويل الى ترميز في هذا المثال المعروض في الأسفل نلاحظ خرج البرنامج عبارة عن list كلها ارقام صحيحة اذ أن كل حرف او علامة او حتى فراغ له ترميز معين محفوظ في جهاز الحاسوب مثل الحرف w ترميزه ١١٩ والحرف a ترميزه ٩٧ كما هو واضح في المثال في الأسفل :

```
4   String name = "wael";
5   print(name.codeUnits) ;
6
7
8   }
9
```

تحويل الاحرف الى ترميز

```
ILEMS   OUTPUT   DEBUG CONSOLE
DEBUG CONSOLE
[119, 97, 101, 108]
Exited
```

خرج البرنامج

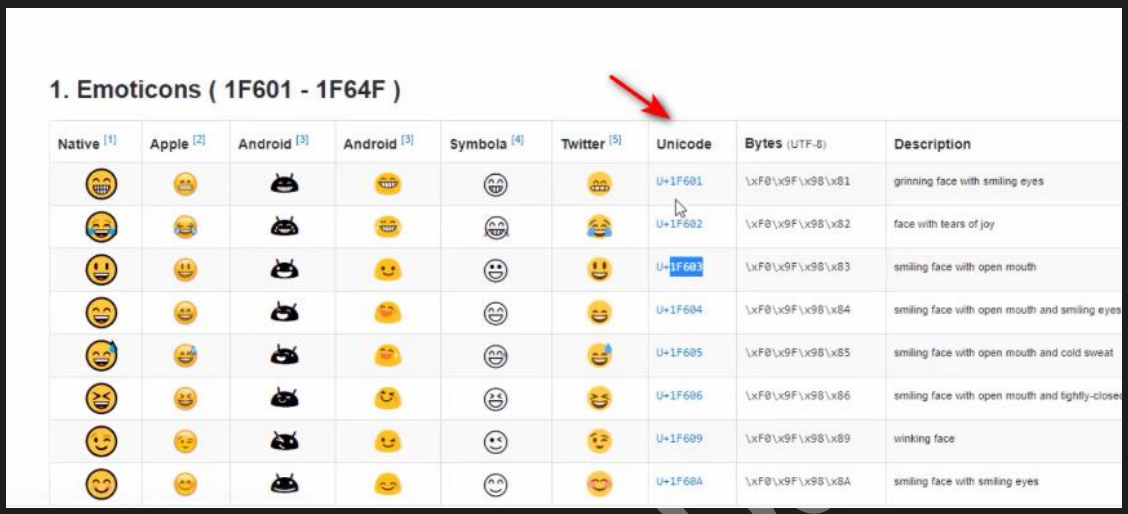
```
4   String name = "wael" ;
5
5   print(name.runes) ;
7
3
9   }
9
```

تحويل الى ترميز

```
LEMS   OUTPUT   DEBUG CONSOLE
EBUG CONSOLE
(119, 97, 101, 108)
Exited
```

خرج البرنامج

نستطيع الاستفادة من codeUnits في اخذ ترميز ايموجي معين واستخدامه



1. Emoticons (1F601 - 1F64F)

Native [1]	Apple [2]	Android [3]	Android [3]	Symbola [4]	Twitter [5]	Unicode	Bytes (UTF-8)	Description
😁	😁	😁	😁	😁	😁	U+1F601	\xF0\x9F\x98\x81	grinning face with smiling eyes
😂	😂	😂	😂	😂	😂	U+1F602	\xF0\x9F\x98\x82	face with tears of joy
😃	😃	😃	😃	😃	😃	U+1F603	\xF0\x9F\x98\x83	smiling face with open mouth
😄	😄	😄	😄	😄	😄	U+1F604	\xF0\x9F\x98\x84	smiling face with open mouth and smiling eyes
😅	😅	😅	😅	😅	😅	U+1F605	\xF0\x9F\x98\x85	smiling face with open mouth and cold sweat
😆	😆	😆	😆	😆	😆	U+1F606	\xF0\x9F\x98\x86	smiling face with open mouth and tightly-closed
😉	😉	😉	😉	😉	😉	U+1F609	\xF0\x9F\x98\x89	winking face
😊	😊	😊	😊	😊	😊	U+1F60A	\xF0\x9F\x98\x8A	smiling face with smiling eyes

كما هو واضح في المثال في الأسفل :

```
3   String emoji = "\u{1F601}" ;
4   print(emoji) ;
5
6
7
8   }
9
```

ترميز الايموجي

BLEMS OUTPUT DEBUG CONSOLE

خرج البرنامج

DEBUG CONSOLE

😁

Exited

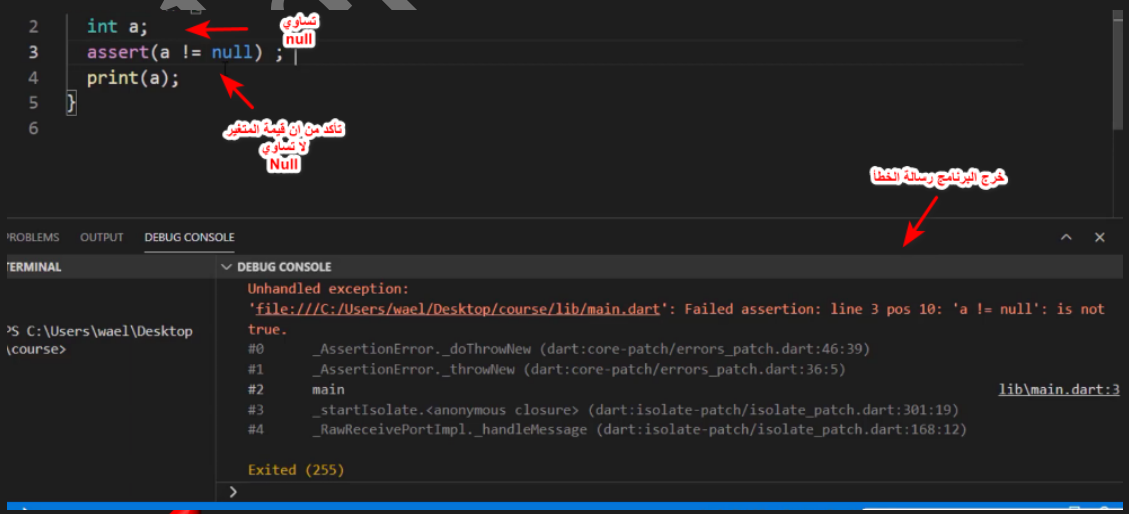
Assert

ليكن لدينا متغير ما في برنامجنا واردنا تأكد من ان قيمته لا تساوي null

هنا نلجأ الى مفهوم assert والتي تستخدم من قبل المبرمجين وتتألف syntax الخاص بها من جزئين الأول هو كتابة الشرط والثاني الرسالة المراد عرضها في debug console كمل هو واضح في الصورة في الأسفل :



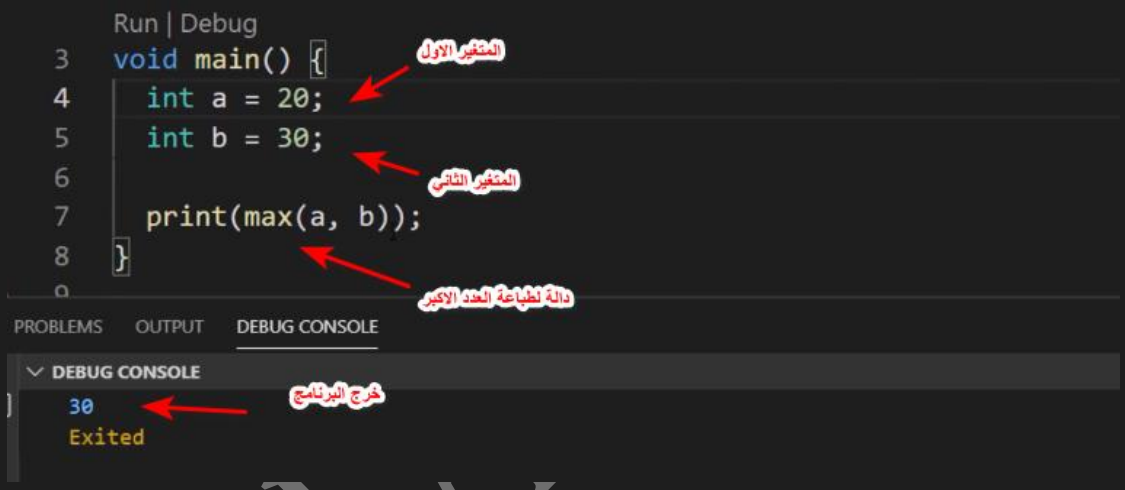
مثال حي عن Assert كما هو واضح ان قيمة المتغير a تساوي null وعند طباعة قام بأظهار رسالة الخطأ



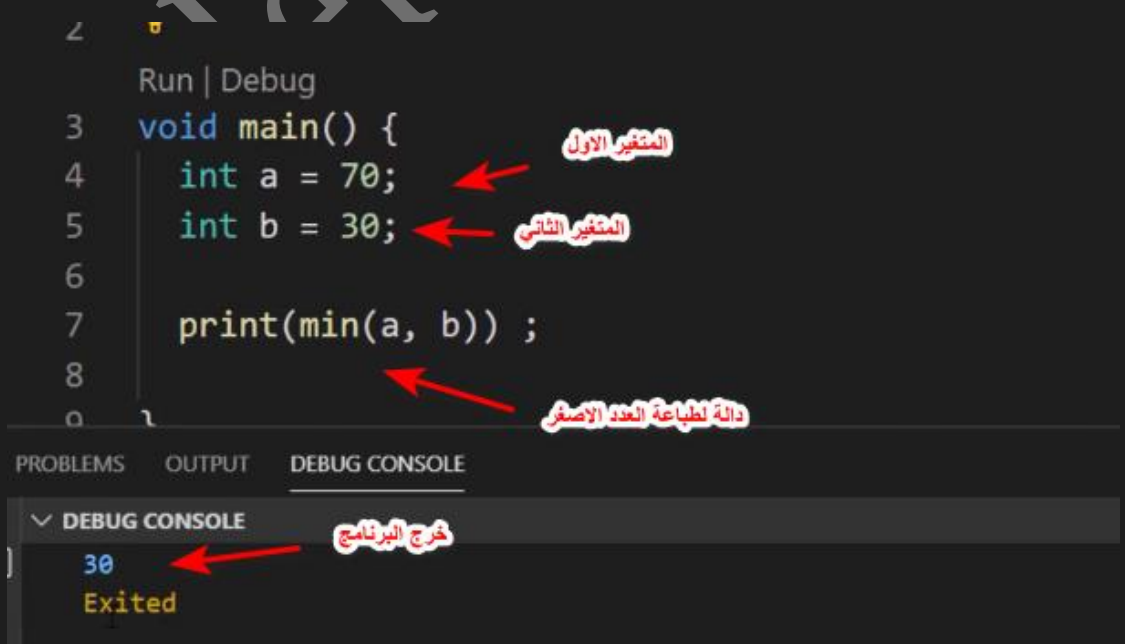
Max And Min

هي function جاهزة من مكتبة math في لغة dart تستخدم لطباعة العدد الأكبر او الأصغر.

مثال عن طباعة العدد الأكبر كما هو واضح في المثال في الأسفل :



مثال عن طباعة العدد الأصغر كما هو واضح في المثال في الأسفل :



DataType

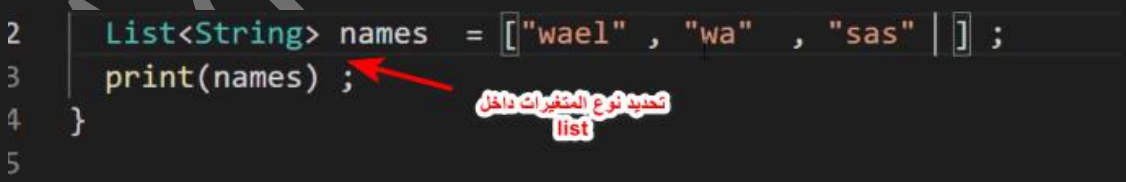
(List - Set - Map)

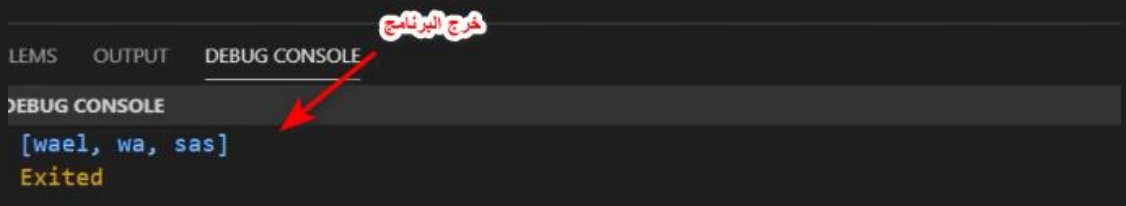
فيما سبق قمنا بشرح أنواع المتغيرات ولكن في هذا الدرس سنتعمق بدراسة أنواع المتغيرات والحالات التي نستطيع اجرائها عليها.

كما نعلم ان list تقبل مجموعة من أنواع مختلفة من متغيرات وتكون dynamic كما هو واضح في المثال في الأسفل :



نستطيع تحديد نوع المتغير داخل list ولنفرض انه يكون من نوع string بهذه الحالة لا نستطيع الا كتابة قيم من نوع string داخل list كما هو واضح في المثال في الأسفل :





abdelhakem

نلاحظ ان لدينا في هذا المثال list قمنا بحديد القيمة التي يجب فقط ان تحويها بداخلها وهي Map.

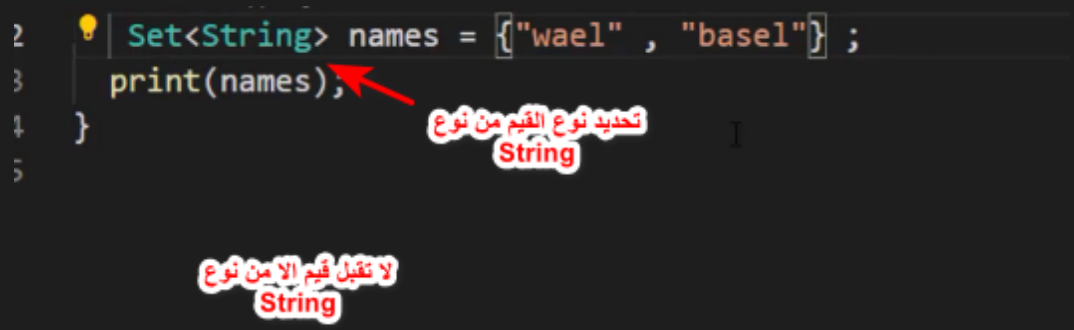
بطريقة متقدمة اكثر نستطيع تحديد نوع القيم داخل Map لكل من key و value

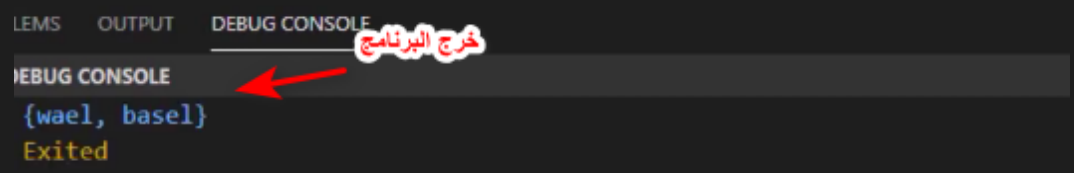
كما هو واضح في المثال في الأسفل ان كل من قيمة key و value هي من نوع String :



مثل هذه الطريقة تصلح لكل من list و Map و Set كما هو واضح :

Adel A





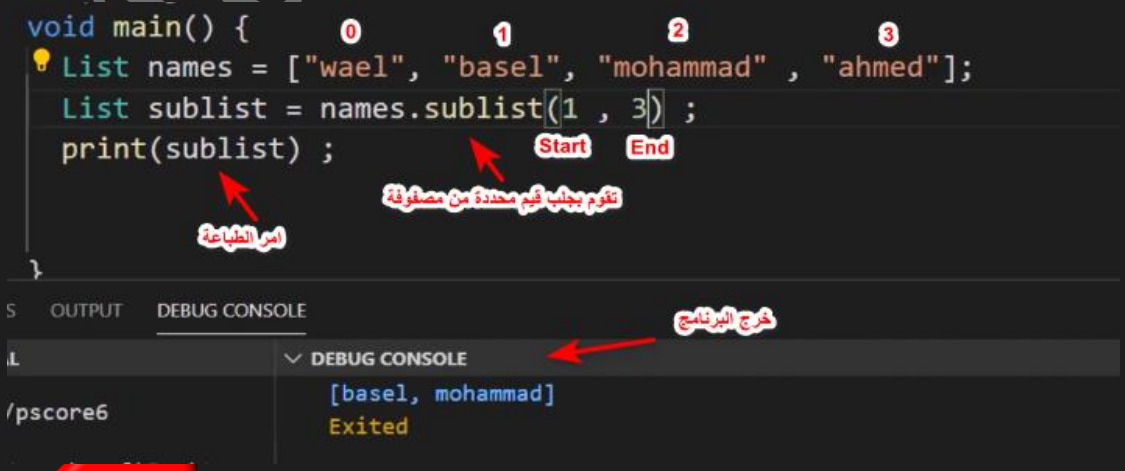
(FirstWhere - AsMap - WhereType)

Sublist تقوم بجلب جزء نقوم بتحديده من list

تبدأ بي (Start)

وتنتهي بي (End نقوم بتحديده ولكن لا يقوم البرنامج بطباعته)

كما هو واضح في المثال في الأسفل :



Shuffle تقوم بأعادة تلرتيب عناصر list في كل مرة نقوم بأعادة تشغيل البرنامج كما هو واضح في المثال في الأسفل :

```
List names = ["wael", "basel", "mohammad" , "ahmed"];
names.shuffle() ;
print(names) ;
```

تقوم بأعادة ترتيب عناصر المصفوفة

```
OUTPUT   DEBUG CONSOLE
DEBUG CONSOLE
[ahmed, mohammad, basel, wael]
Exited
```

خرج البرنامج

asMap تحويل list الى Map كما هو واضح في المثال في الأسفل :

```
List names = ["wael", "basel", "mohammad" , "ahmed"];
Map namesmap = names.asMap() ;
print(namesmap) ;
```

امر الطباعة — اسم المصفوفة — تحويل مصفوفة الى Map

```
OUTPUT   DEBUG CONSOLE
DEBUG CONSOLE
{0: wael, 1: basel, 2: mohammad, 3: ahmed}
Exited
```

خرج البرنامج

whereType ارجاع مجموعة من قيم المحددة من list مع تحديد نوع هذه القيم سواء كانت string او int من مختلف أنواع المتغيرات كما هو ظاهر المثال في الأسفل :

```dart
void main() {
  List names = ["wael", 100 , "basel", "mohammad" , 22 , "ahmed" , 1002 ,
  var namesmap = names.whereType<String>();
  print(namesmap) ;
}
```

اسم list — ارجاع مجموعة قيم محددة من list — تحديد نوع القيم

```
OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
(wael, basel, mohammad, ahmed)
Exited
```

خرج البرنامج

firstWhere تقوم بفحص اول قيمة محققة للشرط وتقوم بتخزينها في المتغير الذي نقوم بتعريفه كما هو واضح في الصورة :

```dart
List names = [100 , 2 , 4 , 6 , 10];
var namesmap = names.firstWhere((n) => n > 10 );
print(namesmap) ;
}
```

اسم List — يقوم بفحص اول قيمة محققة للشرط — return مختصرة — الشرط

```
OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
100
Exited
```

خرج البرنامج

نستطيع استخدمها أيضا مع متغيرات من نوع string كما هو واضح في المثال في الأسفل :

```dart
List names = ["wael" , "basel" , "g"];
var namesmap = names.firstWhere((n) => n.length > 4 );
print(namesmap) ;
}
```

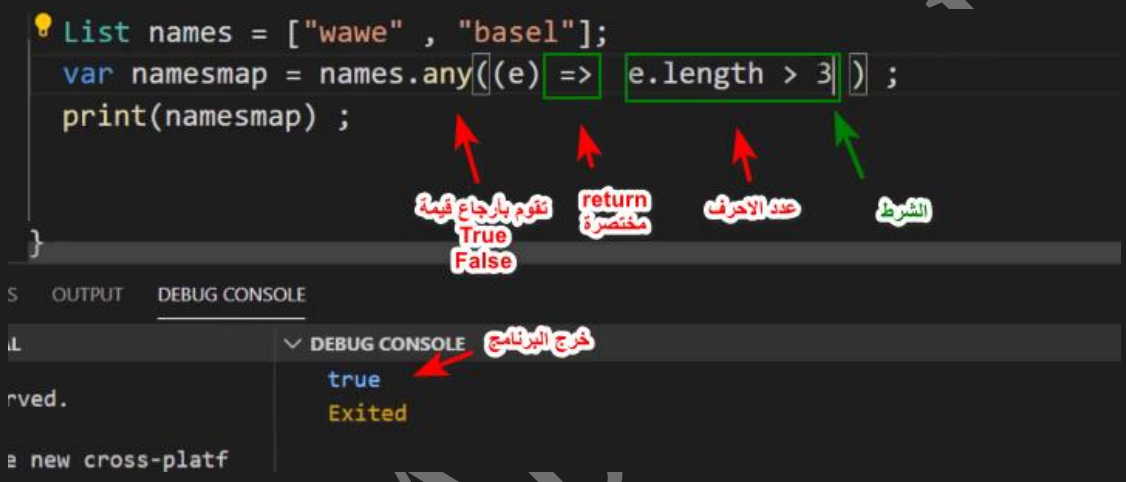
عدد الاحرف — الشرط

```
OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
basel
Exited
```

خرج البرنامج

Any تعني أي قيمة محققة للشرط وترجع قيمة اما true او false في المثال الموضح لدينا في الأسفل :

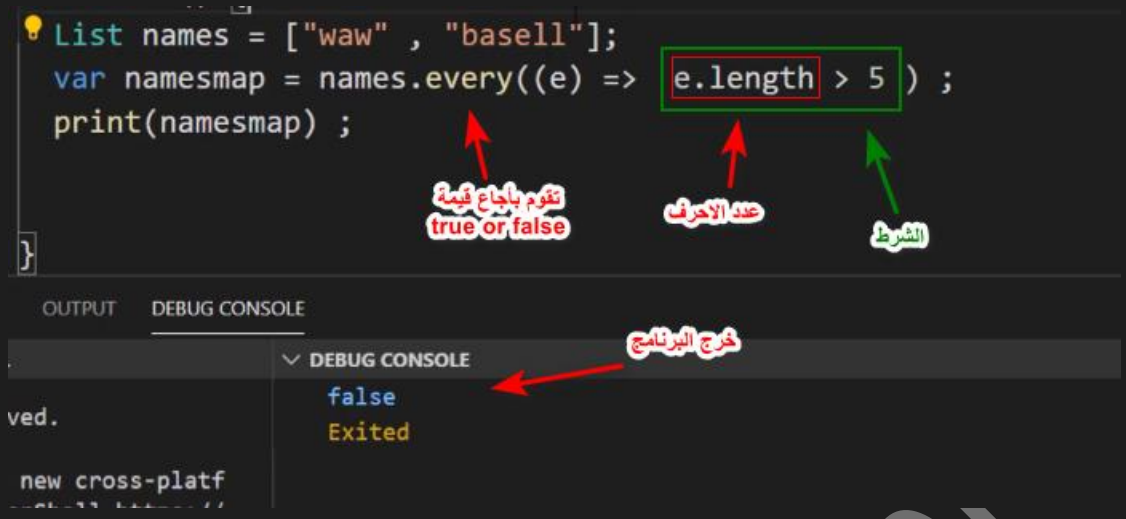
نريد تأكد من ان أي قيمة في list عدد احرفها اكبر من ٣ وبتأكيد فأن الشرط محقق ويكفي واحد من عناصر list ان يكون محقق للشرط حتى تقوم بأرجاع true.



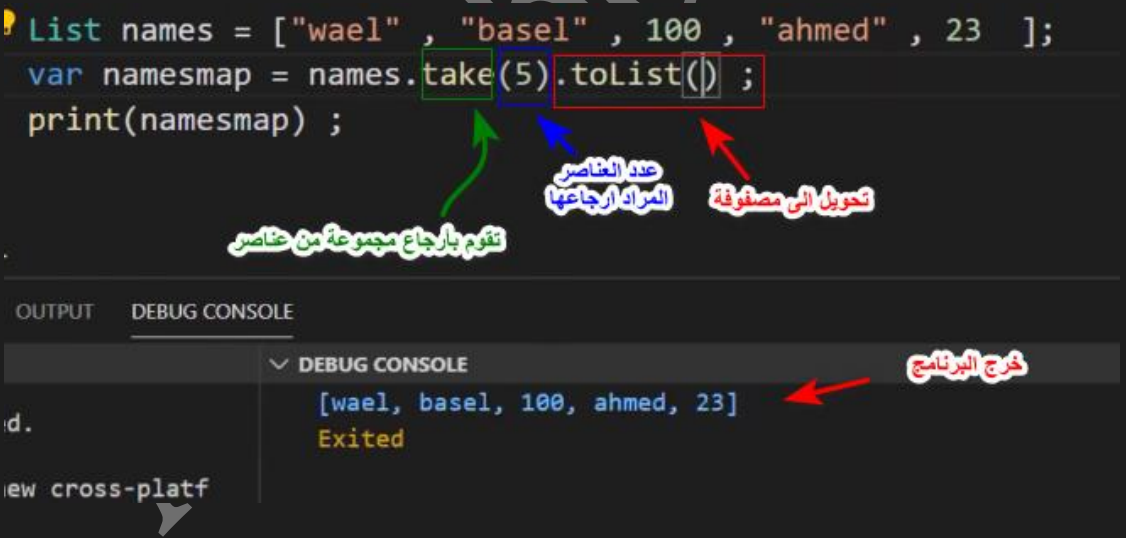
Every يجب ان يكون جميع عناصر list محققة للشرط حتى تقوم بأرجاع قيمة true والا ستقوم بأرجاع قيمة false

لدينا المثال في الأسفل نريد التأكد من ان جميع عناصر list عدد احرف كل عنصر فيها اكبر من ٥ عند التنفيذ سيقوم البرنامج بأرجاع قيمة false لا الشرط غير محقق لوجود كلمة تتألف من ٣ حروف فقط كما هو واضح في المثال :

Adel A



Take سقوم بأرجاع مجموعة محددة من عناصر مثلا نقوم بكتابة رقم ٥ بداخلها تقوم بأرجاع اول ٥ عناصر من list كما هو واضح في المثال في الأسفل :



Where تقوم بنفس عمل firstWhere ولكن الفرق بينهما ان firstWhere تقوم بأرجاع اول قيمة (قيمة واحدة فقط) من list محقق للشرط وتخزينها.

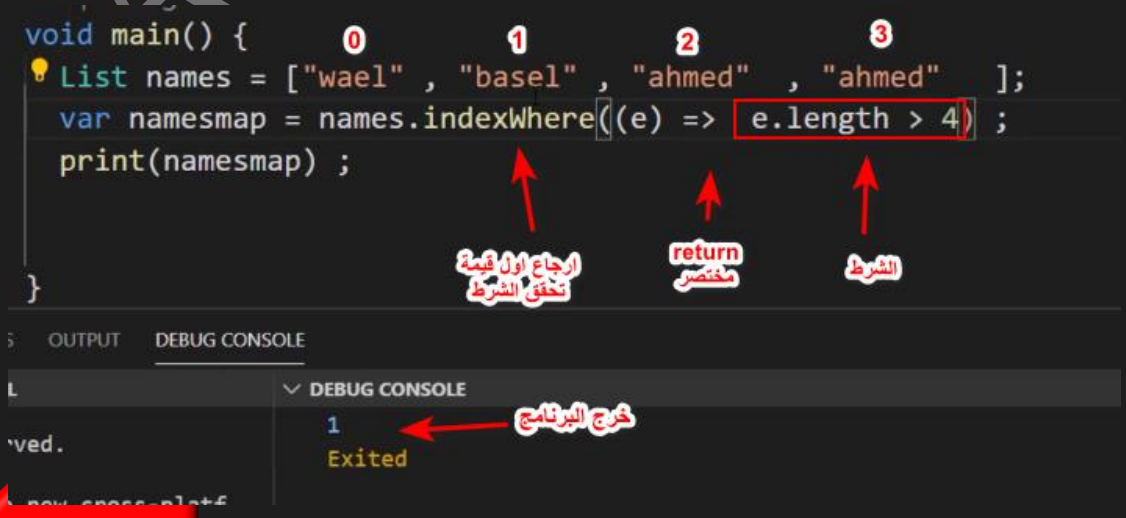
Where تقوم بأرجاع مجموعة من القيم من list المحققة للشرط وتخزينها على هيئة Iterable.

كما هو واضح في المثال في الأسفل :



indexWhere ارجاع اول قيمة محققة للشرط حيث يقوم بأرجاع رقم القيمة المحققة للشرط من list.

بمعنى لدينا مثال في الأسفل الشرط ان يكون عدد احرف العنصر اكبر من ٤ وعند فحص عناصر list واحد تلو الاخر نلاحظ ان اول قيمة محققة للشرط هي basel ورقمها يساوي ١ فيقوم البرنامج بطباعة الرقم ١ عند تنفيذ امر الطباعة.



Adel Abou

startsWith يقوم بفحص اول حرف من الكلمة في حال كان محقق يرجع قيمة true واذا كلن غير محقق يرجع قيمة false.

هل كلمة wael تبدأ بحرف W ..؟؟ اذن سيقوم بأرجاع قيمة true.

```
String name = "wael" ;

print(name.startsWith("w")) ;


}
```

يقوم بأرجاع قيمة True or false

```
OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
true
Exited
```

خرج البرنامج

Endswith يقوم بفحص اخر حرف من الكلمة في حال كان محقق يرجع قيمة true واذا كلن غير محقق يرجع قيمة false.

هل كلمة basel تنتهي بحرف W ..؟؟ اذن سيقوم بأرجاع قيمة false.

```
String name = "basel" ;

print(name.endsWith("w")) ;


}
```

ارجاع قيمة true or false

```
OUTPUT  DEBUG CONSOLE
DEBUG CONSOLE
false
Exited
```

خرج البرنامج

Contains يقوم بفحص كل حرف من الكلمة في حال كان محقق يرجع قيمة true واذا كلن غير محقق يرجع قيمة false.

هل كلمة wael يوجد فيها حرف a ..؟؟ اذن سيقوم بأرجاع قيمة true.

```
String name = "basel" ;

print(name.contains("a")) ;


}
```

```
S  OUTPUT  DEBUG CONSOLE                    خرج البرنامج
AL                          DEBUG CONSOLE
rved.                         true
                              Exited
```

indexOf يقوم بفحص كل حرف من الكلمة وارجاع قيمة موقع الحرف بلكلمة ونستطيع تحديد قيمة للبدأ منها.

كلمة baselle يوجد فيها حرفين (e) ولكن قمنا بحديد قيمة للبدأ منها وهي ٢ سيقوم احاسوب بتجاهل اول ثلاثة احرف من الكلمة كما هو واضح في المثال في الأسفل :

```
String name = "baselle" ;
                 0123456
print(name.indexOf("e", 2)) ;


}
```

سيقوم الحاسوب بتجاهل هذه الأحرف

ارجاع قيمة موقع الحرف بلكلمة

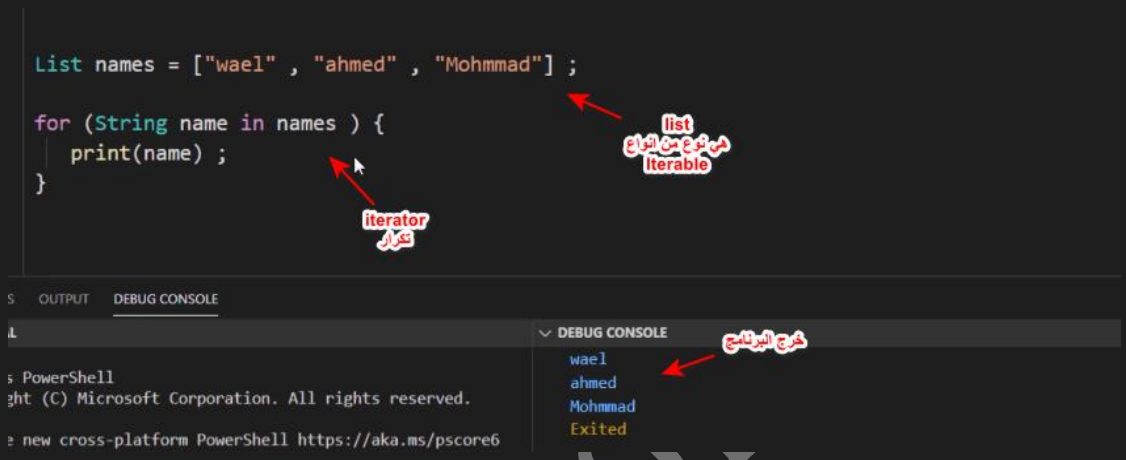
ابدأ التحقق من الحرف الثاني

```
OUTPUT  DEBUG CONSOLE
                          DEBUG CONSOLE      خرج البرنامج
ed.                         6
                            Exited
```

Adel Abo

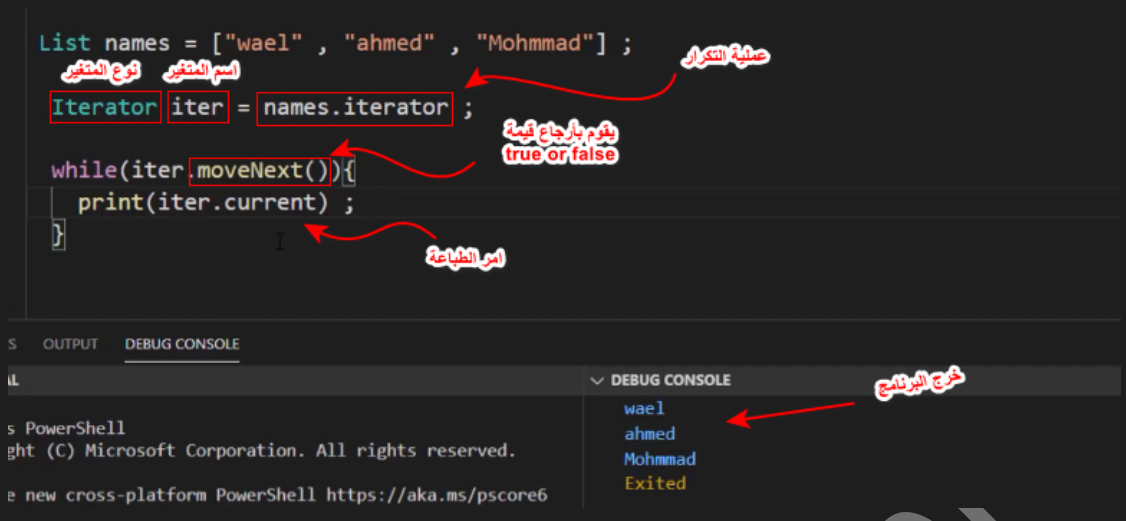
Iterator And Iterable

Iterable هو مفهوم عام يشمل جميع المتغيرات التي نستطيع عمل تكرار عليها ومن ضمنها List و Set ***Iterable*** هو الذي يمكن ان نعمل عليه iterator و iterator هنا تعني التكرار (تشمل جميع أنواع الحلقات التكرارية For او while).



ميثود التكرار ليكن لدينا متغير من نوع iterator نسميه iter ثم نقوم بكتابة حلقة تكرارية من نوع while ونقوم بكتابة الشرط بداخلها والذي يتضمن المتغير iter الذي سنقوم بعمل moveNext() عليه الذي بدوره يقوم بأرجاع اما true او false وسيبقى الشرط محقق حتى يصل لاخر عنصر يقوم بطباعته ليقوم بأعادة الفحص فيرجع قيمة false لانه قام بطباعة جميع عناصر list لا يوجد عنصر تالي ومن ثم يخرج البرنامج من الحلقة while كما هو واضح في المثال في الأسفل :

Adel Abu



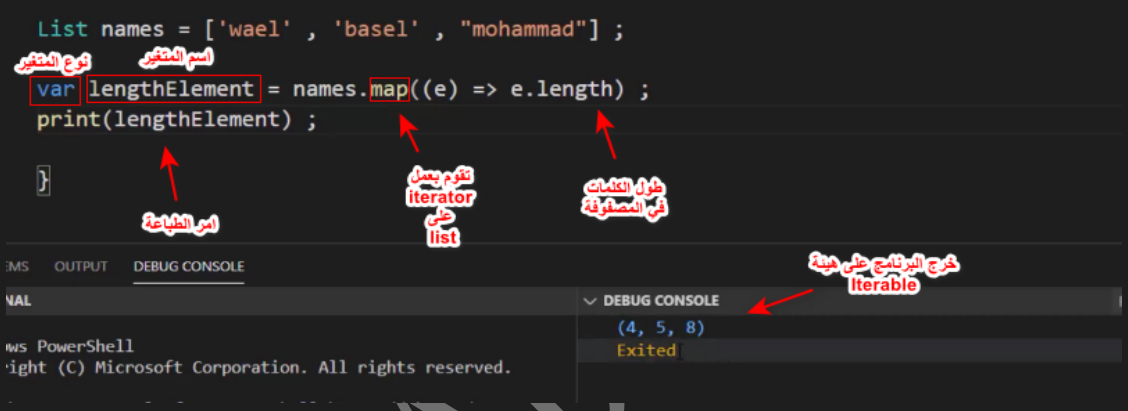
Adel AboBaker

Map Method

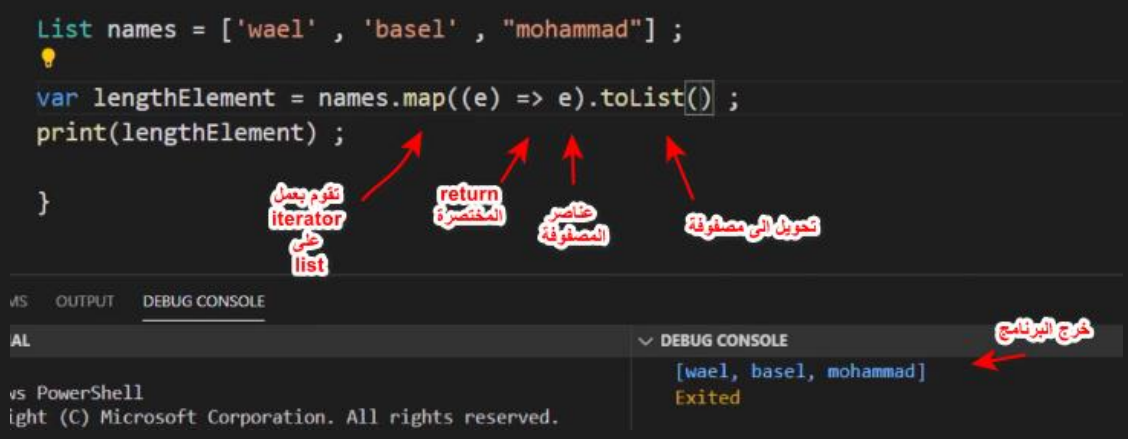
تقوم بعمل iterator على list او Set او Map وتقوم بتخزين عناصرها في متغير.

Map Method تقوم بأرجاع القيم على هيئة Iterable.

كما هو واضح في المثال في الأسفل طباعة طول الكلمات في list :



وأيضا طباعة الكلمات وتحويلها الى list كما هو واضح في المثال في الأسفل :



نستطيع عمل الكثير من العمليات على Map Method كمثال استخدام الدالة if الشرطية داخلها كما هو واضح في المثال في الأسفل :

```
List names = ['wael' , 'basel' , "mohammad"] ;

var lengthElement = names.map((e) {
  if (e == "wael") {
    return "Yes Wael Exist" ;
  }
  return "No" ;
}) ;
print(lengthElement) ;
```

ارجاع القيم على هيئة Iterable

الشرط

```
BLEMS  OUTPUT  DEBUG CONSOLE
MINAL
idows PowerShell
yright (C) Microsoft Corporation. All rights reserved.

the new cross-platform PowerShell https://aka.ms/pscore6

DEBUG CONSOLE
(Yes Wael Exist, No, No)
Exited
```

خرج البرنامج على هيئة Iterable

العملية نفسها تطبق على Map نريد استخراج قيم key من Map سيقوم البرنامج بأرجاعها على هيئة Iterable نقوم بكتابة امر تحويل الى List لان المتغير الذي نقوم بتخزين القيم داخله هو من نوع list في حال لم نقم بكتابة امر تحويل سيقوم البرنامج بأظهار رسالة خطأ.

```
Map info = {"name" : "wael" , "age" : 12 , "phone" : 243531} ;

List infokey  = info.entries.map((e) => e.key).toList() ;

print(infokey) ;


}
```

Key Value

ارجاع قيمة Key

```
ROBLEMS  OUTPUT  DEBUG CONSOLE
ERMINAL
indows PowerShell
opyright (C) Microsoft Corporation. All rights reserved.

DEBUG CONSOLE
[name, age, phone]
Exited
```

خرج البرنامج

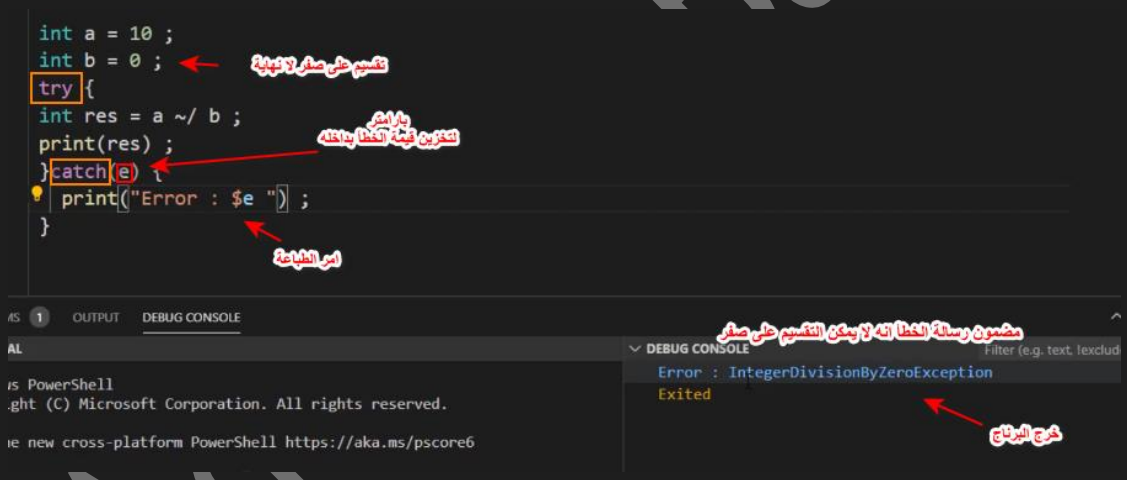
AboBakr

Try And Catch

معناها ان يكون لدينا كود من المتوقع حدوث خطأ فيه نضعه داخل try

مثال على هذا لدينا المثال في الأسفل نريد تقسيم عدد صحيح على صفر بلتأكيد سيقوم بأرجاع خطأ ويتوقف البرنامج عن العمل لان التقسيم على صفر لا نهاية.

نقوم بوضع عملية القسمة وطباعتها ضمن try ومن ثم بكتابة Catch ونضع فيه بارامتر لنقوم بتخزين قيمة الخطأ فيه ثم نقوم بطباعة هذا البارامتر كما هو واضح لدينا في المثال في الأسفل :



Aka Baker

shorthand if

الدالة الشرطية (المختصرة) : تقوم بنفس عمل الدالة الشرطسة تماما ولكن تختلف عنها بطريقة الكتابة انها مختصرة جدا كما هو واضح في المثال في الأسفل :



وبلتأكيد يكننا كتابة اكثر من شرط كما هو واضح في المثال في الأسفل :



OPP

برمجة كائنية التوجه : هي مخطط يساعدنا على تنظيم العمل.

عندما نقوم بتعريف كلاس من ضروري ان نقوم بتكابة اول حرف كبير من اسم الكلاس.

ماذا يمكننا كتابة داخل كلاس :

Varible

method

Constructors

Set & Get

يمكننا عمل نموذج من الكلاس وكما سبق وذكرنا ان الكلاس هو عبارة عن مخطط عمل يجب الالتزام بكل ما يوجد فيه كما هو واضح :



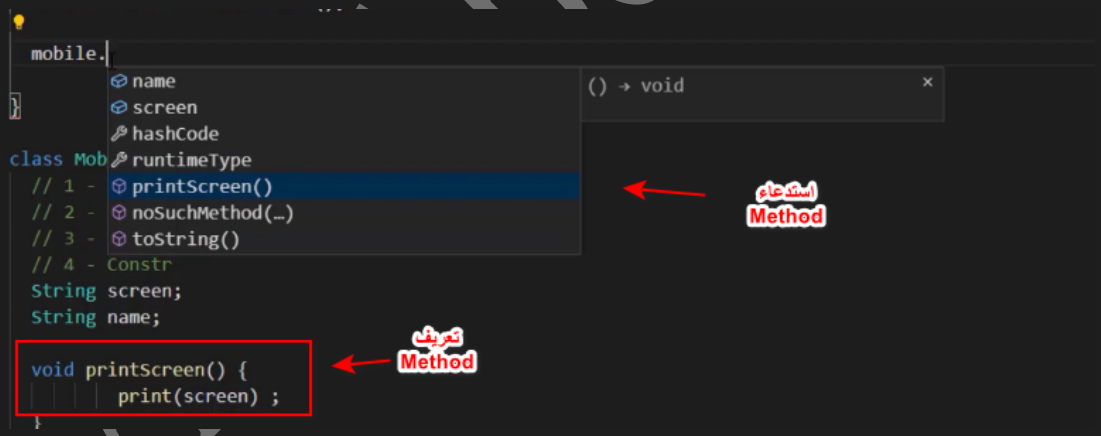
Abdel AbuBaker

يمكننا أيضا تعيين قيمة او إعادة تعيين قيمة للمتغير من خلال النموذج كما هو واضح في الصورة في الأسفل :



عندما نريد الوصول الى مكونات الكلاس يجب بناء نموذج عنه حت نستطيع الولوج اليه.

نفس الامر لل method يمكننا كتابتها في كلاس واستدعائها في نموذج كما هو واضح في الصورة في الأسفل :



hoBak

Reset Variable

طرق إعادة تتعين قيمة للمتغير ضمن الكلاس.

يوجد عدة طرق لاعادة تتعين :

User.name = " value"

cascade opreator

Set & Get

Constractor

الطريقة الأولى هي عن طرق كتابة اسم النموذج ثم كتابة اسم المتغير المعرف مسبقا في الكلاس ثم اسناد قيمة له كما هو واضح في الصورة في الأسفل :

```
class Mobile {
  // 1 - var
  // 2 - method
  // 3 - set And Get
  // 4 - Constr

  String name ;
}
```

تعريف متغير

نموذج عن كلاس

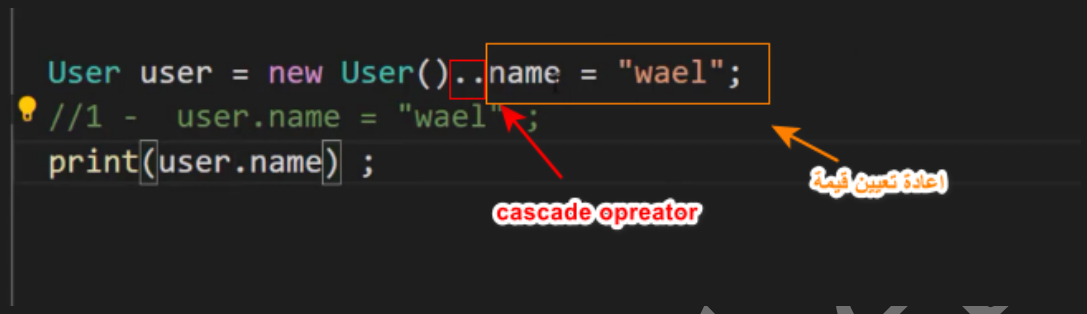
```
Mobile mobile = new Mobile();
mobile.name = "wael" ;
print(mobile.name) ;
```

اعادة تعيين قيمة

MohBaker

الطريقة الثانية عن طريق cascade opreator

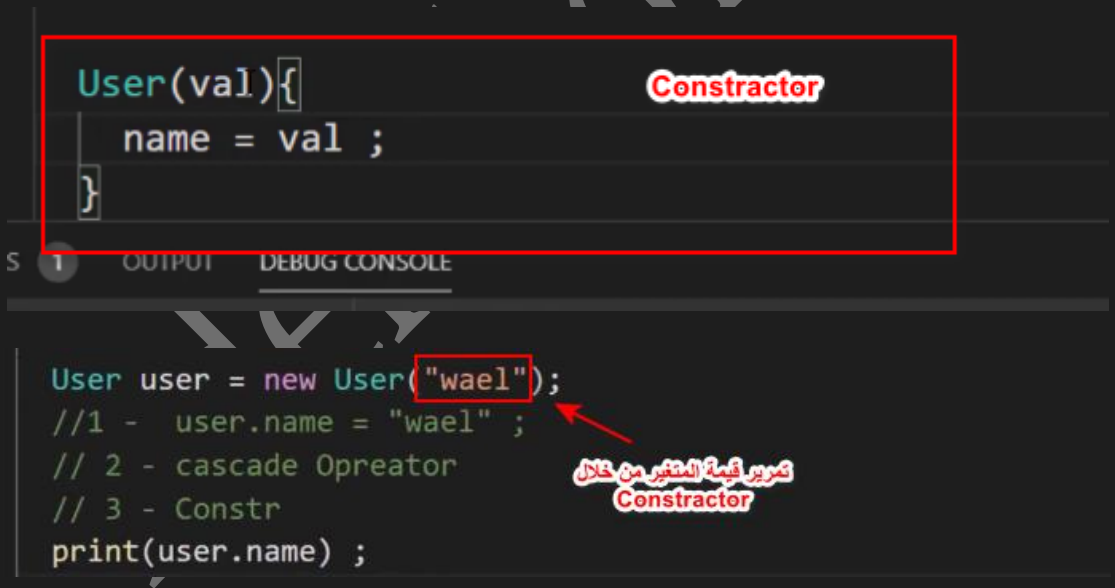
بكتابة نقطتين قبل نموذج الكلاس ثم اسم المتغير ثم اسناد القيمة الجديدة له كما هو واضح في الصورة في الأسفل :



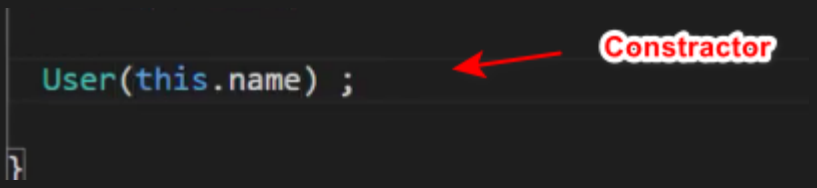
الطريقة الثالثة وهي Constractor

نستطيع إعادة تعيين قيمة متغير من تمرير قيمته من خلال Constractor

كما هو واضح في الصورة في الأسفل :



يوجد طريقة مختصرة لاعادة التعيين عن طريق Constractor عن طريق this يمكن استخدامها مع method أيضا وتكون عائدة على الكلاس الموجودة ضمنها (او انها تشير الى الكلاس الموجود فيه حاليا) كما هو واضح في الصورة في الأسفل :



Adel AboBaker

Setter And Getter

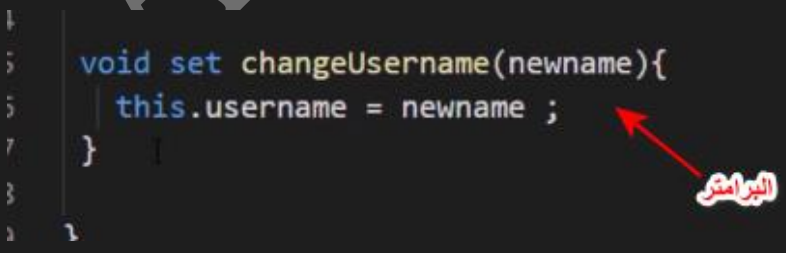
Set & Get هي طريقة الاسلم لاعادة تتعين قيمة متغير او جلب قيمة متغير من كلاس معين بلاضافة للميزات الإضافية التي تتمتع بها.

Set لاعادة تعين قيمة متغير من داخل الكلاس.

كما هو واضح في المثال في الأسفل قمنا بأعادة تعيين قيمة المتغير من خلال Set



قمنا قيمل method من نوع void لا ترجع قيمة وسميناها changeUsername وقمنا بأدخال بارمتر داخلها اسمينها newname لتخزين قيمة المتغير داخله ثم قمنا بتحديد المتغير الموجود ضمن كلاس عن طريق this كما هو واضح في المثال في الأسفل :



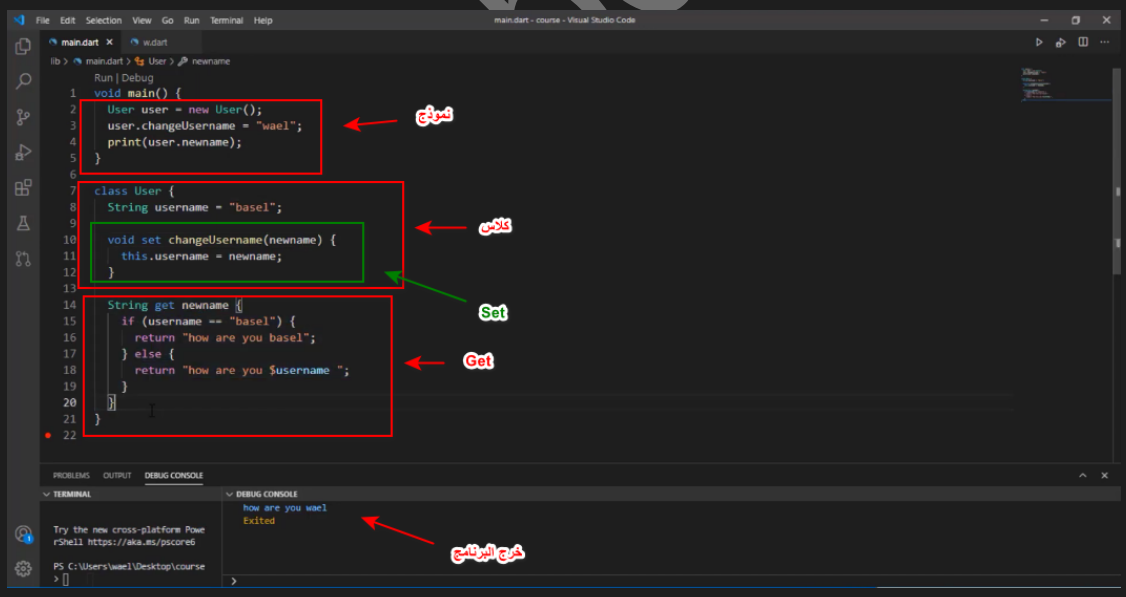
Get لجلب قيمة متغير ما من داخل الكلاس.

```dart
String get newname{
  return username ; |
}
```

نستطيع استخدام method مع Get كمت هو واضح في المثال في الأسفل :

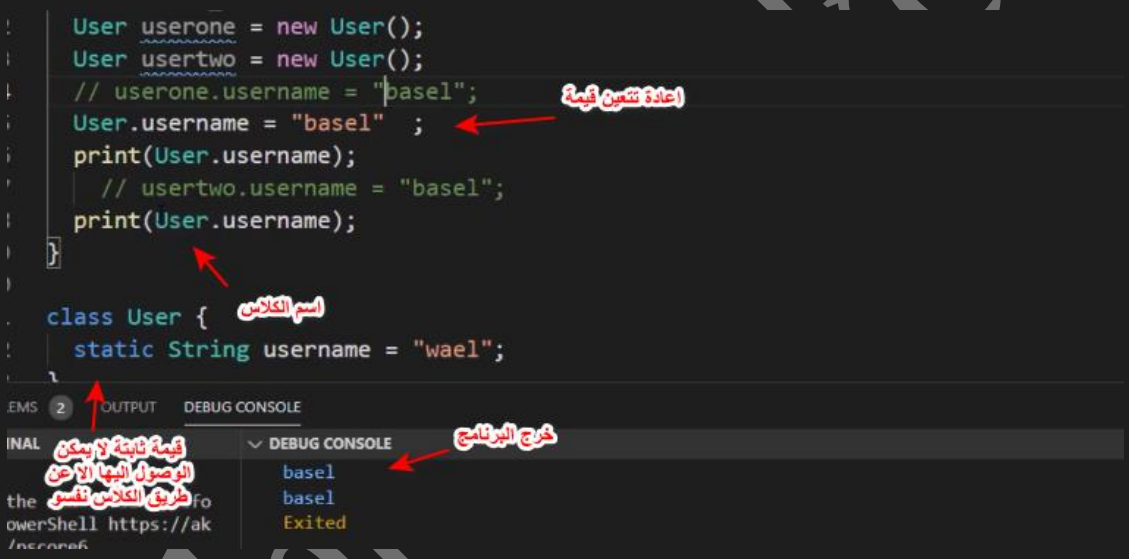
```dart
String get newname{
  if (username == "basel"){
    return  "how are you basel" ;
  }else {
    return "how are you $username " ;
  }
}
```





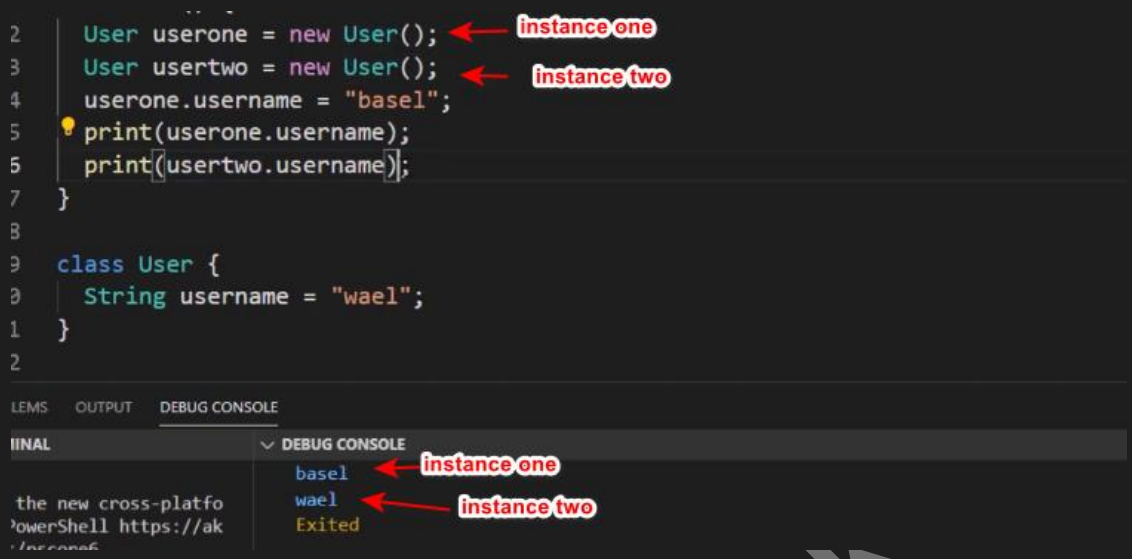
Static

هي قيمة ثابتة لا يمكن الوصول اليها الا من خلال الكلاس نفسو لا نستطيع الوصول اليها ع طريق instance النموذج يتم ذلك عن طريق طتابة اسم الكلاس ثم تنفيذ الامر المطلوب سواء كان إعادة تعيين قيمة او امر طباعة كام هو واضح في الافمثال يف الأسفل :

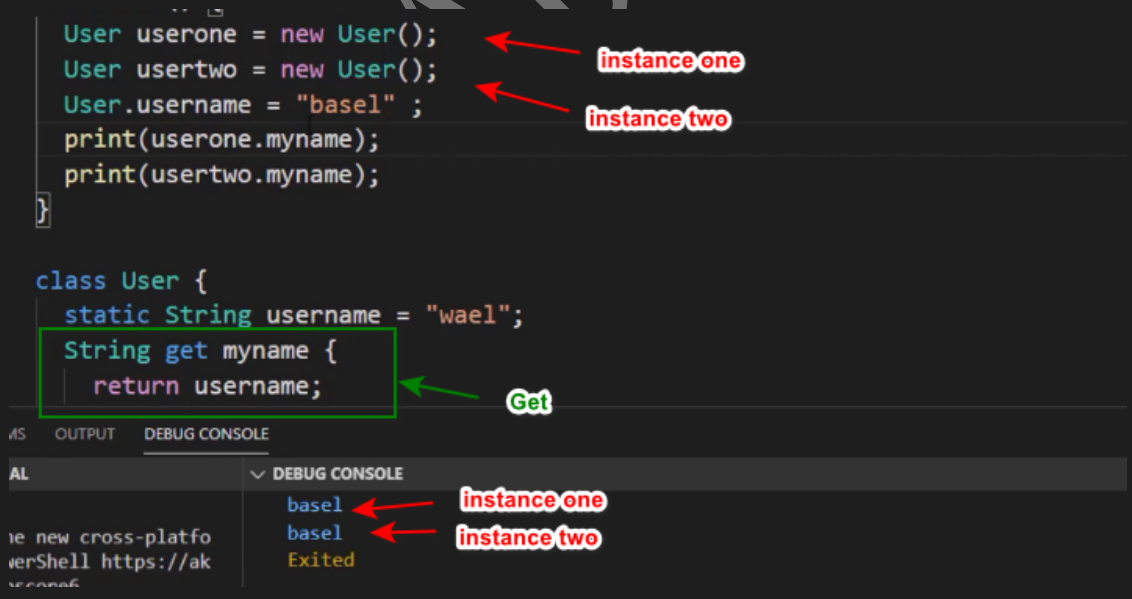


وكمل نعلم انه من الممكن انشاء عدة instance وكل واحد منهم نستطيع إعطائه قيم مختلفة عن الاخر ولكن في حال وجود Static سيتشارك جميع instance نفس قيمة المتغير الموجود ضمن الكلاس.

كما هو واضح في المثال في الأسفل :



نستطيع الوصول لقيمة Static من خلال instance عن طريق Get مهما قمنا بتعريف instance سيقوم بأخذ القيمة مشتركة من متغير username كما هو واضح في المثال في الأسفل :



Cascade operator

عادة عندما نقوم بأستدعاء method معينة من داخل كلاس معين فأننا نقوم بتعريف instance ثم كتابة اسم instance ثم كتابة اسم الميثود المراد استدعائها من الكلاس كما هو واضح في المثال في الأسفل :

```
3   Mobile mob = new Mobile() ;
4   mob.testone() ;
5   mob.testtwo() ;
6
7  }
8  class Mobile {
```

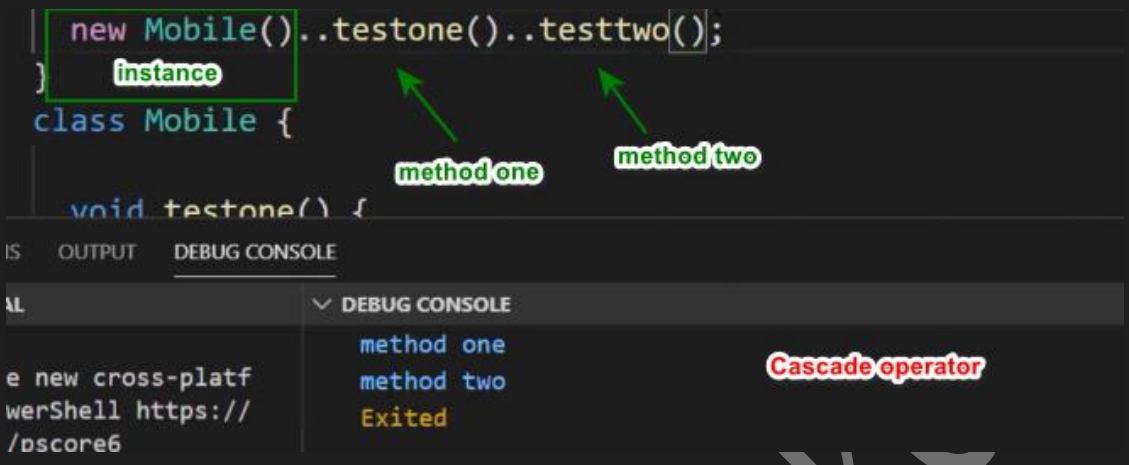
instance

```
BLEMS   OUTPUT   DEBUG CONSOLE
MINAL                DEBUG CONSOLE
                       method one
the new cross-platf    method two
PowerShell https://    Exited
.ms/pscore6
```

خرج البرنامج

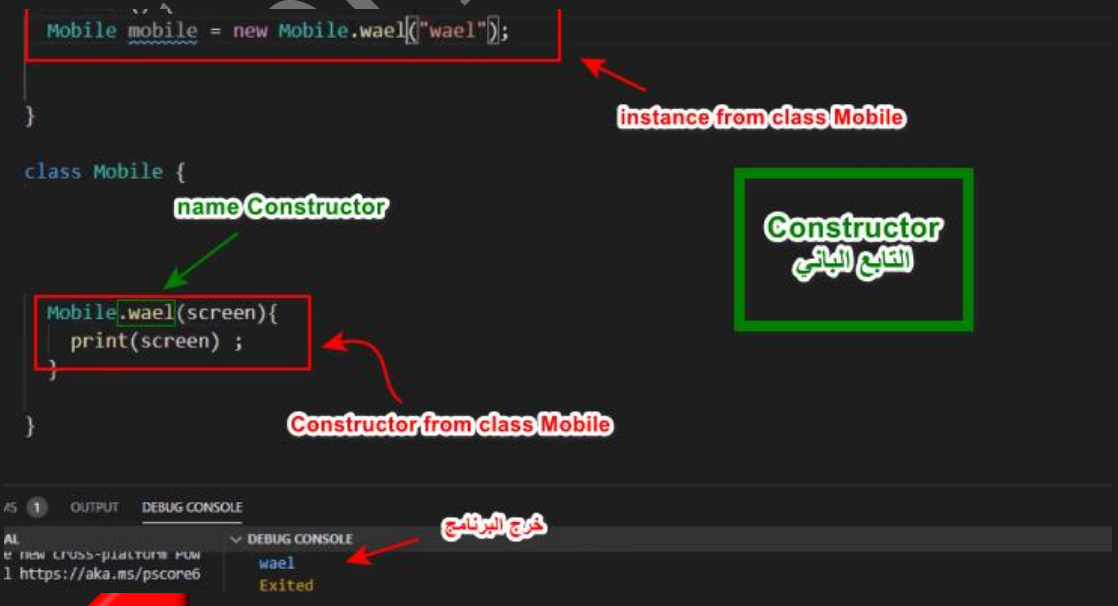
```
void testone() {
  print("method one") ;
}
 void testtwo() {
  print("method two") ;
}
```

ولكن يوجد طريقة مختصر من خلالها نستطيع استدعاء method من كلاس معين عن طريق cascade oprator كما هو واضح في المثال في الأسفل :



Constructor

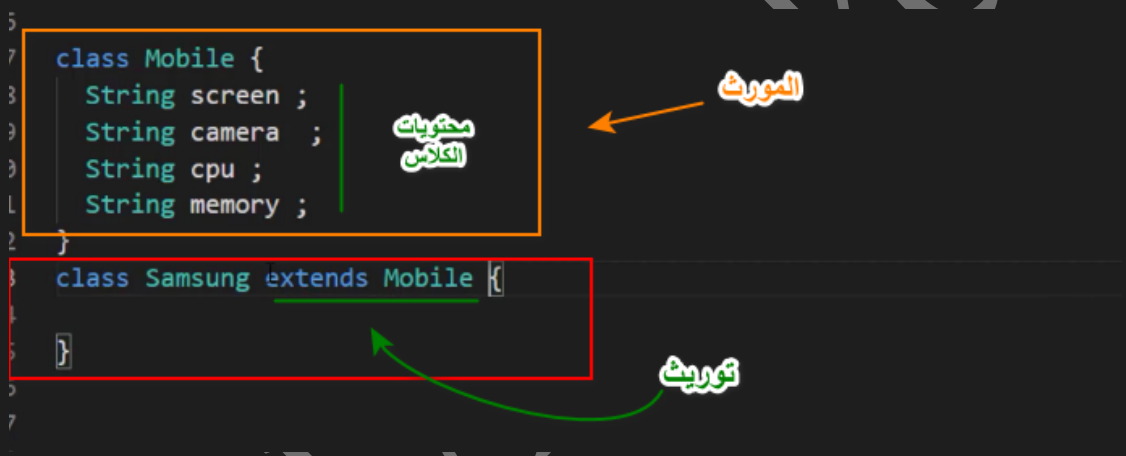
او ما يسمى التابع الباني : يتم استدعائه مباشرة (بشكل تلقائي) عند تعريف instance من كلاس معينة يتم تعريف التابع الباني بكتابة اسم الكلاس ثم فتح قوسين وإدخال البرمترات كما هو واضح لدينا في المثال في الأسفل :



inheritance

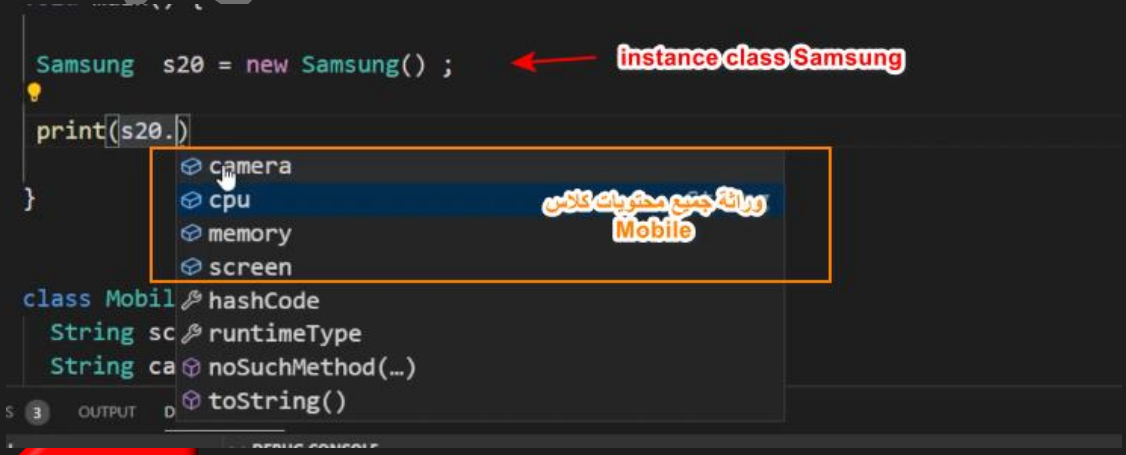
الوراثة : هو ان يتم توريث جميع محتويات كلاس معين لكلاس اخر .

ليكن لدينا كلاس mobile يحوي على عناصر أساسية مثل cpu و camera و memory و screen وقنا بعمل كلاس اخر سميناه Samsung واردنا توريث جميع محتويات كلاس mobile الى كلاس Samsung كما هو واضح لدينا في المثال في الأسفل :



نلاحظ انه تم توريث جميع محتويات كلاس mobile لكلاس Samsung

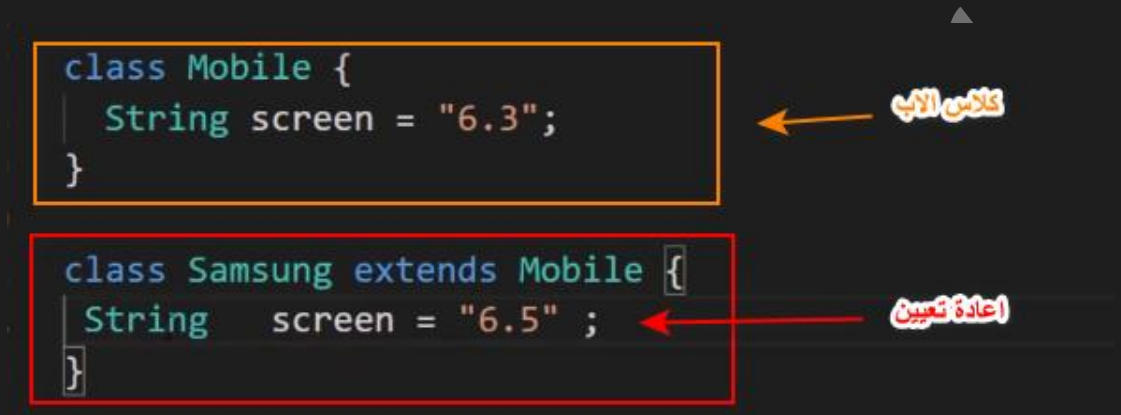
كما هو واضح من class Samsung instance كما هو واضح لدينا في الصورة :



Override

المقصود منها هي إعادة تعيين لمتغير او لميثود معينة داخل كلاس.

ويف حال اردنا إعادة تعيين لقيمة متغير ما يجب الالتزم بنوع المتغير في حال كان نفس الاسم كما هو واضح لدينا في الأسفل :



في حال قمنا بتغير نوع المتغير لن يعمل البرنامج وسيقوم بأظهار رسالة خطأ كما هو واضح لدينا في المثال في الأسفل :

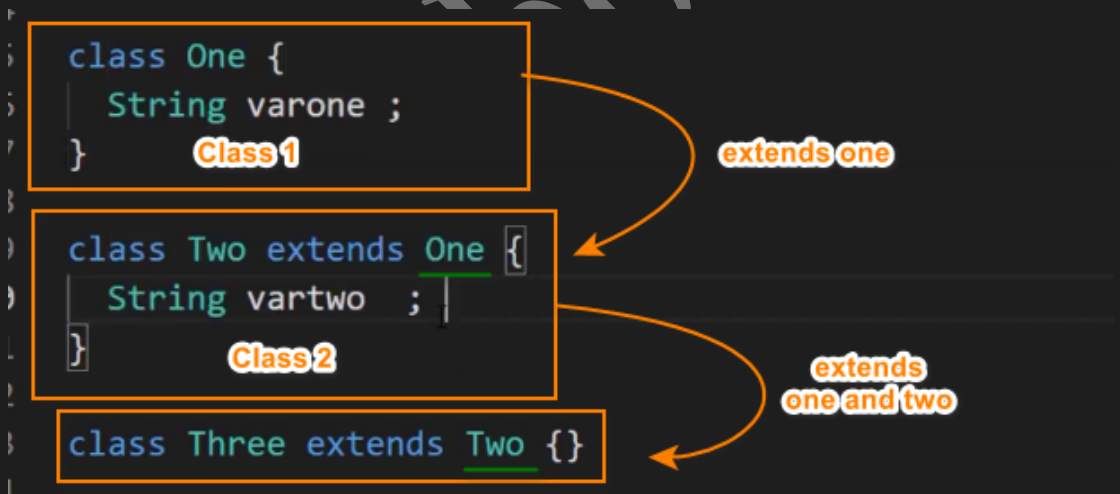
```
7   class Mobile {
8       String screen = "6.3";
9   }
10
11  class Samsung extends Mobile {
12      double screen = "6.5" ;
13  }
```

Mult-level inherate

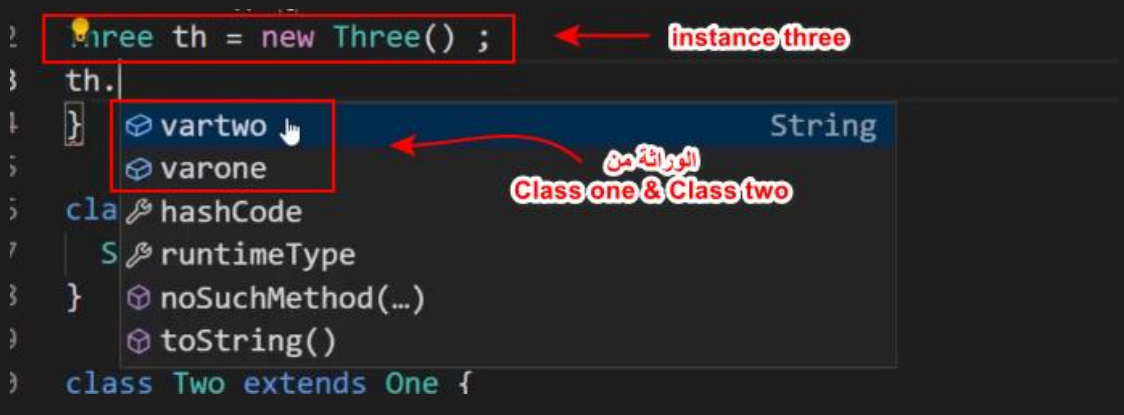
مستويات الوراثة : تعرفنا في الدروس السابقة ان وراثة هي وراثة كلاس من كلاس اخر ولكن الوراثة لها عدة مستويات.

في هذا المثال قمنا بتعريف ثلاث كلاسات وقمنا بتعريف متغييرين في الكلاس one و two وقمنا بتوريث class one ل class two و class two الى class three

كما هو واضح في الصورة في الأسفل :



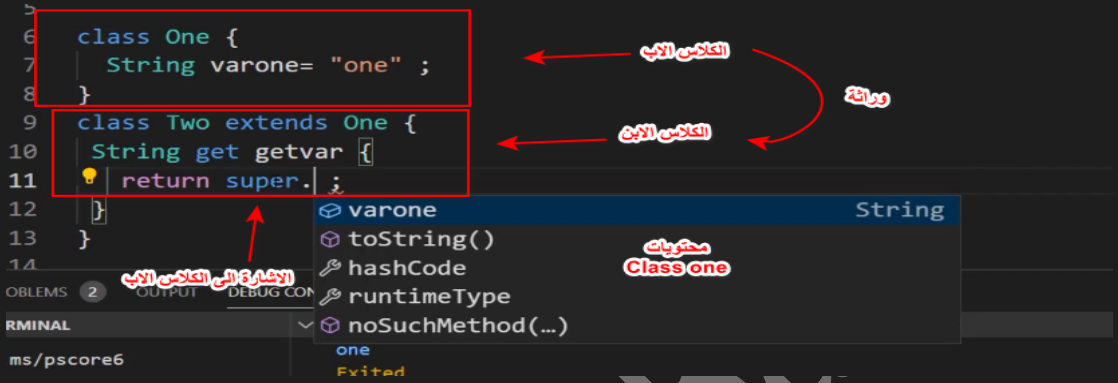
فبذلك فأن class three ورث جميع محتويات class one و class two :



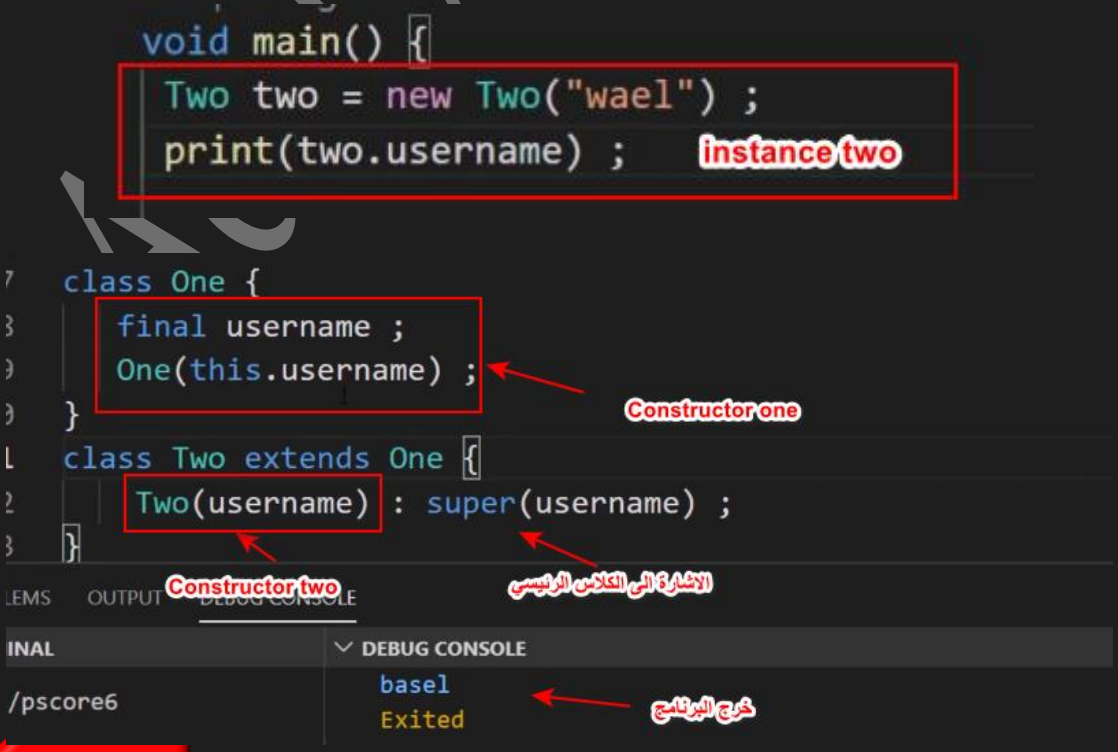
Adel AboBaker

Super Class

Super تعني الإشارة الى الكلاس الرئيسي (الاب) كما هو واضح لدينا في المثال في الأسفل :



اذا كان الكلاس الرئيسي (الاب) يحوي على Constructor فيجب عندها ان يكون الكلاس الابن يحوي على Constructor وفي حال كنا نستخدم نفس الاسم يجب كتابة : ثم super("name Constructor") كما هو واضح لدينا في الامثال في الأسفل :



Abstract class

ببساطة مهمته فقط التوريث ولا يمكن عمل instance منه

عند محاولتنا تعريف instance من class Mobile كما هو واضح لدينا في المثال في الأسفل :

```
2   Mobile mob = new Mobile() ;
3   print(mob.brand) ;
4  }
5
6  abstract class Mobile {
7    String brand = "samsung" ;
8  }
9
```

instance class

abstract class

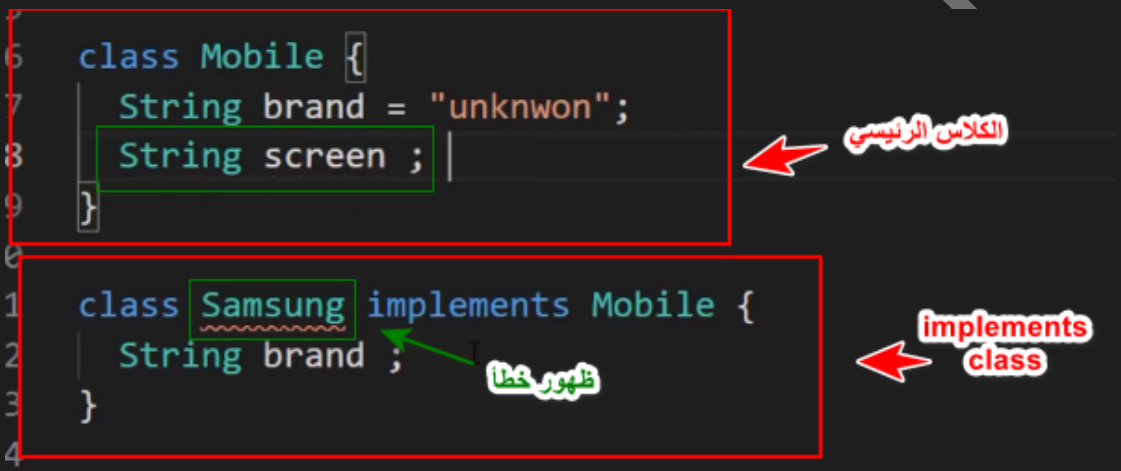
```
5
6  abstract class Mobile {
7      String    brand = "unknwon" ;
8  }
9  class Samsung extends Mobile {
0      String    brand = "samsung" ;
1  }
2
```

abstract class

كلاس وريث

implements class

هو الذي يجبر الكلاس الابن ان يرث جميع خواص الكلاس الاب بمعنى أي ان جميع الخواص الموجودة في الكلاس الرئيسي (الاب) يجب استعمالها في الكلاس الابن كما هو واضح في المثال في الأسفل ظهور خطأ لعدم وراثة الكلاس الابن (جميع) خواص الكلاس الرئيسي :



يتم تعيين القيم في الكلاس الابن أي ان implements تمنع اخذ القيم من الكلاس الرئيسي (الاب).

أي انه عند كتابة الكلاس الرئيسي لا داعي من تعيين القيم.



لا مشكلة من إضافة Properties الى الكلاس الابن في حال وجود implements ولكن في حال إضافة Properties جديدة الى الكلاس الرئيسي يجب اضافتها الى الكلاس الابن لتفادي حدوث خطأ في البرنامج.

```
class Mobile {
  String brand = "unknwon";
  String screen ;
}

class Samsung implements Mobile {
  String brand ;
  String screen ;
  String t ;
}
```


يجب الانتباه لاسماء المتغيرات لان implements حساس للمحارف.

```
class Mobile {
  String brand = "unknwon";
  String screen ;
}

class Samsung implements Mobile {
  String brand ;
  String screenw ;
}
```

Collection

بلدروس السابقة اذا اردنا تعريف list نقوم بكتابة نوع + اسم list الان بعد التعرف على OPP نستطيع تعريف list او set او map بصيغة instance كما هو واضح لدينا في المثال في الأسفل :

```
void main() {
  List names = new List() ;
  names.add("wael") ;
  names.add("basel") ;
  print(names) ;
}
```

تعريف مصفوفة فارغة

اضافة قيم للمصفوفة

```
LEMS   OUTPUT   DEBUG CONSOLE
INAL
lows PowerShell
right (C) Microsoft Corporation. All rights r
DEBUG CONSOLE
[wael, basel]
Exited
```

خرج البرنامج

نستطيع كتابة instance بدون استدعاء اسم الكلاس وسيتم تعرف اخذ النوع تلقائيا على حسب قيمة ولا يشترط كتابة new عند تعريف instance من كلاس معين كما هو واضح لدينا في المثال في الأسفل :

```
var  mob =  Mobile() ;
```

class Mobile

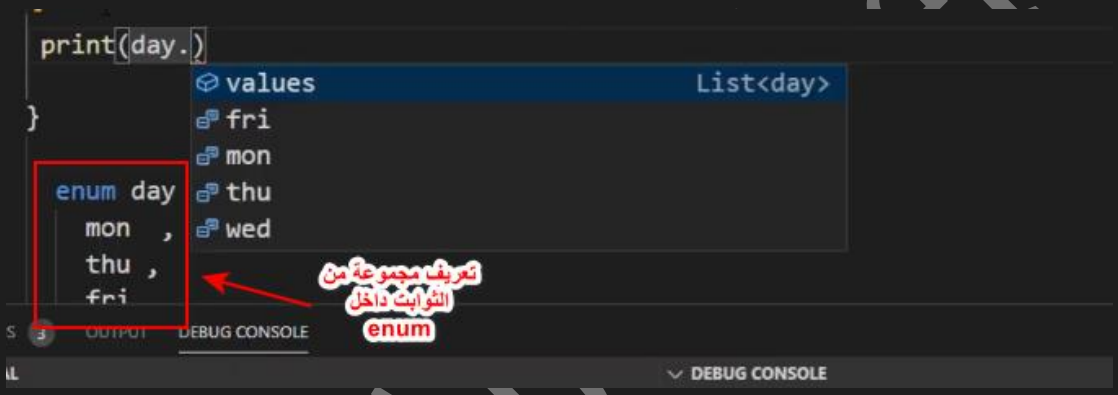
new

instance

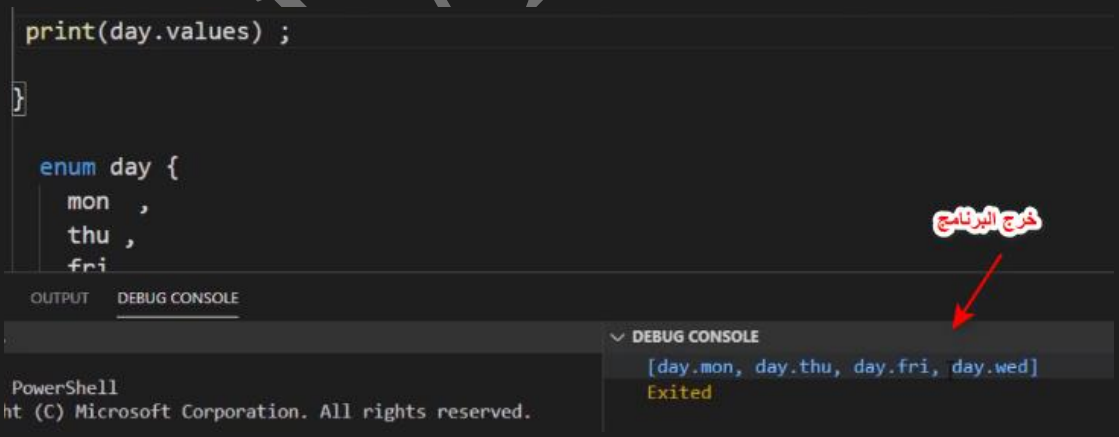
Enum

المقصود فيها نستطيع تعريف مجموعة من ثوابت في برنامجنا نستطيع استخدامها في برنامجنا بشكل دائم وثابت.

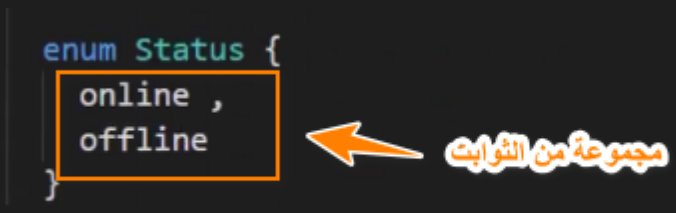
قمنا بتعريف أيام الأسبوع داخل enum كما هو واضح لدينا في المثال في الأسفل :



عند طباعة values سيكون الخرج بلشكل التالي :

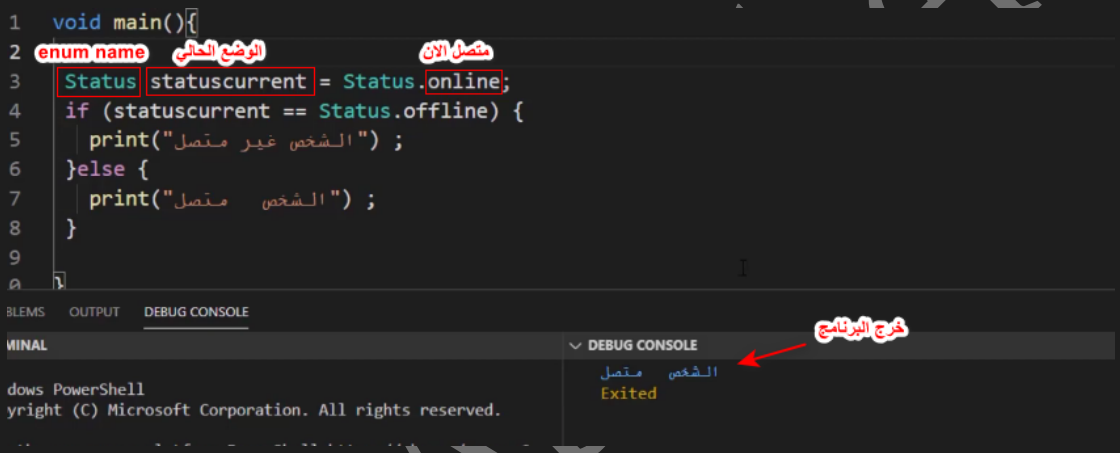


مثال اخر عن enum قمنا بتعريف الثوابت التالية :



واردنا كتابة برنامج لمعرفة الشخص اذا كان متصل الان او غير متصل الان

(المثال اقرب لتطبيق الدردشة واتس اب)



Adel Ashraf

Regular expressions

مهمته التحقق من المدخلات وترجع قيمة اما true او false.

regular expressions هو بحد ذاته علم ليس مطلوب منك انت كمبرمج ان تتعمق فيه انما تقوم بجلب ال pattern جاهزة من على الانترنت في الأمثلة القادمة ستعلم ما معنى هذا المصطلح..!!

على سبيل المثال عند تسجيل دخول الى حسابك على الفيسبوك يطلب منك الموقع ادخال اسم المستخدم وكلمة المرور مثلا..!!

وعندها نتوجه للحقل البريد الالكتروني وقمنا بأدخال معطيات خطأ مثلا قمنا كتابة كلمة (ahmad) قمنا بأدخال مجموعة من الأرقام او الإشارات عند النقر على الزر تسجيل الدخول سيقوم الموقع بأظهار رسالة خطأ مفادها ان (البريد الالكتروني غير صالح).

من هنا نطرح سؤال كيف عرف الموقع ان البريد غير صالح ..؟؟

لكتابة البريد الالكتروني لابد من مجموعة من القواعد و الخصائص التي يحتويها ليكون بريد الكتروني نقوم ببرمجتها ليقوم الحاسوب بتعرف عليه على انه بريد الكتروني كما هو واضح لدينا في المثال يف الأسفل :



Adel AboBaker

ومن هنا تأتي أهمية regular expressions وهي التحقق من المدخلات وسنقوم بطرح مثال عنها بشكل مفصل.

لدينا المثال الذي بأسفل نريد التحقق من ان المدخل فعلا هو بريد الكتروني

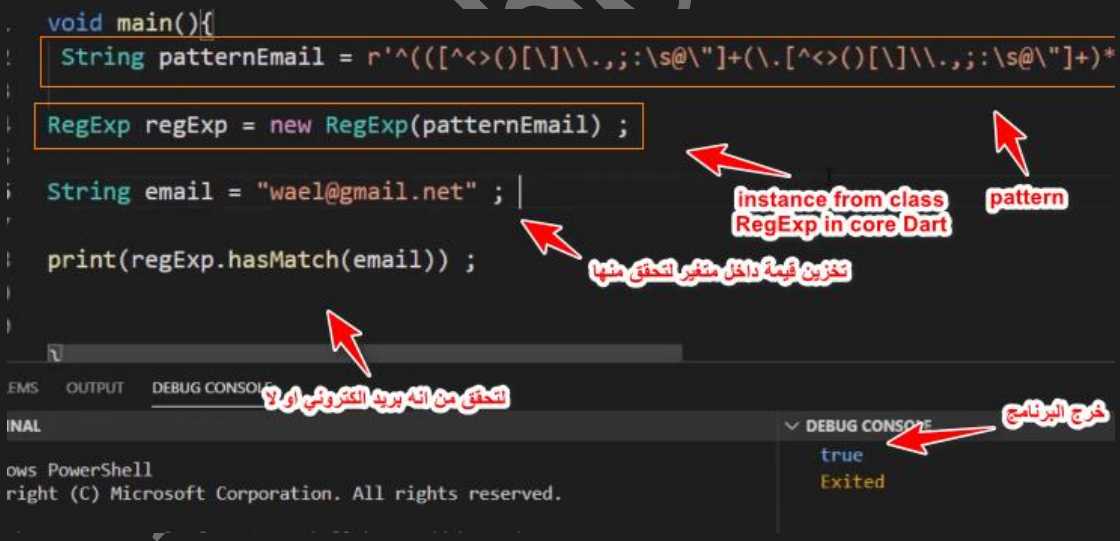
--الخطوة الأولى قمنا بجلب patternEmail من الانترنت جاهزة.

--الخطو الثانية قمنا بأستدعاء instance من كلاس RegExp في لغة dart.

--الخطوة الثالثة قمنا بتعريف متغير لتخزين القيمة داخله لتحقق منها.

--الخطوة الأخير قمنا بكتابة امر الطباعة داخل instance RegExp مع hasMatch مهمتها التحقق من المعطيات.

وعند تشغيل البرنامج قام البرنامج بأرجاع قيمة true لان البريد الالكتروني جميع عناصره متوفر فيه كما هو واضح لدينا في المثال في الأسفل :



TechBaker

لو قمنا بأدخال أي قيمة بشكل عشوائي وتحققنا منها سيقوم البرنامج بأرجاع قيمة false كما هو واضح لدينا في الأسفل :

```
String email = "wael" ;

print(regExp.hasMatch(email)) ;

}
```

ادخال قيمة لفحصها

خرج البرنامج

```
OUTPUT   DEBUG CONSOLE
DEBUG CONSOLE
false
Exited
```

مثال اخر لتحقق من ادخال رقم كما هو واضح لدينا في الأسفل :

```
String patternPhone = r'(^(?:[+0]9)?[0-9]{10,12}$)';

RegExp regExp = new RegExp(patternPhone) ;

String phone = "+96394223545" ;

print(regExp.hasMatch(phone)) ;
```

patternPhone

instance from class RegExp

ادخال رقم لتحقق منه

خرج البرنامج

```
OUTPUT   DEBUG CONSOLE
DEBUG CONSO
true
Exited
```

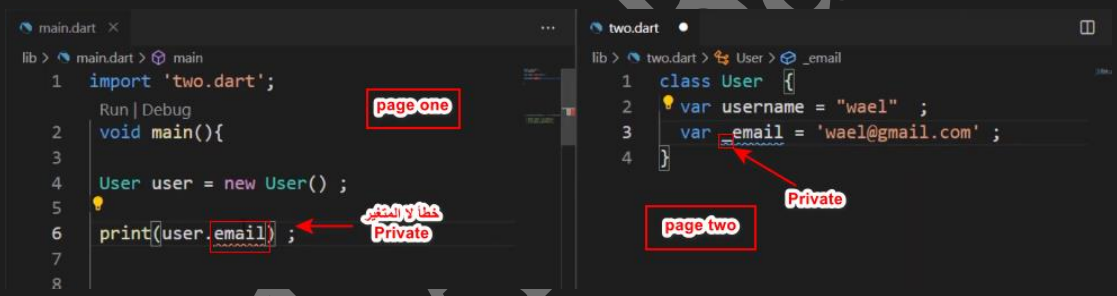
Auto Ra

Private

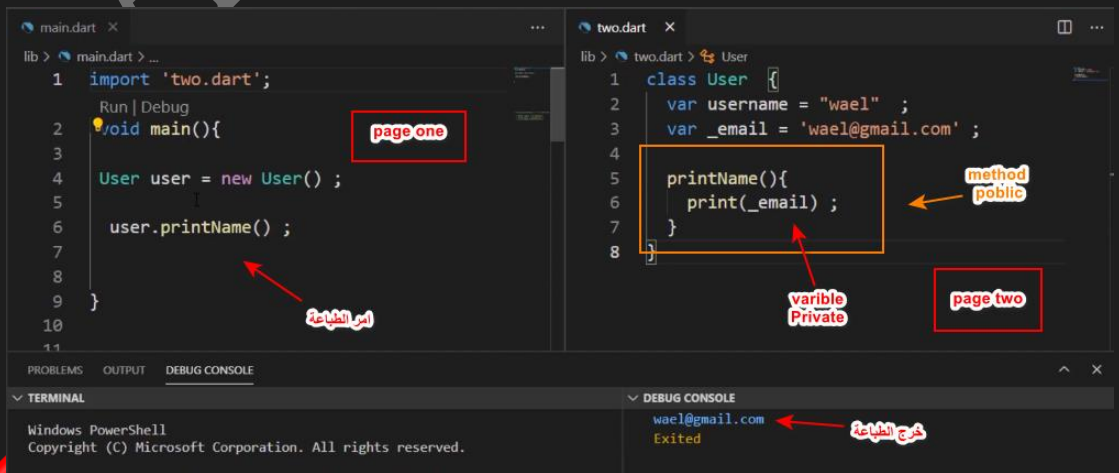
نقوم بأستخدامها داخل class من اجل ان تكون

Varible او method او Constructor خاص بلكلاس نفسو ولا يمكن استخدامها بي page أخرى عكس Private الخاص هو poblic العام.

قمنا بتعريف كلاس بصفحة وتعريف متغيرين بداخله وعمل المتغير emial Private من خلال وضع اشاش (_) قبل اسم المتغير واستدعاء instance من صفحة أخرى تنفيذ امر الطباعة للمتغير email سوف نلاحظ وجود خطأ كما هو واضح لدينا في الأسفل :

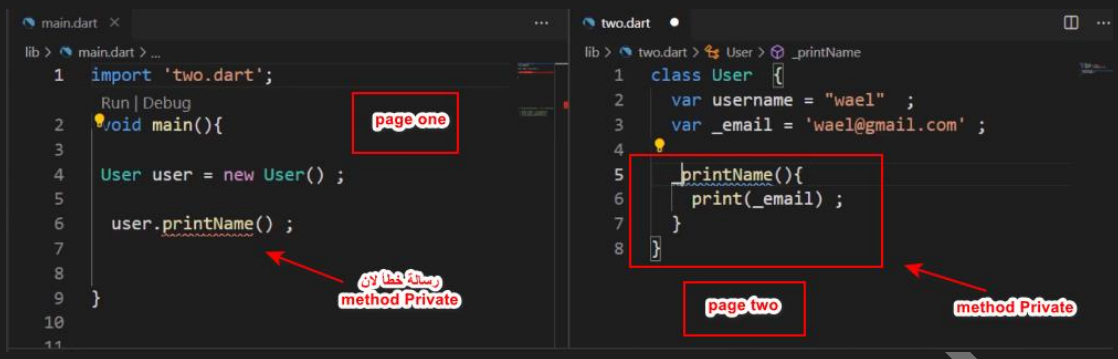


لو قمنا بتمرير المتغير الخاص Private داخل method poblic عامة سيقوم بتنفيذ امر الطباعة من دون أي مشاكل أي اننا نستطيع الوصول الوصول الى المتغير ال Private من خلال poplic method عامة كما هو واضح لدينا في المثال في الأسفل :

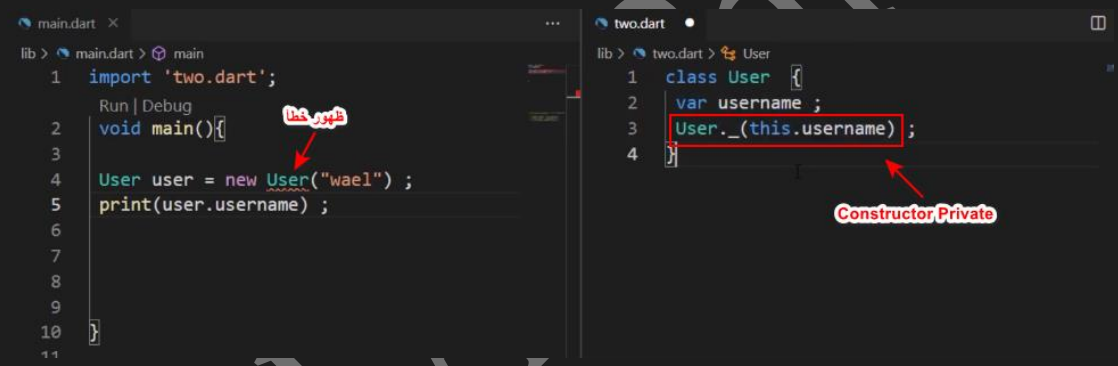


Adel Baker

الامر Private ينطبق تماما على method كما هو واضح لدينا في المثال في الأسفل :



وأيضا كما ذكرنا ان الامر ينطبق تماما على Constructor كما هو واضح لدينا في المثال في الأسفل :

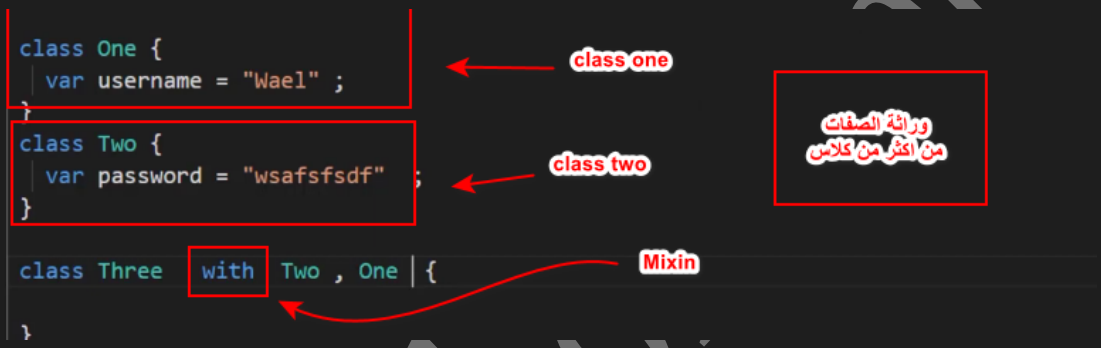


Adel Tarek

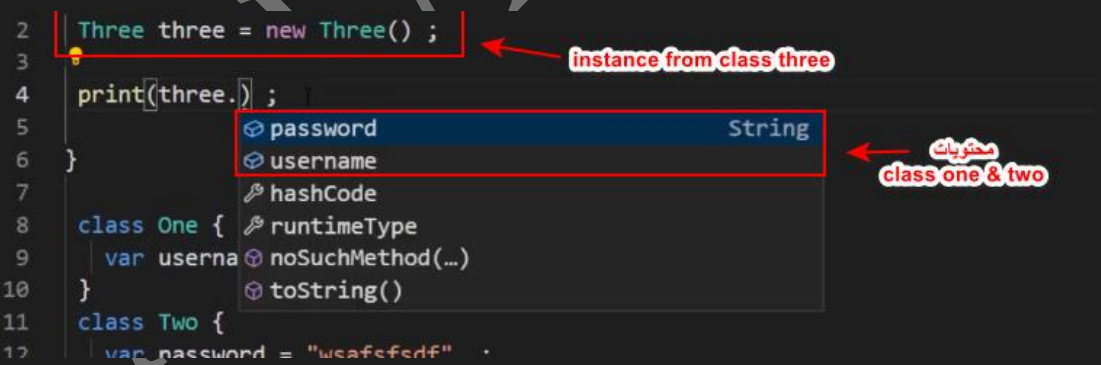
Mixin

في حال كان لدينا كلاس معين في برنامجنا واردنا ان نقوم بتوريثه من اكثر من كلاس يتحقق ذلك من خلال مفهوم mixin.

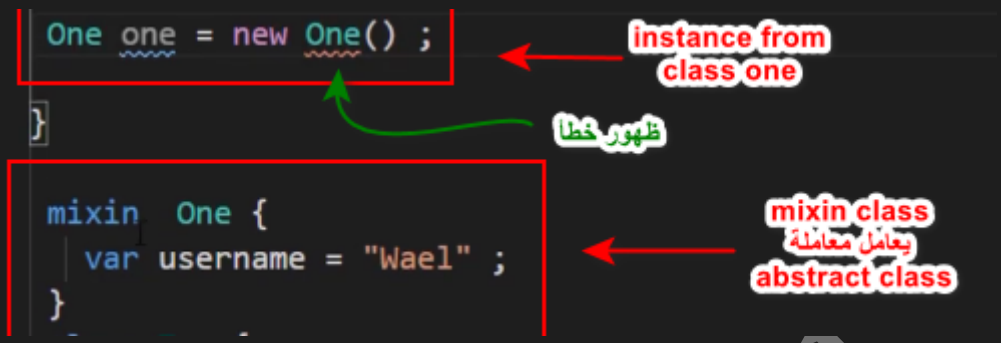
نستخدم with التي تعبر عن mixin كما هو واضح لدينا في المثال في الأسفل في class three :



وعند انشاء instance من class three وتنفيذ امر الطباعة منه نلاحظ ظهور محتويات class one & two كما هو واضح في المثال في الأسفل :



و mixin يعامل معاملة abstract class أي اننا لا نستطيع انشاء instance منه كما هو واضح لدينا في المثال في الأسفل :



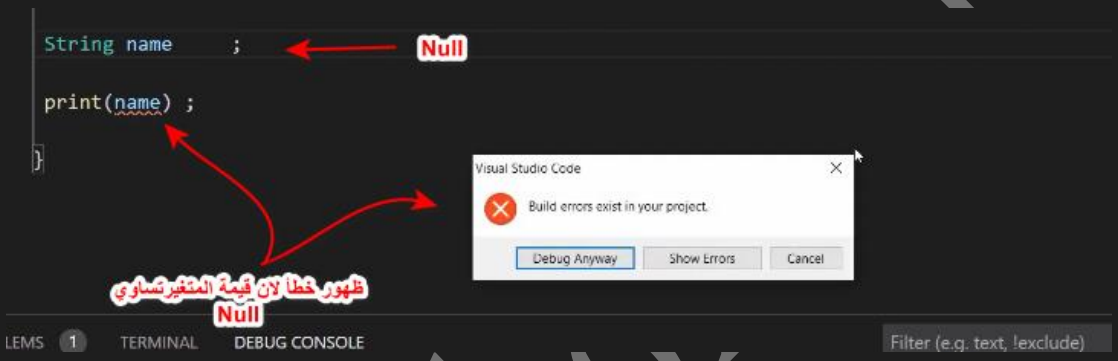
تستخدم mixin فقط مع with في حال استخدامها مع extends سيقوم البرنامج بأظهار خطأ كما هو واضح لدينا في المثال في الأسفل :



Adel Bakel

Null Safety

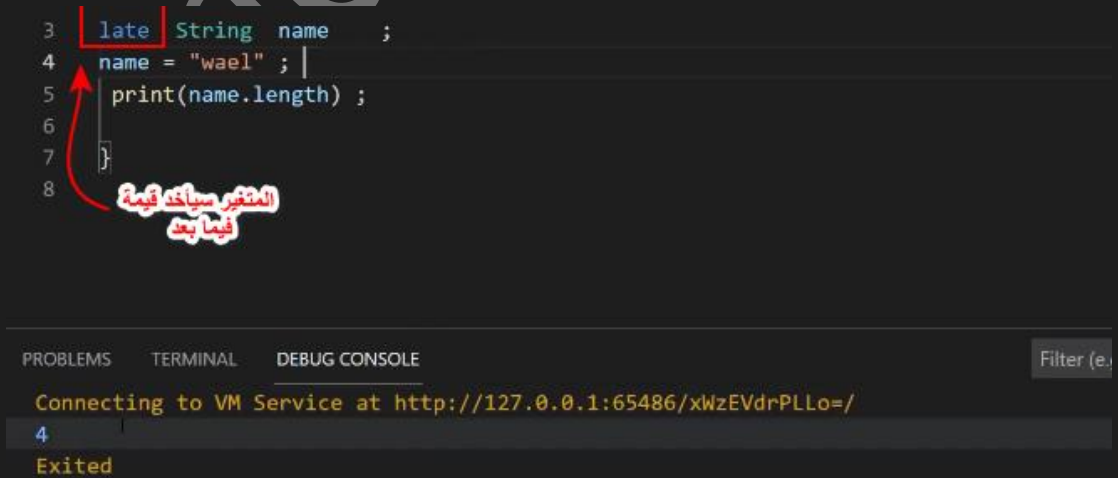
سابقا عند تعريف متغير من دون اسناد قيمة له عند تشغيل البرنامج يقوم الحاسوب بأرجاع قيمة Null من دون أي مشاكل الان بعد ان أصبحت لغة Dart تدعم Null Safety عند طباعة متغير قيمته تساوي null سيقوم البرنامج بأظهار رسالة خطأ كما هو واضح لدينا في المثال في الأسفل :



Null Safety تساعدنا على اكتشاف الخطأ قبل وقوعه وأيضا تقدم أداء أفضل في التطبيقات.

في اردنا ترك المتغير من دون قيمة يمكننا استخدام late او إشارة (?)

Late و question mark تعني أي يمكن تعيين قيمة للمتغير بوقت لاحق :



The END

تم جمع هذا المعلومات من كورس لغة Dart للمهندس وائل أبو حمزة من قناته على اليوتيوب